
Oracle Database 11g: SQL Fundamentals II

Electronic Presentation

D49994GC11
Edition 1.0
May 2009

ORACLE®

Author

Chaitanya Koratamaddi

Brian Pottle

Technical Contributors and Reviewers

Claire Bennett

Yanti Chang

Ken Cooper

Laszlo Czinkocski

Burt Demchick

Gerlinde Frenzen

Laura Garza

Joel Goodman

Richard Green

Nancy Greenberg

Akira Kinutani

Wendy Lo

Isabelle Marchand

Timothy Mcglue

Alan Paulson

Srinivas Putrevu

Bryan Roberts

Clinton Shaffer

Abhishek Singh

Jenny Tsai Smith

James Spiller

Lori Tritz

Lex van der Werff

Marcie Young

Editors

Nita Pavitran

Arijit Ghosh

Raj Kumar

Graphic Designer

Satish Bettgowda

Publisher

Syed Ali

Jayanthi Keshavamurthy

Copyright © 2009, Oracle. All rights reserved.

Disclaimer

This course provides an overview of features and enhancements planned in release 11g. It is intended solely to help you assess the business benefits of upgrading to 11g and to plan your IT projects.

This course in any form, including its course labs and printed matter, contains proprietary information that is the exclusive property of Oracle. This course and the information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This course and its contents are not part of your license agreement nor can they be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

This course is for informational purposes only and is intended solely to assist you in planning for the implementation and upgrade of the product features described. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remain at the sole discretion of Oracle.

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice. If you find any problems in the document, please report them in writing to: Oracle University, 500 Oracle Parkway, Redwood Shores, California 94065 USA. This document is not warranted to be error-free.

Restricted Rights Notice

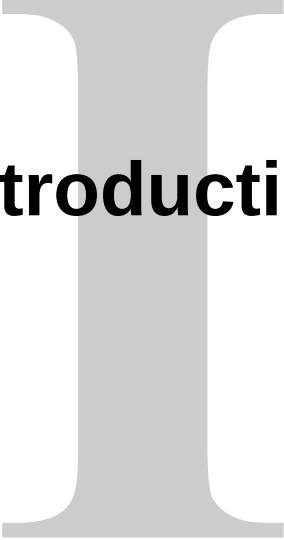
If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT RIGHTS

The U.S. Government's rights to use, modify, reproduce, release, perform, display, or disclose these training materials are restricted by the terms of the applicable Oracle license agreement and/or the applicable U.S. Government contract.

Trademark Notice

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.



Introduction

Lesson Objectives

After completing this lesson, you should be able to do the following:

- Discuss the goals of the course
- Describe the database schema and tables that are used in the course
- Identify the available environments that can be used in the course
- Review some of the basic concepts of SQL

Lesson Agenda

- **Course objectives and course agenda**
- The database schema and appendixes used in the course and the available development environment in this course
- Review of some basic concepts of SQL
- Oracle Database 11g documentation and additional resources

Course Objectives

After completing this course, you should be able to do the following:

- Control database access to specific objects
- Add new users with different levels of access privileges
- Manage schema objects
- Manage objects with data dictionary views
- Manipulate large data sets in the Oracle database by using subqueries
- Manage data in different time zones
- Write multiple-column subqueries
- Use scalar and correlated subqueries
- Use the regular expression support in SQL

Course Prerequisites

The *Oracle Database 11g: SQL Fundamentals I* course is a prerequisite for this course.

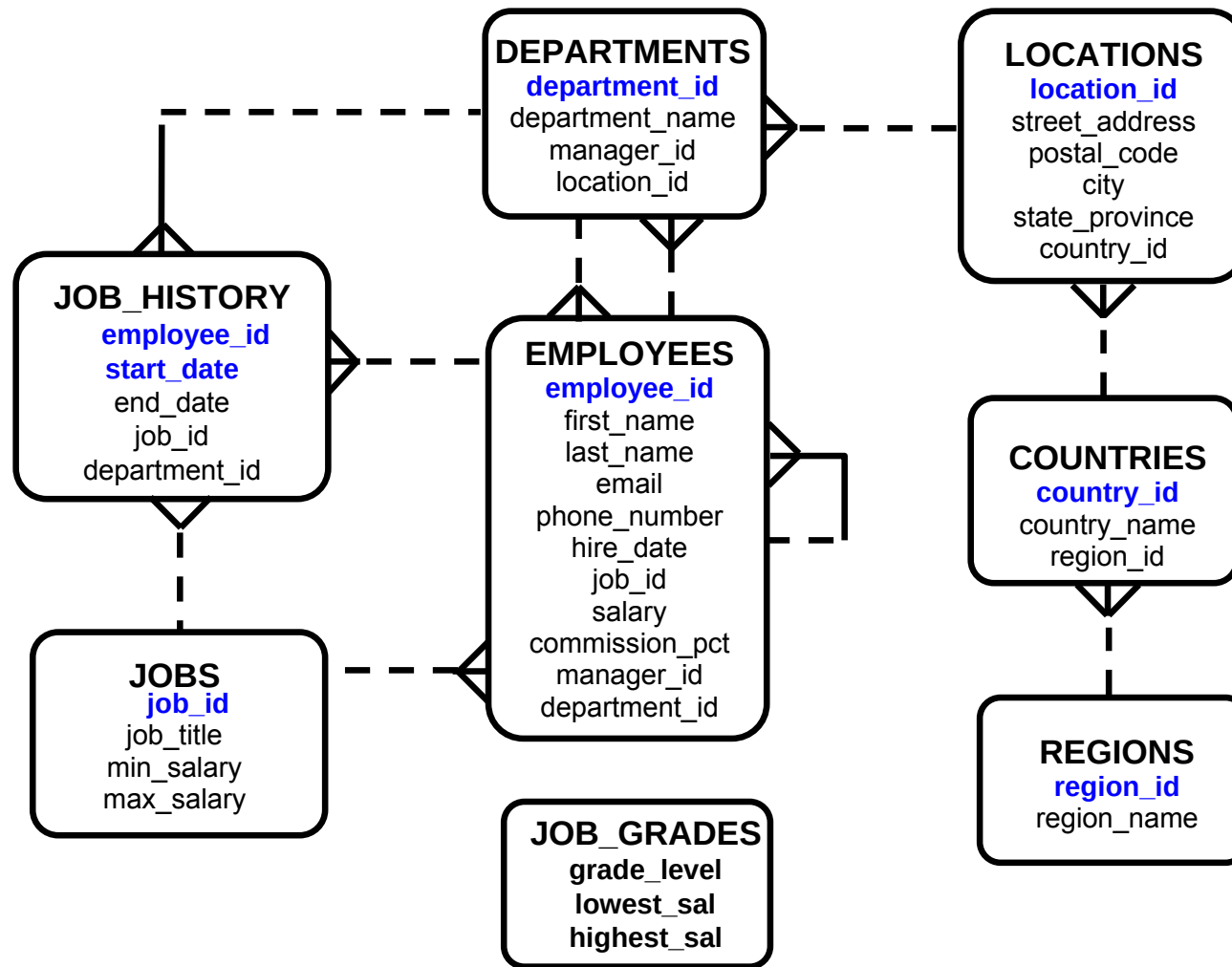
Course Agenda

- Day 1:
 - Introduction
 - Controlling User Access
 - Managing Schema Objects
 - Managing Objects with Data Dictionary Views
- Day 2:
 - Manipulating Large Data Sets
 - Managing Data in Different Time Zones
 - Retrieving Data Using Subqueries
 - Regular Expression Support

Lesson Agenda

- Course objectives and course agenda
- The database schema and appendixes used in the course and the available development environment in this course
- Review of some basic concepts of SQL
- Oracle Database 11g documentation and additional resources

Human Resources (HR) Schema Used in This Course



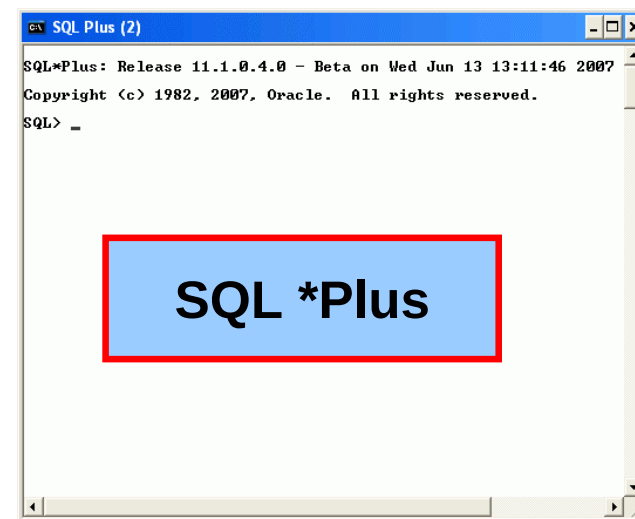
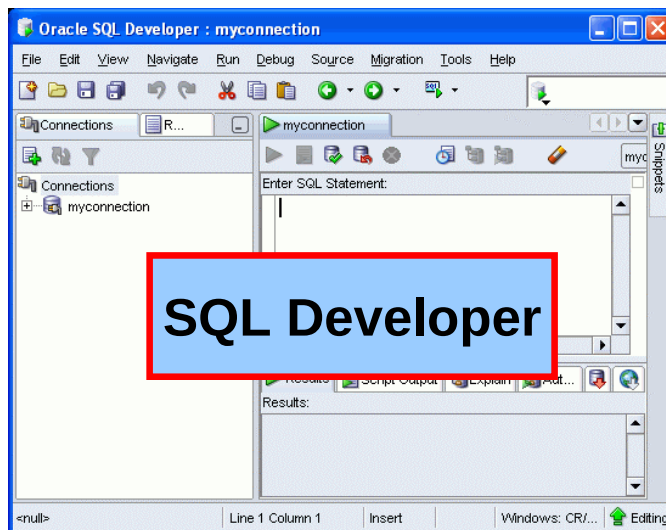
Appendixes Used in This Course

- Appendix A: Practice Solutions
- Appendix B: Table Descriptions
- Appendix C: Using SQL Developer
- Appendix D: Using SQL*Plus
- Appendix E: Generating Reports by Grouping Related Data
- Appendix F: Hierarchical Retrieval
- Appendix G: Writing Advanced Scripts
- Appendix H: Oracle Database Architectural Components

Development Environments

There are two development environments for this course:

- Primary tool is Oracle SQL Developer
- SQL*Plus command line interface may also be used



Lesson Agenda

- Course objectives and course agenda
- The database schema and appendixes used in the course and the available development environment in this course
- **Review of some basic concepts of SQL**
- Oracle Database 11g documentation and additional resources

Review of Restricting Data

- Restrict the rows that are returned by using the `WHERE` clause.
- Use comparison conditions to compare one expression with another value or expression.

Operator	Meaning
<code>BETWEEN ...AND...</code>	Between two values (inclusive)
<code>IN (set)</code>	Match any of a list of values
<code>LIKE</code>	Match a character pattern

- Use logical conditions to combine the result of two component conditions and produce a single result based on those conditions.

Review of Sorting Data

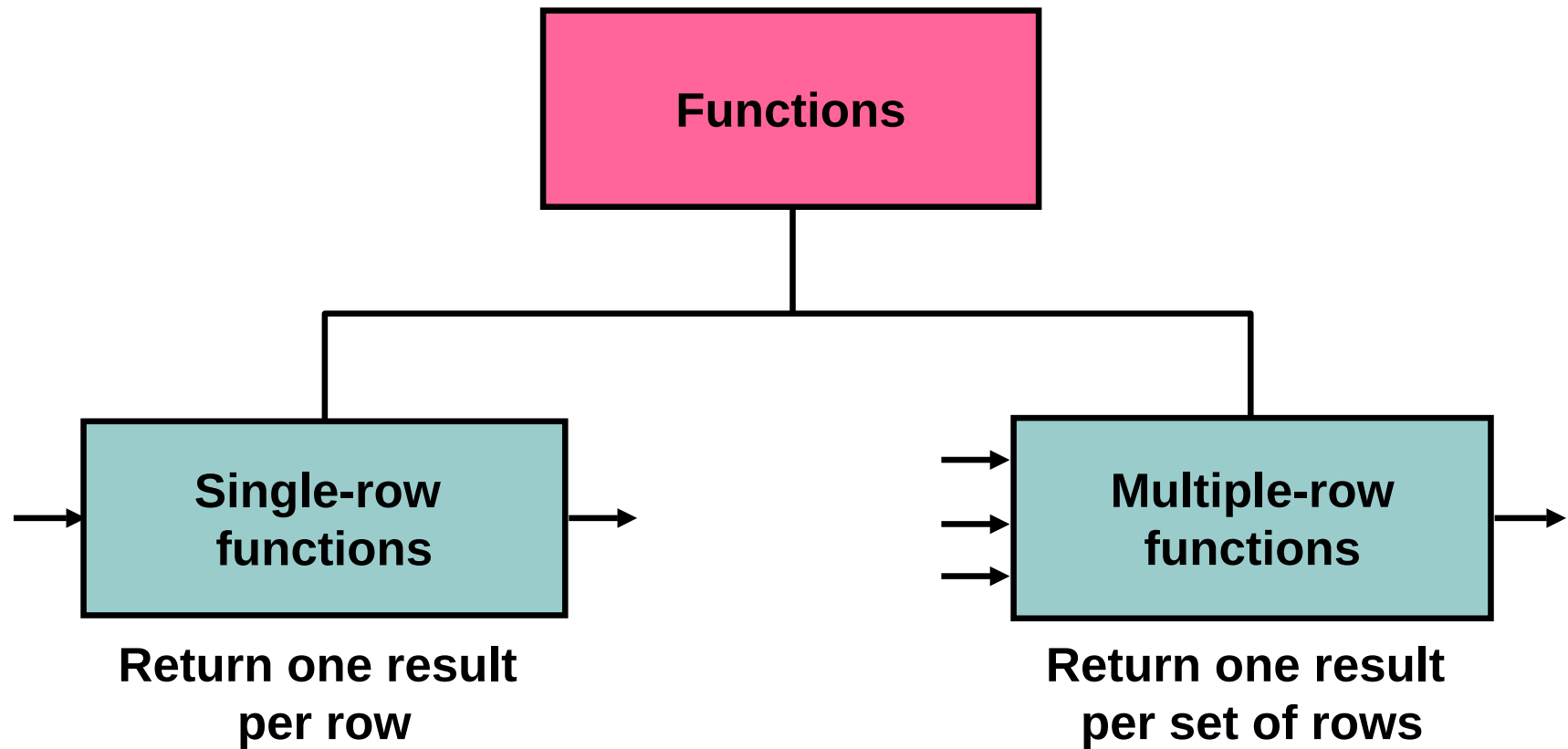
- Sort retrieved rows with the ORDER BY clause:
 - ASC: Ascending order, default
 - DESC: Descending order
- The ORDER BY clause comes last in the SELECT statement:

```
SELECT    last_name, job_id, department_id, hire_date
FROM      employees
ORDER BY  hire_date ;
```

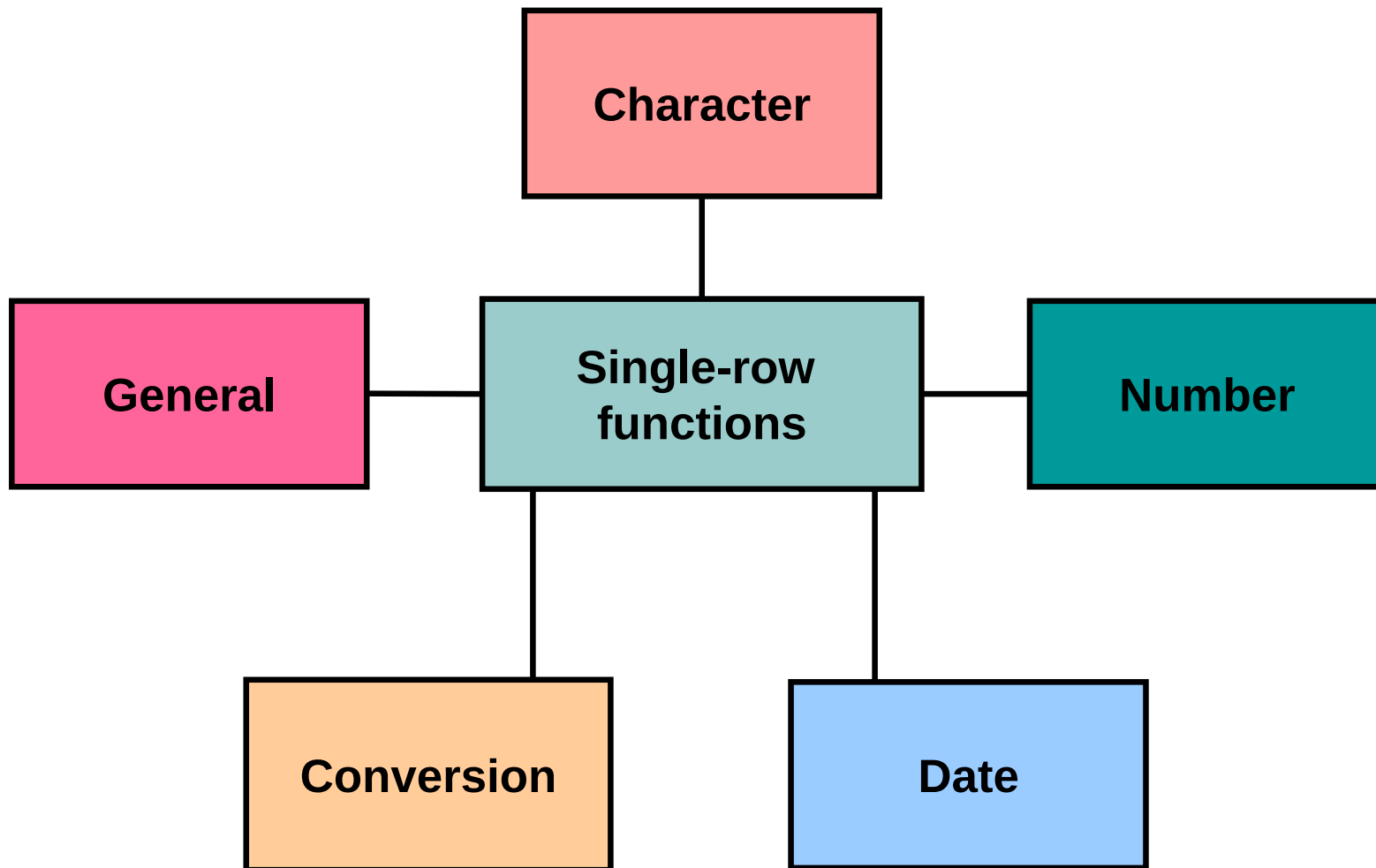
	 LAST_NAME	 JOB_ID	 DEPARTMENT_ID	HIRE_DATE
1	King	AD_PRES	90	17-JUN-87
2	Whalen	AD_ASST	10	17-SEP-87
3	Kochhar	AD_VP	90	21-SEP-89
4	Hunold	IT_PROG	60	03-JAN-90
5	Ernst	IT_PROG	60	21-MAY-91
6	De Haan	AD_VP	90	13-JAN-93

...

Review of SQL Functions

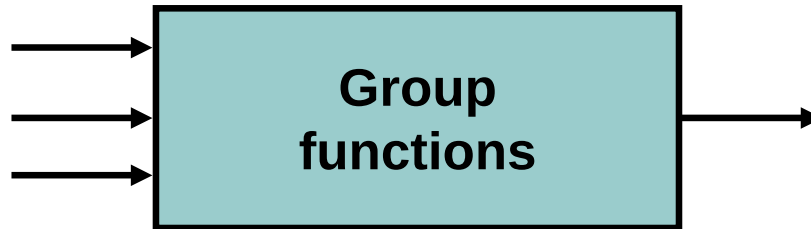


Review of Single-Row Functions



Review of Types of Group Functions

- AVG
- COUNT
- MAX
- MIN
- STDDEV
- SUM
- VARIANCE



Review of Using Subqueries

- A subquery is a `SELECT` statement nested in a clause of another `SELECT` statement.

- Syntax:

```
SELECT select_list
FROM   table
WHERE  expr operator
              (SELECT select_list
              FROM   table );
```

- Types of subqueries:

Single-row subquery	Multiple-row subquery
Returns only one row	Returns more than one row
Uses single-row comparison operators	Uses multiple-row comparison operators

Review of Manipulating Data

A data manipulation language (DML) statement is executed when you:

- Add new rows to a table
- Modify existing rows in a table
- Remove existing rows from a table

Function	Description
INSERT	Adds a new row to the table
UPDATE	Modifies existing rows in the table
DELETE	Removes existing rows from the table
MERGE	Updates, inserts, or deletes a row conditionally into/from a table

Lesson Agenda

- Course objectives and course agenda
- The database schema and appendixes used in the course and the available development environment in this course
- Review of some basic concepts of SQL
- **Oracle Database 11g documentation and additional resources**

Oracle Database 11g SQL Documentation

- *Oracle Database New Features Guide 11g Release 1 (11.1)*
- *Oracle Database Reference 11g Release 1 (11.1)*
- *Oracle Database SQL Language Reference 11g Release 1 (11.1)*
- *Oracle Database Concepts 11g Release 1 (11.1)*
- *Oracle Database SQL Developer User's Guide Release 1.2*

Additional Resources

For additional information about the new Oracle 11g SQL, refer to the following:

- *Oracle Database 11g: New Features eStudies*
- *Oracle by Example series (OBE): Oracle Database 11g*
 - http://www.oracle.com/technology/obe/11gr1_db/index.htm

Summary

In this lesson, you should have learned the following:

- The course objectives
- The sample tables used in the course

Practice I: Overview

This practice covers the following topics:

- Running the SQL Developer online tutorial
- Starting SQL Developer and creating a new database connection and browsing the HR tables
- Executing SQL statements using the SQL Worksheet
- Reviewing the basic concepts of SQL

1

Controlling User Access

Objectives

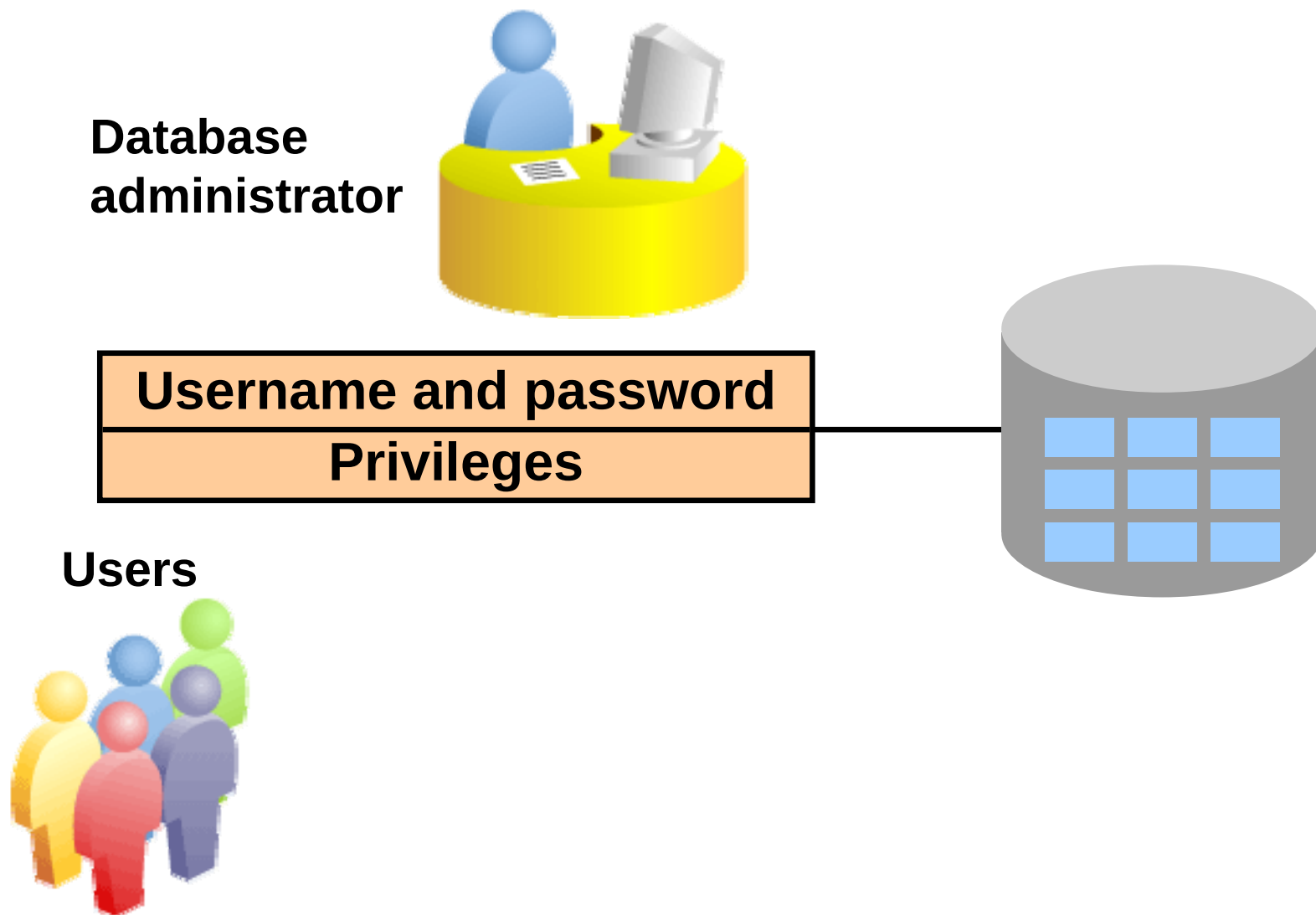
After completing this lesson, you should be able to do the following:

- Differentiate system privileges from object privileges
- Grant privileges on tables
- Grant roles
- Distinguish between privileges and roles

Lesson Agenda

- System privileges
- Creating a role
- Object privileges
- Revoking object privileges

Controlling User Access



Privileges

- Database security:
 - System security
 - Data security
- System privileges: Performing a particular action within the database
- Object privileges: Manipulating the content of the database objects
- Schemas: Collection of objects such as tables, views, and sequences

System Privileges

- More than 100 privileges are available.
- The database administrator has high-level system privileges for tasks such as:
 - Creating new users
 - Removing users
 - Removing tables
 - Backing up tables

Creating Users

The database administrator (DBA) creates users with the CREATE USER statement.

```
CREATE USER user  
IDENTIFIED BY password;
```

```
CREATE USER demo  
IDENTIFIED BY demo;
```


User System Privileges

- After a user is created, the DBA can grant specific system privileges to that user.

```
GRANT privilege [, privilege...]  
TO user [, user/ role, PUBLIC...];
```

- An application developer, for example, may have the following system privileges:
 - CREATE SESSION
 - CREATE TABLE
 - CREATE SEQUENCE
 - CREATE VIEW
 - CREATE PROCEDURE

Granting System Privileges

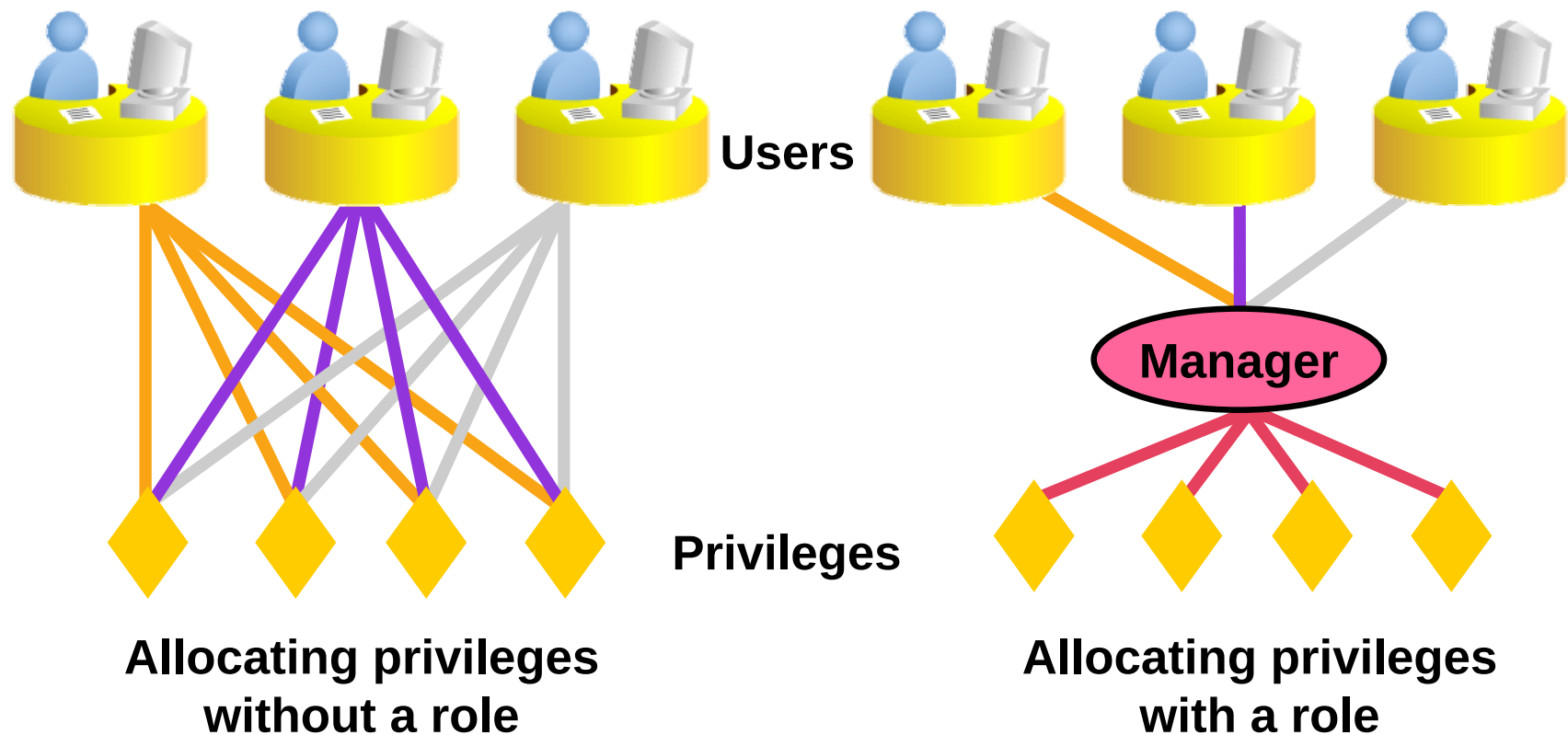
The DBA can grant specific system privileges to a user.

```
GRANT  create session, create table,  
       create sequence, create view  
TO     demo;
```

Lesson Agenda

- System privileges
- **Creating a role**
- Object privileges
- Revoking object privileges

What Is a Role?



Creating and Granting Privileges to a Role

- Create a role:

```
CREATE ROLE manager;
```

- Grant privileges to a role:

```
GRANT create table, create view  
TO manager;
```

- Grant a role to users:

```
GRANT manager TO BELL, KOCHHAR;
```

Changing Your Password

- The DBA creates your user account and initializes your password.
- You can change your password by using the `ALTER USER` statement.

```
ALTER USER demo  
IDENTIFIED BY employ;
```

Lesson Agenda

- System privileges
- Creating a role
- **Object privileges**
- Revoking object privileges

Object Privileges

Object privilege Table		View	Sequence
ALTER	✓		✓
DELETE	✓	✓	
INDEX	✓		
INSERT	✓	✓	
REFERENCES	✓		
SELECT	✓	✓	✓
UPDATE	✓	✓	

Object Privileges

- Object privileges vary from object to object.
- An owner has all the privileges on the object.
- An owner can give specific privileges on that owner's object.

```
GRANT      object_priv [(columns)]  
ON         object  
TO         {user | role | PUBLIC}  
[WITH GRANT OPTION];
```

Granting Object Privileges

- Grant query privileges on the EMPLOYEES table:

```
GRANT  select
ON     employees
TO     demo;
```

- Grant privileges to update specific columns to users and roles:

```
GRANT  update (department_name, location_id)
ON     departments
TO     demo, manager;
```

Passing On Your Privileges

- Give a user authority to pass along privileges:

```
GRANT  select, insert
ON     departments
TO     demo
WITH   GRANT OPTION;
```

- Allow all users on the system to query data from Alice's DEPARTMENTS table:

```
GRANT  select
ON     alice.departments
TO     PUBLIC;
```

Confirming Granted Privileges

Data Dictionary View	Description
ROLE_SYS_PRIVS	System privileges granted to roles
ROLE_TAB_PRIVS	Table privileges granted to roles
USER_ROLE_PRIVS	Roles accessible by the user
USER_SYS_PRIVS	System privileges granted to the user
USER_TAB_PRIVS_MADE	Object privileges granted on the user's objects
USER_TAB_PRIVS_RECD	Object privileges granted to the user
USER_COL_PRIVS_MADE	Object privileges granted on the columns of the user's objects
USER_COL_PRIVS_RECD	Object privileges granted to the user on specific columns

Lesson Agenda

- System privileges
- Creating a role
- Object privileges
- Revoking object privileges

Revoking Object Privileges

- You use the REVOKE statement to revoke privileges granted to other users.
- Privileges granted to others through the WITH GRANT OPTION clause are also revoked.

```
REVOKE {privilege [, privilege...] | ALL}
ON      object
FROM    {user[, user...] | role | PUBLIC}
[CASCADE CONSTRAINTS];
```

Revoking Object Privileges

Revoke the SELECT and INSERT privileges given to the demo user on the DEPARTMENTS table.

```
REVOKE  select, insert
ON      departments
FROM    demo;
```

Quiz

Which of the following statements are true?

1. After a user creates an object, the user can pass along any of the available object privileges to other users by using the `GRANT` statement.
2. A user can create roles by using the `CREATE ROLE` statement to pass along a collection of system or object privileges to other users.
3. Users can change their own passwords.
4. Users can view the privileges granted to them and those that are granted on their objects.

Summary

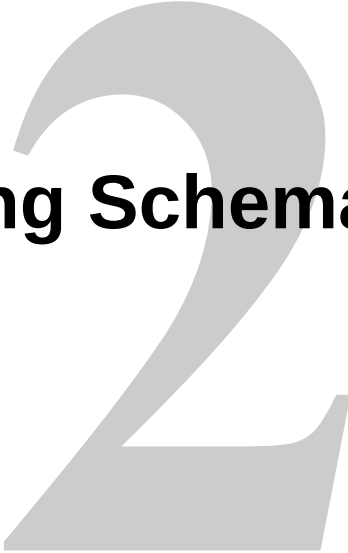
In this lesson, you should have learned about statements that control access to the database and database objects.

Statement	Action
CREATE USER	Creates a user (usually performed by a DBA)
GRANT	Gives other users privileges to access the objects
CREATE ROLE	Creates a collection of privileges (usually performed by a DBA)
ALTER USER	Changes a user's password
REVOKE	Removes privileges on an object from users

Practice 1: Overview

This practice covers the following topics:

- Granting other users privileges to your table
- Modifying another user's table through the privileges granted to you
- Creating a synonym



Managing Schema Objects

Objectives

After completing this lesson, you should be able to do the following:

- Add constraints
- Create indexes
- Create indexes using the `CREATE TABLE` statement
- Create function-based indexes
- Drop columns and set columns as `UNUSED`
- Perform `FLASHBACK` operations
- Create and use external tables

Lesson Agenda

- Using the ALTER TABLE statement to add, modify, and drop a column
- Managing constraints
 - Adding and dropping a constraint
 - Deferring constraints
 - Enabling and disabling a constraint
- Creating indexes
 - Using the CREATE TABLE statement
 - Creating function-based indexes
 - Removing an index
- Performing flashback operations
- Creating and using external tables

ALTER TABLE Statement

Use the ALTER TABLE statement to:

- Add a new column
- Modify an existing column
- Define a default value for the new column
- Drop a column

ALTER TABLE Statement

Use the ALTER TABLE statement to add, modify, or drop columns:

```
ALTER TABLE table
ADD          (column datatype [DEFAULT expr]
             [, column datatype]...);
```

```
ALTER TABLE table
MODIFY       (column datatype [DEFAULT expr]
             [, column datatype]...);
```

```
ALTER TABLE table
DROP        (column);
```

Adding a Column

- You use the ADD clause to add columns:

```
ALTER TABLE dept80  
ADD          (job_id VARCHAR2(9));
```

```
ALTER TABLE dept80 succeeded.
```

- The new column becomes the last column:

	EMPLOYEE_ID	LAST_NAME	ANNSAL	HIRE_DATE	JOB_ID
1	149	Zlotkey	10500	29-JAN-00	(null)
2	174	Abel	11000	11-MAY-96	(null)
3	176	Taylor	8600	24-MAR-98	(null)

Modifying a Column

- You can change a column's data type, size, and default value.

```
ALTER TABLE dept80  
MODIFY      (last_name VARCHAR2(30));
```

```
ALTER TABLE dept80 succeeded.
```

- A change to the default value affects only subsequent insertions to the table.

Dropping a Column

Use the DROP COLUMN clause to drop columns you no longer need from the table:

```
ALTER TABLE dept80  
DROP COLUMN job_id;
```

```
ALTER TABLE dept80 succeeded.
```

	 EMPLOYEE_ID	 LAST_NAME	 ANNSAL	HIRE_DATE
1	149	Zlotkey	10500	29-JAN-00
2	174	Abel	11000	11-MAY-96
3	176	Taylor	8600	24-MAR-98

SET UNUSED Option

- You use the SET UNUSED option to mark one or more columns as unused.
- You use the DROP UNUSED COLUMNS option to remove the columns that are marked as unused.

```
ALTER TABLE <table_name>  
SET UNUSED(<column name>);
```

OR

```
ALTER TABLE <table_name>  
SET UNUSED COLUMN <column_name>;
```

```
ALTER TABLE <table_name>  
DROP UNUSED COLUMNS;
```

Lesson Agenda

- Using the ALTER TABLE statement to add, modify, and drop a column
- Managing constraints
 - Adding and dropping a constraint
 - Deferring constraints
 - Enabling and disabling a constraint
- Creating indexes
 - Using the CREATE TABLE statement
 - Creating function-based indexes
 - Removing an index
- Performing flashback operations
- Creating and using external tables

Adding a Constraint Syntax

Use the ALTER TABLE statement to:

- Add or drop a constraint, but not modify its structure
- Enable or disable constraints
- Add a NOT NULL constraint by using the MODIFY clause

```
ALTER TABLE  <table_name>  
ADD [CONSTRAINT <constraint_name>]  
type (<column_name>);
```

Adding a Constraint

Add a FOREIGN KEY constraint to the EMP2 table indicating that a manager must already exist as a valid employee in the EMP2 table.

```
ALTER TABLE emp2  
MODIFY employee_id Primary Key;
```

```
ALTER TABLE emp2 succeeded.
```

```
ALTER TABLE emp2  
ADD CONSTRAINT emp_mgr_fk  
FOREIGN KEY(manager_id)  
REFERENCES emp2(employee_id);
```

```
ALTER TABLE succeeded.
```

ON DELETE CASCADE

Delete child rows when a parent key is deleted:

```
ALTER TABLE Emp2 ADD CONSTRAINT emp_dt_fk  
FOREIGN KEY (Department_id)  
REFERENCES departments(department_id) ON DELETE CASCADE;
```

```
ALTER TABLE Emp2 succeeded.
```

Deferring Constraints

Constraints can have the following attributes:

- DEFERRABLE or NOT DEFERRABLE
- INITIALLY DEFERRED or INITIALLY IMMEDIATE

```
ALTER TABLE dept2  
ADD CONSTRAINT dept2_id_pk  
PRIMARY KEY (department_id)  
DEFERRABLE INITIALLY DEFERRED
```

Deferring constraint on creation

```
SET CONSTRAINTS dept2_id_pk IMMEDIATE
```

Changing a specific constraint attribute

```
ALTER SESSION  
SET CONSTRAINTS= IMMEDIATE
```

Changing all constraints for a session

Difference Between INITIALLY DEFERRED and INITIALLY IMMEDIATE

INITIALLY DEFERRED	Waits to check the constraint until the transaction ends
INITIALLY IMMEDIATE	Checks the constraint at the end of the statement execution

```
CREATE TABLE emp_new_sal (salary NUMBER
    CONSTRAINT sal_ck
    CHECK (salary > 100)
    DEFERRABLE INITIALLY IMMEDIATE,
    bonus NUMBER
    CONSTRAINT bonus_ck
    CHECK (bonus > 0 )
    DEFERRABLE INITIALLY DEFERRED );
```

```
create table succeeded.
```

Dropping a Constraint

- Remove the manager constraint from the EMP2 table:

```
ALTER TABLE emp2  
DROP CONSTRAINT emp_mgr_fk;
```

```
ALTER TABLE Emp2 succeeded.
```

- Remove the PRIMARY KEY constraint on the DEPT2 table and drop the associated FOREIGN KEY constraint on the EMP2.DEPARTMENT_ID column:

```
ALTER TABLE dept2  
DROP PRIMARY KEY CASCADE;
```

```
ALTER TABLE dept2 succeeded.
```

Disabling Constraints

- Execute the `DISABLE` clause of the `ALTER TABLE` statement to deactivate an integrity constraint.
- Apply the `CASCADE` option to disable dependent integrity constraints.

```
ALTER TABLE emp2  
DISABLE CONSTRAINT emp_dt_fk;
```

```
ALTER TABLE Emp2 succeeded.
```

Enabling Constraints

- Activate an integrity constraint currently disabled in the table definition by using the `ENABLE` clause.

```
ALTER TABLE      emp2  
ENABLE CONSTRAINT emp_dt_fk;
```

```
ALTER TABLE Emp2 succeeded.
```

- A `UNIQUE` index is automatically created if you enable a `UNIQUE` key or a `PRIMARY KEY` constraint.

Cascading Constraints

- The CASCADE CONSTRAINTS clause is used along with the DROP COLUMN clause.
- The CASCADE CONSTRAINTS clause drops all referential integrity constraints that refer to the primary and unique keys defined on the dropped columns.
- The CASCADE CONSTRAINTS clause also drops all multicolumn constraints defined on the dropped columns.

Cascading Constraints

Example:

```
ALTER TABLE emp2  
DROP COLUMN employee_id CASCADE CONSTRAINTS;
```

```
ALTER TABLE Emp2 succeeded.
```

```
ALTER TABLE test1  
DROP (col1_pk, col2_fk, col1) CASCADE CONSTRAINTS;
```

```
ALTER TABLE test1 succeeded.
```

Renaming Table Columns and Constraints

Use the RENAME COLUMN clause of the ALTER TABLE statement to rename table columns.

a

```
ALTER TABLE marketing RENAME COLUMN team_id  
TO id;
```

```
ALTER TABLE marketing succeeded.
```

Use the RENAME CONSTRAINT clause of the ALTER TABLE statement to rename any existing constraint for a table.

b

```
ALTER TABLE marketing RENAME CONSTRAINT mktg_pk  
TO new_mktg_pk;
```

```
ALTER TABLE marketing succeeded.
```

Lesson Agenda

- Using the `ALTER TABLE` statement to add, modify, and drop a column
- Managing constraints
 - Adding and dropping a constraint
 - Deferring constraints
 - Enabling and disabling a constraint
- **Creating indexes**
 - Using the `CREATE TABLE` statement
 - Creating function-based indexes
 - Removing an index
- Performing flashback operations
- Creating and using external tables

Overview of Indexes

Indexes are created:

- Automatically
 - PRIMARY KEY creation
 - UNIQUE KEY creation
- Manually
 - The CREATE INDEX statement
 - The CREATE TABLE statement

CREATE INDEX with the CREATE TABLE Statement

```
CREATE TABLE NEW_EMP  
(employee_id NUMBER(6)  
    PRIMARY KEY USING INDEX  
    (CREATE INDEX emp_id_idx ON  
    NEW_EMP(employee_id)),  
first_name VARCHAR2(20),  
last_name VARCHAR2(25));
```

CREATE TABLE succeeded.

```
SELECT INDEX_NAME, TABLE_NAME  
FROM USER_INDEXES  
WHERE TABLE_NAME = 'NEW_EMP';
```

	INDEX_NAME	TABLE_NAME
1	EMP_ID_IDX	NEW_EMP

Function-Based Indexes

- A function-based index is based on expressions.
- The index expression is built from table columns, constants, SQL functions, and user-defined functions.

```
CREATE INDEX upper_dept_name_idx  
ON dept2 (UPPER(department_name));
```

```
CREATE INDEX succeeded.
```

```
SELECT *  
FROM   dept2  
WHERE  UPPER(department_name) = 'SALES';
```

Removing an Index

- Remove an index from the data dictionary by using the DROP INDEX command:

```
DROP INDEX index;
```

- Remove the UPPER_DEPT_NAME_IDX index from the data dictionary:

```
DROP INDEX upper_dept_name_idx;
```

```
DROP INDEX upper_dept_name_idx succeeded.
```

- To drop an index, you must be the owner of the index or have the DROP ANY INDEX privilege.

DROP TABLE ... PURGE

```
DROP TABLE dept80 PURGE;
```

```
DROP TABLE dept80 succeeded.
```

Lesson Agenda

- Using the ALTER TABLE statement to add, modify, and drop a column
- Managing constraints
 - Adding and dropping a constraint
 - Deferring constraints
 - Enabling and disabling a constraint
- Creating indexes
 - Using the CREATE TABLE statement
 - Creating function-based indexes
 - Removing an index
- Performing flashback operations
- Creating and using external tables

FLASHBACK TABLE Statement

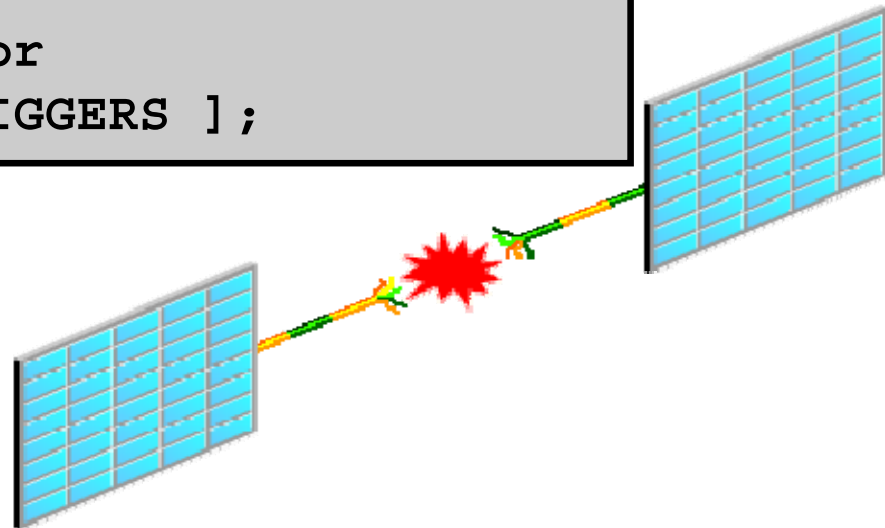
- Enables you to recover tables to a specified point in time with a single statement
- Restores table data along with associated indexes, and constraints
- Enables you to revert the table and its contents to a certain point in time or SCN



FLASHBACK TABLE Statement

- Repair tool for accidental table modifications
 - Restores a table to an earlier point in time
 - Benefits: Ease of use, availability, and fast execution
 - Is performed in place
- Syntax:

```
FLASHBACK TABLE [schema.] table [,  
[ schema.] table ] ...  
TO { TIMESTAMP | SCN } expr  
[ { ENABLE | DISABLE } TRIGGERS ] ;
```



Using the FLASHBACK TABLE Statement

```
DROP TABLE emp2;
```

```
DROP TABLE emp2 succeeded.
```

```
SELECT original_name, operation, droptime FROM  
recyclebin;
```

ORIGINAL_NAME	OPERATION	DROPTIME
EMP2	DROP	2007-07-02:06:07:41

...

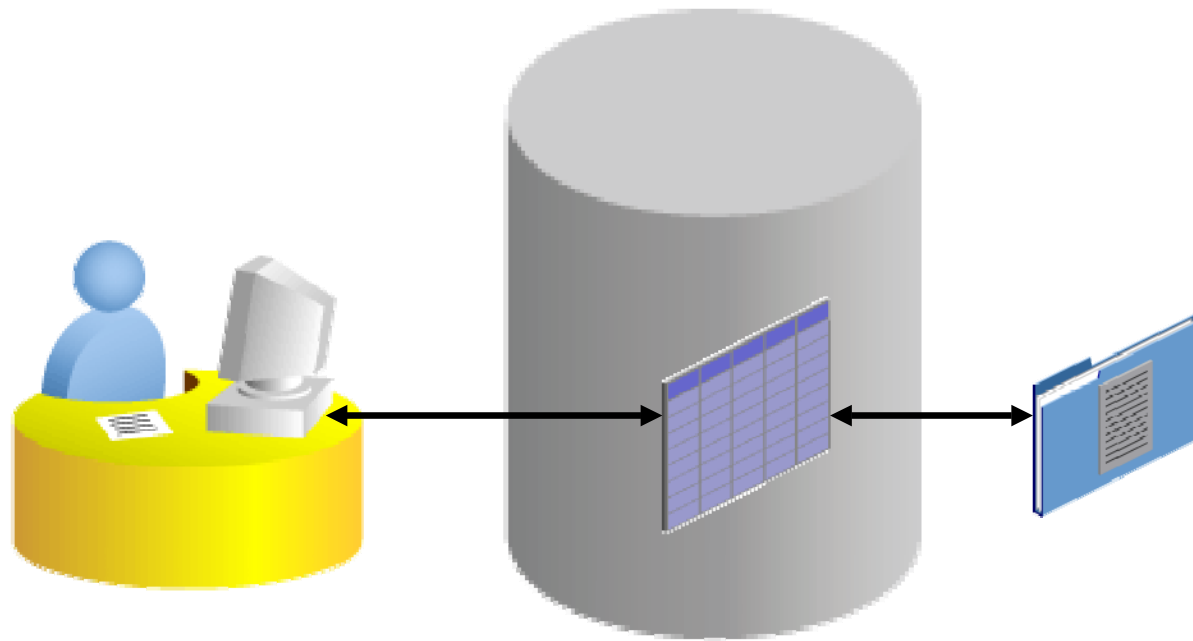
```
FLASHBACK TABLE emp2 TO BEFORE DROP;
```

```
FLASHBACK TABLE succeeded.
```

Lesson Agenda

- Using the ALTER TABLE statement to add, modify, and drop a column
- Managing constraints
 - Adding and dropping a constraint
 - Deferring constraints
 - Enabling and disabling a constraint
- Creating indexes
 - Using the CREATE TABLE statement
 - Creating function-based indexes
 - Removing an index
- Performing flashback operations
- Creating and using external tables

External Tables



Creating a Directory for the External Table

Create a DIRECTORY object that corresponds to the directory on the file system where the external data source resides.

```
CREATE OR REPLACE DIRECTORY emp_dir  
AS '/.../emp_dir';  
  
GRANT READ ON DIRECTORY emp_dir TO hr;
```

Creating an External Table

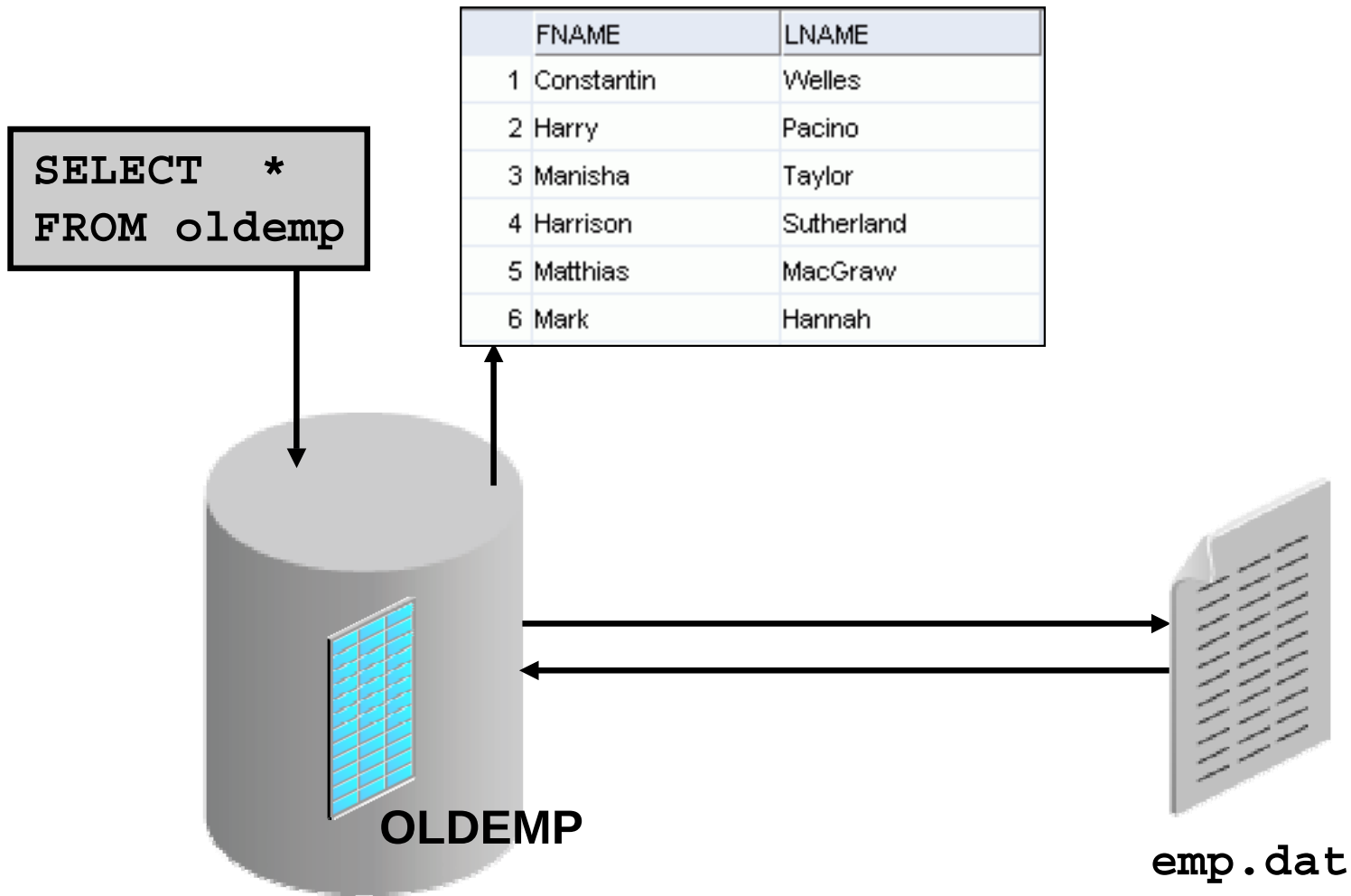
```
CREATE TABLE <table_name>
  ( <col_name> <datatype>, ... )
ORGANIZATION EXTERNAL
  (TYPE <access_driver_type>
   DEFAULT DIRECTORY <directory_name>
   ACCESS PARAMETERS
     (... ) )
  LOCATION ('<location_specifier>')
REJECT LIMIT [0 | <number> | UNLIMITED];
```

Creating an External Table by Using ORACLE_LOADER

```
CREATE TABLE oldemp (  
  fname char(25), lname CHAR(25))  
  ORGANIZATION EXTERNAL  
  (TYPE ORACLE_LOADER  
  DEFAULT DIRECTORY emp_dir  
  ACCESS PARAMETERS  
  (RECORDS DELIMITED BY NEWLINE  
  NOBADFILE  
  NOLOGFILE  
  FIELDS TERMINATED BY ', '  
  (fname POSITION ( 1:20) CHAR,  
  lname POSITION (22:41) CHAR))  
  LOCATION ('emp.dat'))  
  PARALLEL 5  
  REJECT LIMIT 200;
```

CREATE TABLE succeeded.

Querying External Tables



Creating an External Table by Using ORACLE_DATAPUMP: Example

```
CREATE TABLE emp_ext
(employee_id, first_name, last_name)
ORGANIZATION EXTERNAL
(
  TYPE ORACLE_DATAPUMP
  DEFAULT DIRECTORY emp_dir
  LOCATION
    ('emp1.exp', 'emp2.exp')
)
PARALLEL
AS
SELECT employee_id, first_name, last_name
FROM   employees;
```


Quiz

A FOREIGN KEY constraint enforces the following action:
When the data in the parent key is deleted, all the rows in the child table that depend on the deleted parent key values are also deleted.

1. True
2. False

Quiz

In all the cases, when you execute a `DROP TABLE` command, the database renames the table and places it in a recycle bin, from where it can later be recovered by using the `FLASHBACK TABLE` statement.

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Add constraints
- Create indexes
- Create indexes by using the `CREATE TABLE` statement
- Create function-based indexes
- Drop columns and set columns as `UNUSED`
- Perform `FLASHBACK` operations
- Create and use external tables

Practice 2: Overview

This practice covers the following topics:

- Altering tables
- Adding columns
- Dropping columns
- Creating indexes
- Creating external tables



Managing Objects with Data Dictionary Views

Objectives

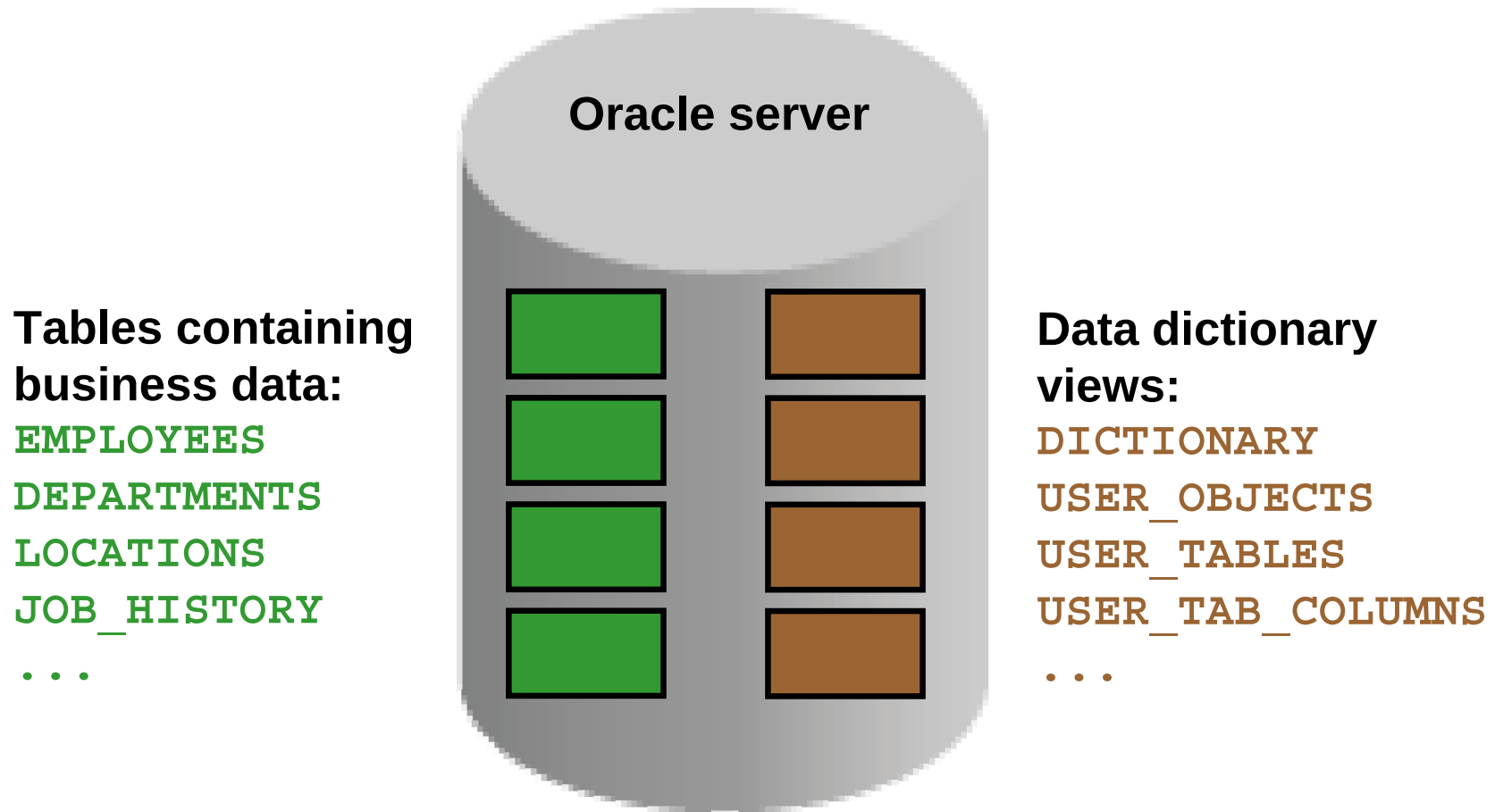
After completing this lesson, you should be able to do the following:

- Use the data dictionary views to research data on your objects
- Query various data dictionary views

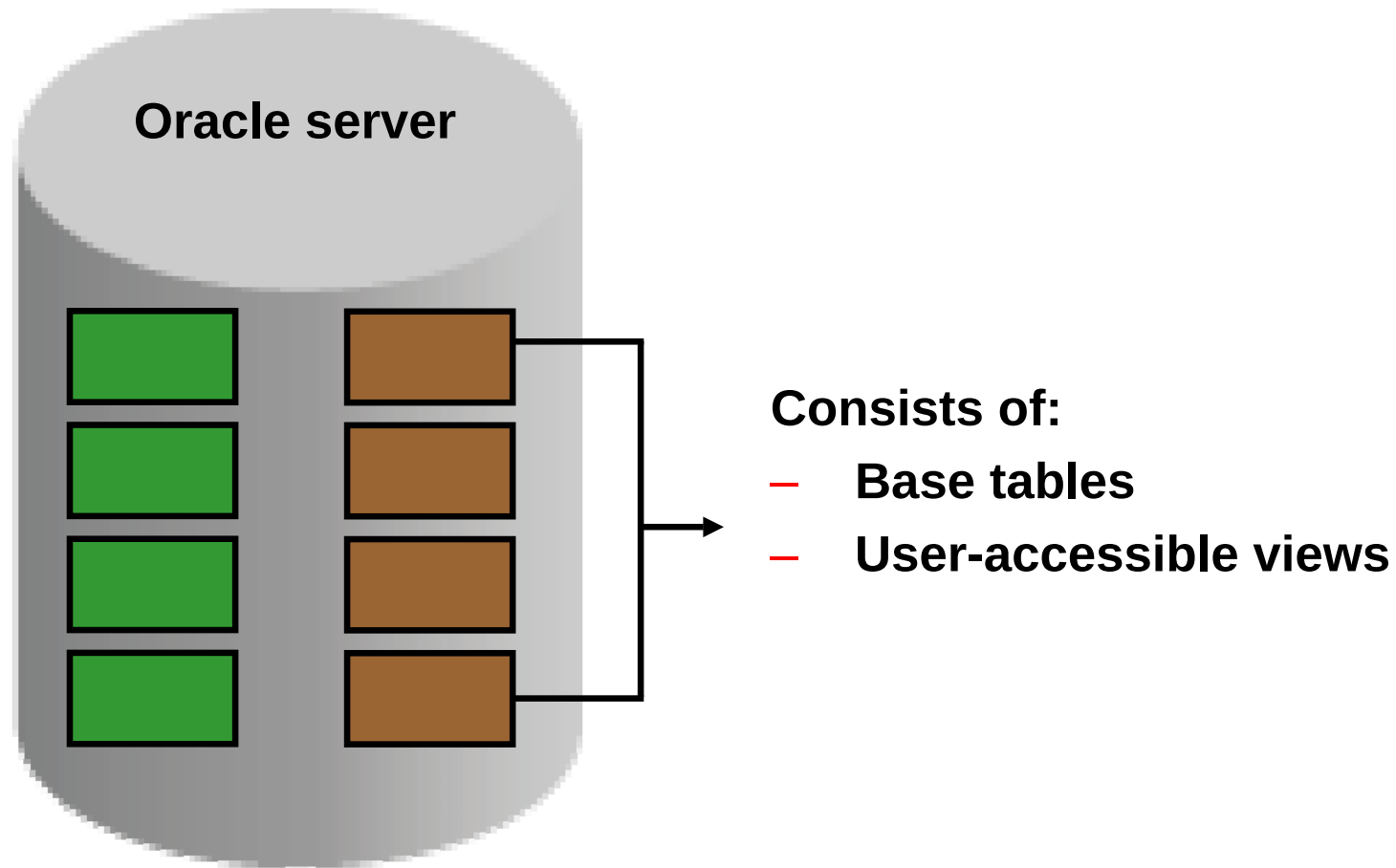
Lesson Agenda

- Introduction to data dictionary
- Querying the dictionary views for the following:
 - Table information
 - Column information
 - Constraint information
- Querying the dictionary views for the following:
 - View information
 - Sequence information
 - Synonym information
 - Index information
- Adding a comment to a table and querying the dictionary views for comment information

Data Dictionary



Data Dictionary Structure



Data Dictionary Structure

View naming convention:

View Prefix	Purpose
USER	User's view (what is in your schema; what you own)
ALL	Expanded user's view (what you can access)
DBA	Database administrator's view (what is in everyone's schemas)
V\$	Performance-related data

How to Use the Dictionary Views

Start with `DICTIONARY`. It contains the names and descriptions of the dictionary tables and views.

```
DESCRIBE DICTIONARY
```

Name	Null	Type
TABLE_NAME		VARCHAR2(30)
COMMENTS		VARCHAR2(4000)

2 rows selected

```
SELECT *  
FROM   dictionary  
WHERE  table_name = 'USER_OBJECTS';
```

	TABLE_NAME	COMMENTS
1	USER_OBJECTS	Objects owned by the user

USER_OBJECTS and ALL_OBJECTS Views

USER_OBJECTS:

- Query USER_OBJECTS to see all the objects that you own
- Is a useful way to obtain a listing of all object names and types in your schema, plus the following information:
 - Date created
 - Date of last modification
 - Status (valid or invalid)

ALL_OBJECTS:

- Query ALL_OBJECTS to see all objects to which you have access

USER_OBJECTS View

```
SELECT object_name, object_type, created, status
FROM   user_objects
ORDER BY object_type;
```

	 OBJECT_NAME	 OBJECT_TYPE	CREATED	 STATUS
1	REG_ID_PK	INDEX	30-MAY-07	VALID

...

22	DEPARTMENTS_SEQ	SEQUENCE	30-MAY-07	VALID
23	LOCATIONS	TABLE	30-MAY-07	VALID
24	DEPARTMENTS	TABLE	30-MAY-07	VALID
25	REGIONS	TABLE	30-MAY-07	VALID
26	JOB_HISTORY	TABLE	30-MAY-07	VALID
27	JOBS	TABLE	30-MAY-07	VALID
28	EMPLOYEES	TABLE	30-MAY-07	VALID
29	COUNTRIES	TABLE	30-MAY-07	VALID
30	JOB_GRADES	TABLE	30-MAY-07	VALID
31	EMP_DETAILS_VIEW	VIEW	30-MAY-07	VALID

Lesson Agenda

- Introduction to data dictionary
- Querying the dictionary views for the following:
 - Table information
 - Column information
 - Constraint information
- Querying the dictionary views for the following:
 - View information
 - Sequence information
 - Synonym information
 - Index information
- Adding a comment to a table and querying the dictionary views for comment information

Table Information

USER_TABLES:

```
DESCRIBE user_tables
```

Name	Null	Type
-----	-----	-----
TABLE_NAME	NOT NULL	VARCHAR2(30)
TABLESPACE_NAME		VARCHAR2(30)
CLUSTER_NAME		VARCHAR2(30)
IOT_NAME		VARCHAR2(30)

...

```
SELECT table_name
FROM   user_tables;
```

	TABLE_NAME
1	REGIONS
2	LOCATIONS
3	DEPARTMENTS
4	JOBS
5	EMPLOYEES
6	JOB_HISTORY

...

Column Information

USER_TAB_COLUMNS:

```
DESCRIBE user_tab_columns
```

Name	Null	Type
-----	-----	-----
TABLE_NAME	NOT NULL	VARCHAR2(30)
COLUMN_NAME	NOT NULL	VARCHAR2(30)
DATA_TYPE		VARCHAR2(106)
DATA_TYPE_MOD		VARCHAR2(3)
DATA_TYPE_OWNER		VARCHAR2(30)
DATA_LENGTH	NOT NULL	NUMBER
DATA_PRECISION		NUMBER
DATA_SCALE		NUMBER
NULLABLE		VARCHAR2(1)
COLUMN_ID		NUMBER
DEFAULT_LENGTH		NUMBER
DATA_DEFAULT		LONG()

...

Column Information

```
SELECT column_name, data_type, data_length,
       data_precision, data_scale, nullable
FROM   user_tab_columns
WHERE  table_name = 'EMPLOYEES';
```

	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	DATA_PRECISION	DATA_SCALE	NULLABLE
1	EMPLOYEE_ID	NUMBER	22	6	0	N
2	FIRST_NAME	VARCHAR2	20	(null)	(null)	Y
3	LAST_NAME	VARCHAR2	25	(null)	(null)	N
4	EMAIL	VARCHAR2	25	(null)	(null)	N
5	PHONE_NUMBER	VARCHAR2	20	(null)	(null)	Y
6	HIRE_DATE	DATE	7	(null)	(null)	N
7	JOB_ID	VARCHAR2	10	(null)	(null)	N
8	SALARY	NUMBER	22	8	2	Y
9	COMMISSION_PCT	NUMBER	22	2	2	Y
10	MANAGER_ID	NUMBER	22	6	0	Y
11	DEPARTMENT_ID	NUMBER	22	4	0	Y

Constraint Information

- USER_CONSTRAINTS describes the constraint definitions on your tables.
- USER_CONS_COLUMNS describes columns that are owned by you and that are specified in constraints.

```
DESCRIBE user_constraints
```

Name	Null	Type
OWNER	NOT NULL	VARCHAR2(30)
CONSTRAINT_NAME	NOT NULL	VARCHAR2(30)
CONSTRAINT_TYPE		VARCHAR2(1)
TABLE_NAME	NOT NULL	VARCHAR2(30)
SEARCH_CONDITION		LONG()
R_OWNER		VARCHAR2(30)
R_CONSTRAINT_NAME		VARCHAR2(30)
DELETE_RULE		VARCHAR2(9)
STATUS		VARCHAR2(8)

...

USER_CONSTRAINTS: Example

```
SELECT constraint_name, constraint_type,
       search_condition, r_constraint_name,
       delete_rule, status
FROM   user_constraints
WHERE  table_name = 'EMPLOYEES';
```

	CONSTRAINT...	CONSTRAINT_T...	SEARCH_CONDITION	R_CONSTRAINT...	DELETE_...	STATUS
1	EMP_LAST_NAME...	C	"LAST_NAME" IS NOT NULL	(null)	(null)	ENABLED
2	EMP_EMAIL_NN	C	"EMAIL" IS NOT NULL	(null)	(null)	ENABLED
3	EMP_HIRE_DATE_...	C	"HIRE_DATE" IS NOT NULL	(null)	(null)	ENABLED
4	EMP_JOB_NN	C	"JOB_ID" IS NOT NULL	(null)	(null)	ENABLED
5	EMP_SALARY_MIN	C	salary > 0	(null)	(null)	ENABLED
6	EMP_EMAIL_UK	U	(null)	(null)	(null)	ENABLED
7	EMP_EMP_ID_PK	P	(null)	(null)	(null)	ENABLED
8	EMP_DEPT_FK	R	(null)	DEPT_ID_PK	NO ACTION	ENABLED
9	EMP_JOB_FK	R	(null)	JOB_ID_PK	NO ACTION	ENABLED
10	EMP_MANAGER_FK	R	(null)	EMP_EMP_ID_PK	NO ACTION	ENABLED

Querying USER_CONS_COLUMNS

```
DESCRIBE user_cons_columns
```

Name	Null	Type
OWNER	NOT NULL	VARCHAR2(30)
CONSTRAINT_NAME	NOT NULL	VARCHAR2(30)
TABLE_NAME	NOT NULL	VARCHAR2(30)
COLUMN_NAME		VARCHAR2(4000)
POSITION		NUMBER

```
SELECT constraint_name, column_name
FROM   user_cons_columns
WHERE  table_name = 'EMPLOYEES';
```

	CONSTRAINT_NAME	COLUMN_NAME
1	EMP_LAST_NAME_NN	LAST_NAME
2	EMP_EMAIL_NN	EMAIL
3	EMP_HIRE_DATE_NN	HIRE_DATE
4	EMP_JOB_NN	JOB_ID

...

Lesson Agenda

- Introduction to data dictionary
- Querying the dictionary views for the following:
 - Table information
 - Column information
 - Constraint information
- Querying the dictionary views for the following:
 - View information
 - Sequence information
 - Synonym information
 - Index information
- Adding a comment to a table and querying the dictionary views for comment information

View Information

1 **DESCRIBE user_views**

Name	Null	Type
-----	-----	-----
VIEW_NAME	NOT NULL	VARCHAR2(30)
TEXT_LENGTH		NUMBER
TEXT		LONG()

2 **SELECT view_name FROM user_views;**

VIEW_NAME
1 EMP_DETAILS_VIEW

3 **SELECT text FROM user_views
WHERE view_name = 'EMP_DETAILS_VIEW';**

TEXT
1 SELECT e.employee_id, e.job_id, e.manager_id, e.department_id, d.location_id, l.country_id, e.first_name, e.last_name,

...

c.region_id = r.region_id AND j.job_id = e.job_id WITH READ ONLY
--

Sequence Information

```
DESCRIBE user_sequences
```

Name	Null	Type

SEQUENCE_NAME	NOT NULL	VARCHAR2(30)
MIN_VALUE		NUMBER
MAX_VALUE		NUMBER
INCREMENT_BY	NOT NULL	NUMBER
CYCLE_FLAG		VARCHAR2(1)
ORDER_FLAG		VARCHAR2(1)
CACHE_SIZE	NOT NULL	NUMBER
LAST_NUMBER	NOT NULL	NUMBER

Confirming Sequences

- Verify your sequence values in the USER_SEQUENCES data dictionary table.

```
SELECT      sequence_name, min_value, max_value,
            increment_by, last_number
FROM        user_sequences;
```

[illegible]

- The `LAST_NUMBER` column displays the next available sequence number if `NOCACHE` is specified.

Index Information

- USER_INDEXES provides information about your indexes.
- USER_IND_COLUMNS describes columns comprising your indexes and columns of indexes on your tables.

DESCRIBE user_indexes

Name	Null	Type

INDEX_NAME	NOT NULL	VARCHAR2(30)
INDEX_TYPE		VARCHAR2(27)
TABLE_OWNER	NOT NULL	VARCHAR2(30)
TABLE_NAME	NOT NULL	VARCHAR2(30)
TABLE_TYPE		VARCHAR2(11)
UNIQUENESS		VARCHAR2(9)

...

USER_INDEXES: Examples

a

```
SELECT index_name, table_name, uniqueness
FROM   user_indexes
WHERE  table_name = 'EMPLOYEES';
```

	 INDEX_NAME	 TABLE_NAME	 UNIQUENESS
1	EMP_JOB_IX	EMPLOYEES	NONUNIQUE
2	EMP_MANAGER_IX	EMPLOYEES	NONUNIQUE
3	EMP_NAME_IX	EMPLOYEES	NONUNIQUE
4	EMP_EMP_ID_PK	EMPLOYEES	UNIQUE
5	EMP_EMAIL_UK	EMPLOYEES	UNIQUE
6	EMP_DEPARTMENT_IX	EMPLOYEES	NONUNIQUE

b

```
SELECT INDEX_NAME, TABLE_NAME
FROM   USER_INDEXES
WHERE  TABLE_NAME = 'EMP_LIB';
```

	 INDEX_NAME	 TABLE_NAME
1	SYS_C0017560	EMP_LIB

Querying USER_IND_COLUMNS

```
DESCRIBE user_ind_columns
```

Name	Null	Type

INDEX_NAME		VARCHAR2(30)
TABLE_NAME		VARCHAR2(30)
COLUMN_NAME		VARCHAR2(4000)
COLUMN_POSITION		NUMBER
COLUMN_LENGTH		NUMBER
CHAR_LENGTH		NUMBER
DESCEND		VARCHAR2(4)

```
SELECT INDEX_NAME, COLUMN_NAME, TABLE_NAME
FROM   user_ind_columns
WHERE  INDEX_NAME = 'LNAME_IDX';
```

	INDEX_NAME	COLUMN_NAME	TABLE_NAME
1	LNAME_IDX	LAST_NAME	EMP_TEST

Synonym Information

```
DESCRIBE user_synonyms
```

Name	Null	Type
-----	-----	-----
SYNONYM_NAME	NOT NULL	VARCHAR2(30)
TABLE_OWNER		VARCHAR2(30)
TABLE_NAME	NOT NULL	VARCHAR2(30)
DB_LINK		VARCHAR2(128)

```
SELECT *  
FROM   user_synonyms;
```

	SYNONYM_NAME	TABLE_OWNER	TABLE_NAME	DB_LINK
1	TEAM2	ORA22	DEPARTMENTS	(null)

Lesson Agenda

- Introduction to data dictionary
- Querying the dictionary views for the following:
 - Table information
 - Column information
 - Constraint information
- Querying the dictionary views for the following:
 - View information
 - Sequence information
 - Synonym information
 - Index information
- Adding a comment to a table and querying the dictionary views for comment information

Adding Comments to a Table

- You can add comments to a table or column by using the COMMENT statement:

```
COMMENT ON TABLE employees  
IS 'Employee Information';
```

```
COMMENT ON COLUMN employees.first_name  
IS 'First name of the employee';
```

- Comments can be viewed through the data dictionary views:
 - ALL_COL_COMMENTS
 - USER_COL_COMMENTS
 - ALL_TAB_COMMENTS
 - USER_TAB_COMMENTS

Quiz

The dictionary views that are based on the dictionary tables contain information such as:

1. Definitions of all the schema objects in the database
2. Default values for the columns
3. Integrity constraint information
4. Oracle usernames and passwords
5. Privileges and roles that each user has been granted

Summary

In this lesson, you should have learned how to find information about your objects through the following dictionary views:

- `DICTIONARY`
- `USER_OBJECTS`
- `USER_TABLES`
- `USER_TAB_COLUMNS`
- `USER_CONSTRAINTS`
- `USER_CONS_COLUMNS`
- `USER_VIEWS`
- `USER_SEQUENCES`
- `USER_INDEXES`
- `USER_SYNONYMS`

Practice 3: Overview

This practice covers the following topics:

- Querying the dictionary views for table and column information
- Querying the dictionary views for constraint information
- Querying the dictionary views for view information
- Querying the dictionary views for sequence information
- Querying the dictionary views for synonym information
- Querying the dictionary views for index information
- Adding a comment to a table and querying the dictionary views for comment information

4

Manipulating Large Data Sets

Objectives

After completing this lesson, you should be able to do the following:

- Manipulate data using subqueries
- Specify explicit default values in the INSERT and UPDATE statements
- Describe the features of multitable INSERTs
- Use the following types of multitable INSERTs:
 - Unconditional INSERT
 - Pivoting INSERT
 - Conditional INSERT ALL
 - Conditional INSERT FIRST
- Merge rows in a table
- Track the changes to data over a period of time

Lesson Agenda

- Manipulating data using subqueries
- Specifying explicit default values in the `INSERT` and `UPDATE` statements
- Using the following types of multitable `INSERT`s:
 - Unconditional `INSERT`
 - Pivoting `INSERT`
 - Conditional `INSERT ALL`
 - Conditional `INSERT FIRST`
- Merging rows in a table
- Tracking the changes to data over a period of time

Using Subqueries to Manipulate Data

You can use subqueries in data manipulation language (DML) statements to:

- Retrieve data using an inline view
- Copy data from one table to another
- Update data in one table based on the values of another table
- Delete rows from one table based on rows in another table

Retrieving Data Using a Subquery as Source

```
SELECT department_name, city
FROM departments
NATURAL JOIN (SELECT l.location_id, l.city, l.country_id
              FROM loc l
              JOIN countries c
              ON(l.country_id = c.country_id)
              JOIN regions USING(region_id)
              WHERE region_name = 'Europe');
```

DEPARTMENT_NAME	CITY
1 Sales	Oxford

Inserting Using a Subquery as a Target

```
INSERT INTO (SELECT l.location_id, l.city, l.country_id
              FROM    loc l
              JOIN    countries c
              ON(l.country_id = c.country_id)
              JOIN regions USING(region_id)
              WHERE region_name = 'Europe')
VALUES (3300, 'Cardiff', 'UK');
```

1 rows inserted

Inserting Using a Subquery as a Target

Verify the results.

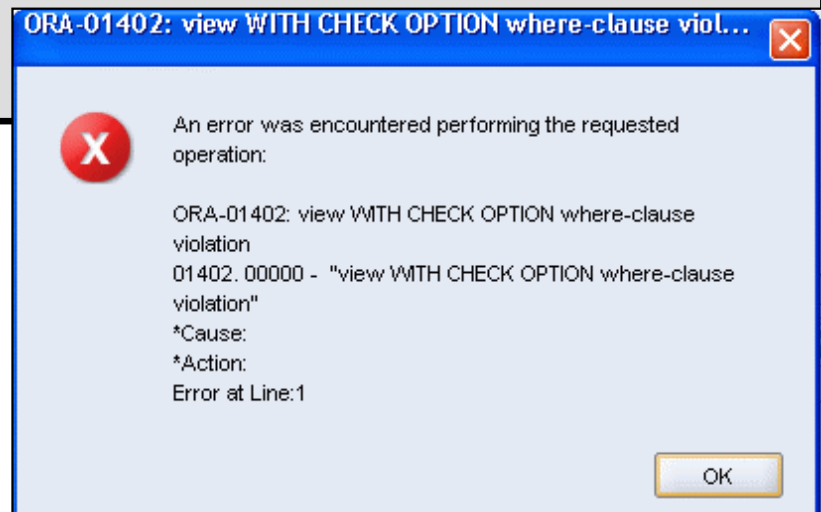
```
SELECT location_id, city, country_id  
FROM    loc
```

	LOCATION_ID	CITY	COUNTRY_ID
1	1400	Southlake	US
2	1500	South San Francisco	US
3	1700	Seattle	US
4	1800	Toronto	CA
5	2500	Oxford	UK
6	3300	Cardiff	UK

Using the WITH CHECK OPTION Keyword on DML Statements

The WITH CHECK OPTION keyword prohibits you from changing rows that are not in the subquery.

```
INSERT INTO ( SELECT location_id, city, country_id
              FROM   loc
              WHERE  country_id IN
                    (SELECT country_id
                     FROM   countries
                     NATURAL JOIN regions
                     WHERE  region name = 'Europe' )
              WITH CHECK OPTION )
VALUES (3600, 'Washington', 'US');
```



Lesson Agenda

- Manipulating data using subqueries
- Specifying explicit default values in the INSERT and UPDATE statements
- Using the following types of multitable INSERTs:
 - Unconditional INSERT
 - Pivoting INSERT
 - Conditional INSERT ALL
 - Conditional INSERT FIRST
- Merging rows in a table
- Tracking the changes to data over a period of time

Overview of the Explicit Default Feature

- Use the `DEFAULT` keyword as a column value where the default column value is desired.
- This allows the user to control where and when the default value should be applied to data.
- Explicit defaults can be used in `INSERT` and `UPDATE` statements.

Using Explicit Default Values

- DEFAULT with INSERT:

```
INSERT INTO deptm3  
  (department_id, department_name, manager_id)  
VALUES (300, 'Engineering', DEFAULT);
```

- DEFAULT with UPDATE:

```
UPDATE deptm3  
SET manager_id = DEFAULT  
WHERE department_id = 10;
```

Copying Rows from Another Table

- Write your INSERT statement with a subquery.

```
INSERT INTO sales_reps(id, name, salary, commission_pct)
  SELECT employee_id, last_name, salary, commission_pct
 FROM   employees
WHERE  job_id LIKE '%REP%';
```

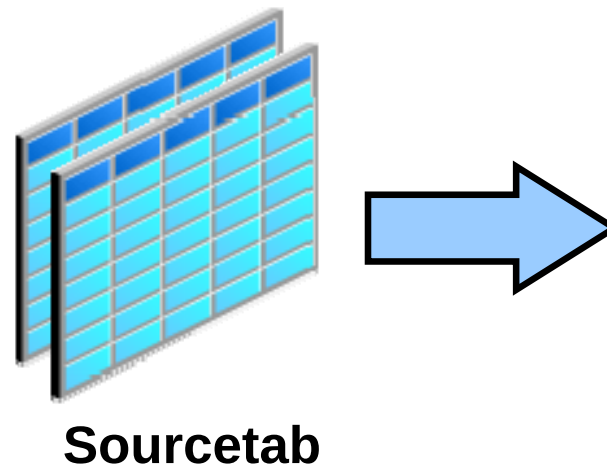
4 rows inserted

- Do not use the VALUES clause.
- Match the number of columns in the INSERT clause with that in the subquery.

Lesson Agenda

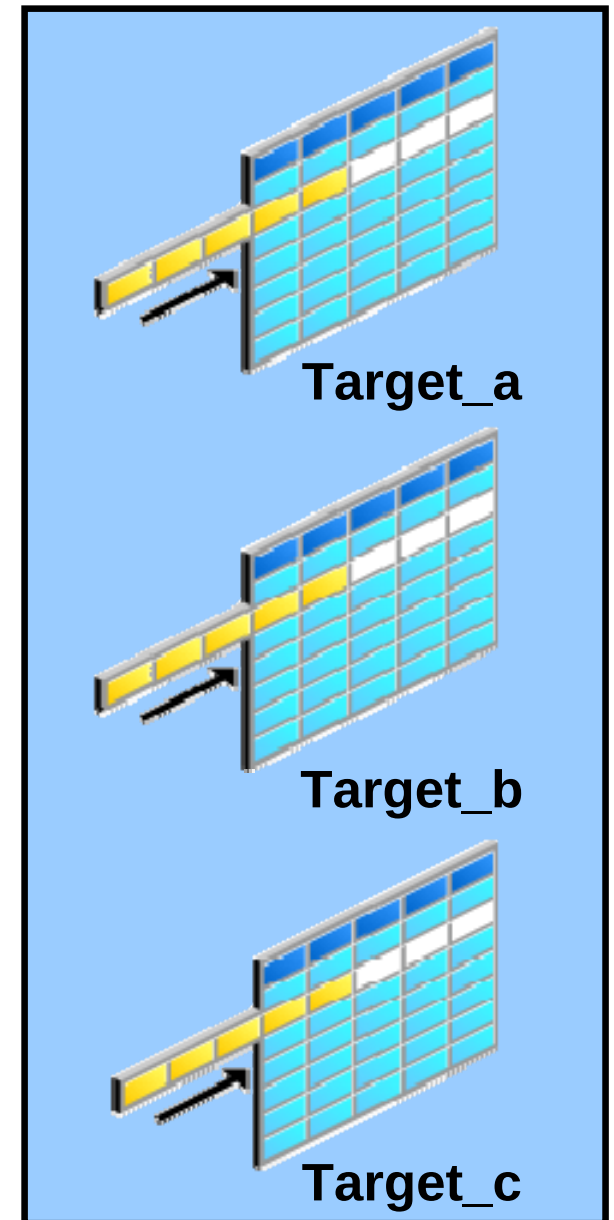
- Manipulating data using subqueries
- Specifying explicit default values in the `INSERT` and `UPDATE` statements
- **Using the following types of multitable `INSERT`s:**
 - Unconditional `INSERT`
 - Pivoting `INSERT`
 - Conditional `INSERT ALL`
 - Conditional `INSERT FIRST`
- Merging rows in a table
- Tracking the changes to data over a period of time

Overview of Multitable INSERT Statements



```
INSERT ALL  
  INTO target_a VALUES (...,...)  
  INTO target_b VALUES (...,...)  
  INTO target_c VALUES (...,...)  
  SELECT ...  
  FROM sourcetab  
  WHERE ...;
```

Subquery



Overview of Multitable INSERT Statements

- Use the `INSERT...SELECT` statement to insert rows into multiple tables as part of a single DML statement.
- Multitable `INSERT` statements are used in data warehousing systems to transfer data from one or more operational sources to a set of target tables.
- They provide significant performance improvement over:
 - Single DML versus multiple `INSERT...SELECT` statements
 - Single DML versus a procedure to perform multiple inserts by using the `IF . . . THEN` syntax

Types of Multitable INSERT Statements

The different types of multitable INSERT statements are:

- Unconditional INSERT
- Conditional INSERT ALL
- Pivoting INSERT
- Conditional INSERT FIRST

Multitable INSERT Statements

- Syntax for multitable INSERT:

```
INSERT [conditional_insert_clause]  
[insert_into_clause values_clause] (subquery)
```

- conditional_insert_clause:

```
[ALL] [FIRST]  
[WHEN condition THEN] [insert_into_clause values_clause]  
[ELSE] [insert_into_clause values_clause]
```

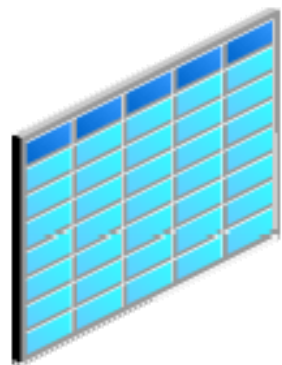
Unconditional INSERT ALL

- Select the EMPLOYEE_ID, HIRE_DATE, SALARY, and MANAGER_ID values from the EMPLOYEES table for those employees whose EMPLOYEE_ID is greater than 200.
- Insert these values into the SAL_HISTORY and MGR_HISTORY tables by using a multitable INSERT.

```
INSERT ALL
  INTO sal_history VALUES (EMPID, HIREDATE, SAL)
  INTO mgr_history VALUES (EMPID, MGR, SAL)
  SELECT employee_id EMPID, hire_date HIREDATE,
         salary SAL, manager_id MGR
  FROM employees
  WHERE employee_id > 200;
```

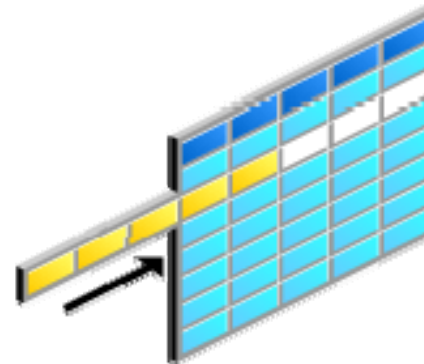
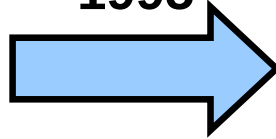
```
8 rows inserted
```

Conditional INSERT ALL: Example

A 3D isometric icon of a table with 5 columns and 10 rows. The top row is dark blue, and the other rows are light blue.

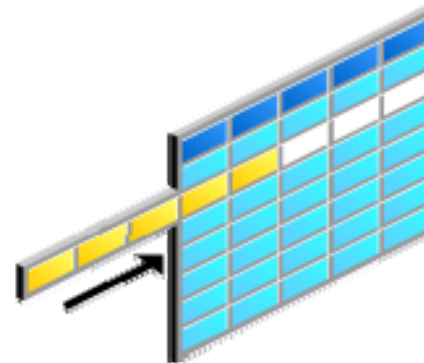
Employees

**Hired before
1995**

A 3D isometric icon of a table with 5 columns and 10 rows. The top row is dark blue, and the other rows are light blue. A yellow bar highlights the first column, and a black arrow points to it.

EMP_HISTORY

**With sales
commission**

A 3D isometric icon of a table with 5 columns and 10 rows. The top row is dark blue, and the other rows are light blue. A yellow bar highlights the first column, and a black arrow points to it.

EMP_SALES

Conditional INSERT ALL

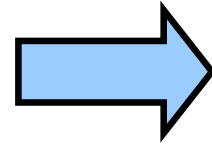
```
INSERT ALL
  WHEN HIREDATE < '01-JAN-95' THEN
    INTO emp_history VALUES (EMPID, HIREDATE, SAL)
  WHEN COMM IS NOT NULL THEN
    INTO emp_sales VALUES (EMPID, COMM, SAL)
  SELECT employee_id EMPID, hire_date HIREDATE,
         salary SAL, commission_pct COMM
  FROM employees
```

12 rows inserted

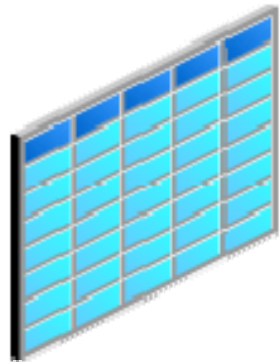
Conditional INSERT FIRST: Example

Scenario: If an employee salary is 2,000, the record is inserted into the SAL_LOW table only.

Salary < 5,000

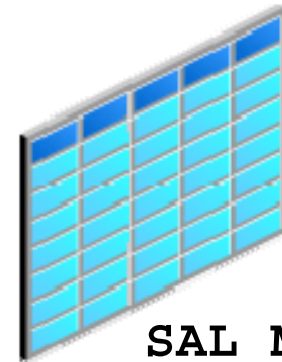


SAL_LOW



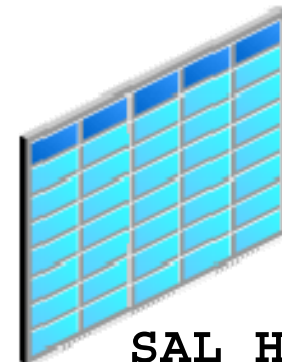
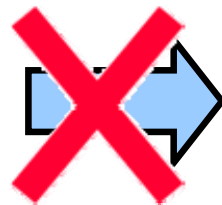
EMPLOYEES

5000 <= Salary
<= 10,000



SAL_MID

Otherwise



SAL_HIGH

Conditional INSERT FIRST

```
INSERT FIRST
```

```
WHEN salary < 5000 THEN
```

```
    INTO sal_low VALUES (employee_id, last_name, salary)
```

```
WHEN salary between 5000 and 10000 THEN
```

```
    INTO sal_mid VALUES (employee_id, last_name, salary)
```

```
ELSE
```

```
    INTO sal_high VALUES (employee_id, last_name, salary)
```

```
SELECT employee_id, last_name, salary
```

```
FROM employees
```

```
20 rows inserted
```

Pivoting INSERT

Convert the set of sales records from the nonrelational database table to relational format.

Emp_ID	Week_ID	MON	TUES	WED	THUR	FRI
176	6	2000	3000	4000	5000	6000



Employee_ID	WEEK	SALES
176	6	2000
176	6	3000
176	6	4000
176	6	5000
176	6	6000

Pivoting INSERT

```
INSERT ALL
  INTO sales_info VALUES (employee_id, week_id, sales_MON)
  INTO sales_info VALUES (employee_id, week_id, sales_TUE)
  INTO sales_info VALUES (employee_id, week_id, sales_WED)
  INTO sales_info VALUES (employee_id, week_id, sales_THUR)
  INTO sales_info VALUES (employee_id, week_id, sales_FRI)
SELECT EMPLOYEE_ID, week_id, sales_MON, sales_TUE,
       sales_WED, sales_THUR, sales_FRI
FROM sales_source_data;
```

5 rows inserted

Lesson Agenda

- Manipulating data using subqueries
- Specifying explicit default values in the `INSERT` and `UPDATE` statements
- Using the following types of multitable `INSERT`s:
 - Unconditional `INSERT`
 - Pivoting `INSERT`
 - Conditional `INSERT ALL`
 - Conditional `INSERT FIRST`
- **Merging rows in a table**
- Tracking the changes to data over a period of time

MERGE Statement

- Provides the ability to conditionally update, insert, or delete data into a database table
- Performs an `UPDATE` if the row exists, and an `INSERT` if it is a new row:
 - Avoids separate updates
 - Increases performance and ease of use
 - Is useful in data warehousing applications

MERGE Statement Syntax

You can conditionally insert, update, or delete rows in a table by using the MERGE statement.

```
MERGE INTO table_name table_alias
  USING (table/view/sub_query) alias
  ON (join condition)
  WHEN MATCHED THEN
    UPDATE SET
      col1 = col1_val,
      col2 = col2_val
  WHEN NOT MATCHED THEN
    INSERT (column_list)
    VALUES (column_values);
```

Merging Rows: Example

Insert or update rows in the COPY_EMP3 table to match the EMPLOYEES table.

```
MERGE INTO copy_emp3 c
USING (SELECT * FROM EMPLOYEES ) e
ON (c.employee_id = e.employee_id)
WHEN MATCHED THEN
UPDATE SET
c.first_name = e.first_name,
c.last_name = e.last_name,
...
DELETE WHERE (E.COMMISSION_PCT IS NOT NULL)
WHEN NOT MATCHED THEN
INSERT VALUES(e.employee_id, e.first_name, e.last_name,
e.email, e.phone_number, e.hire_date, e.job_id,
e.salary, e.commission_pct, e.manager_id,
e.department_id);
```

Merging Rows: Example

```
TRUNCATE TABLE copy_emp3;  
SELECT * FROM copy_emp3;  
0 rows selected
```

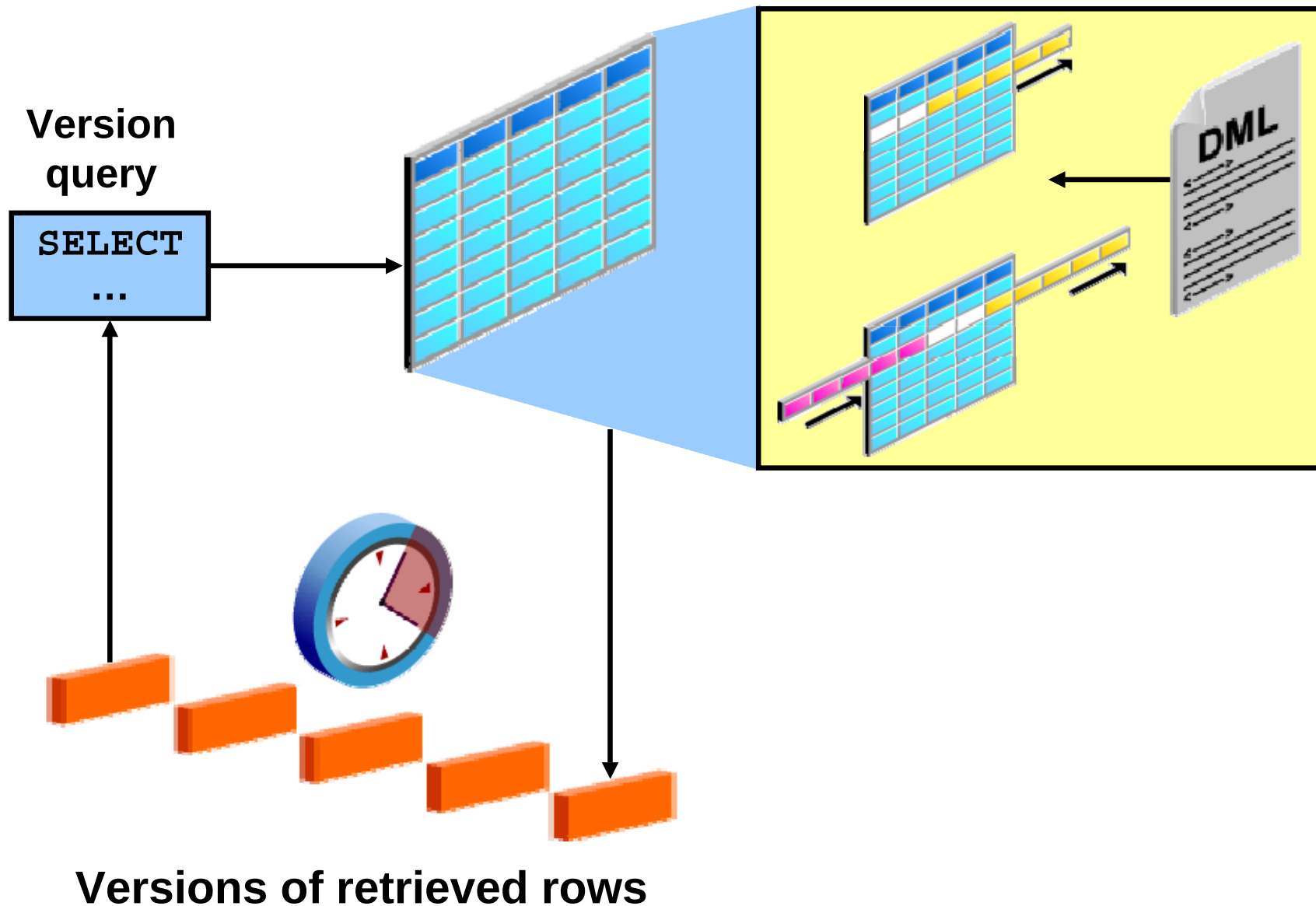
```
MERGE INTO copy_emp3 c  
USING (SELECT * FROM EMPLOYEES ) e  
ON (c.employee_id = e.employee_id)  
WHEN MATCHED THEN  
UPDATE SET  
c.first_name = e.first_name,  
c.last_name = e.last_name,  
...  
DELETE WHERE (E.COMMISSION_PCT IS NOT NULL)  
WHEN NOT MATCHED THEN  
INSERT VALUES(e.employee_id, e.first_name, ...
```

```
SELECT * FROM copy_emp3;  
20 rows selected.
```

Lesson Agenda

- Manipulating data using subqueries
- Specifying explicit default values in the `INSERT` and `UPDATE` statements
- Using the following types of multitable `INSERT`s:
 - Unconditional `INSERT`
 - Pivoting `INSERT`
 - Conditional `INSERT ALL`
 - Conditional `INSERT FIRST`
- Merging rows in a table
- Tracking the changes to data over a period of time

Tracking Changes in Data



Example of the Flashback Version Query

```
SELECT salary FROM employees3  
WHERE employee_id = 107;
```

1

```
UPDATE employees3 SET salary = salary * 1.30  
WHERE employee_id = 107;
```

2

```
COMMIT;
```

```
SELECT salary FROM employees3  
  VERSIONS BETWEEN SCN MINVALUE AND MAXVALUE  
WHERE employee_id = 107;
```

3

1

	AZ	SALARY
1		4200

3

	AZ	SALARY
1		5460
2		4200

VERSIONS BETWEEN Clause

```
SELECT versions_starttime "START_DATE",
       versions_endtime   "END_DATE",
       salary
FROM   employees
       VERSIONS BETWEEN SCN MINVALUE
       AND MAXVALUE
WHERE  last_name = 'Lorentz';
```

	START_DATE	END_DATE	 SALARY
1	10-JUL-07 10.55.49.0000000000	(null)	5460
2	(null)	10-JUL-07 10.55.49.0000000000	4200

Quiz

When you use the `INSERT` or `UPDATE` command, the `DEFAULT` keyword saves you from hard-coding the default value in your programs or querying the dictionary to find it.

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Use DML statements and control transactions
- Describe the features of multitable INSERTs
- Use the following types of multitable INSERTs:
 - Unconditional INSERT
 - Pivoting INSERT
 - Conditional INSERT ALL
 - Conditional INSERT FIRST
- Merge rows in a table
- Manipulate data by using subqueries
- Track the changes to data over a period of time

Practice 4: Overview

This practice covers the following topics:

- Performing multitable `INSERTs`
- Performing `MERGE` operations
- Tracking row versions

A large, light gray number '5' is positioned in the background, centered vertically and horizontally.

Managing Data in Different Time Zones

Objectives

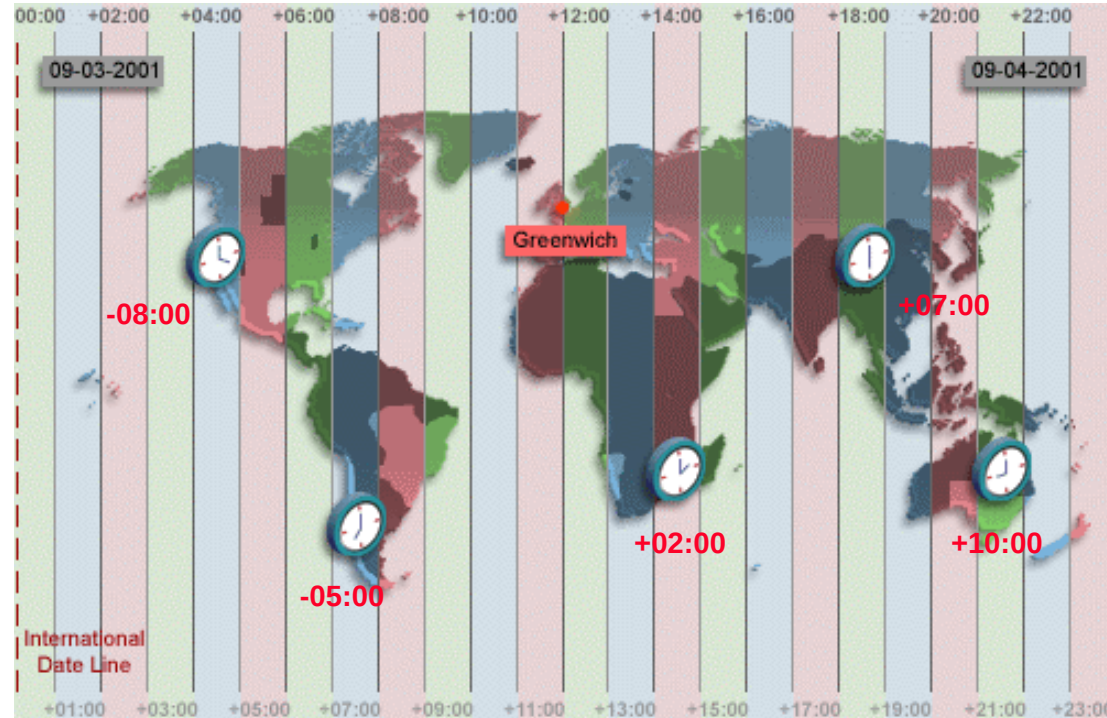
After completing this lesson, you should be able to do the following:

- Use data types similar to `DATE` that store fractional seconds and track time zones
- Use data types that store the difference between two datetime values
- Use the following datetime functions:
 - `CURRENT_DATE`
 - `CURRENT_TIMESTAMP`
 - `LOCALTIMESTAMP`
 - `DBTIMEZONE`
 - `SESSIONTIMEZONE`
 - `EXTRACT`
 - `TZ_OFFSET`
 - `FROM_TZ`
 - `TO_TIMESTAMP`
 - `TO_YMINTERVAL`
 - `TO_DSINTERVAL`

Lesson Agenda

- `CURRENT_DATE`, `CURRENT_TIMESTAMP`,
and `LOCALTIMESTAMP`
- `INTERVAL` data types
- Using the following functions:
 - `EXTRACT`
 - `TZ_OFFSET`
 - `FROM_TZ`
 - `TO_TIMESTAMP`
 - `TO_YMINTERVAL`
 - `TO_DSINTERVAL`

Time Zones



The image represents the time for each time zone when Greenwich time is 12:00.

TIME_ZONE Session Parameter

TIME_ZONE may be set to:

- An absolute offset
- Database time zone
- OS local time zone
- A named region

```
ALTER SESSION SET TIME_ZONE = '-05:00';  
ALTER SESSION SET TIME_ZONE = dbtimezone;  
ALTER SESSION SET TIME_ZONE = local;  
ALTER SESSION SET TIME_ZONE = 'America/New_York';
```

CURRENT_DATE, CURRENT_TIMESTAMP, and LOCALTIMESTAMP

- **CURRENT_DATE:**
 - Returns the current date from the user session
 - Has a data type of DATE
- **CURRENT_TIMESTAMP:**
 - Returns the current date and time from the user session
 - Has a data type of `TIMESTAMP WITH TIME ZONE`
- **LOCALTIMESTAMP:**
 - Returns the current date and time from the user session
 - Has a data type of `TIMESTAMP`

Comparing Date and Time in a Session's Time Zone

The `TIME_ZONE` parameter is set to `-5:00` and then `SELECT` statements for each date and time is executed to compare differences.

```
ALTER SESSION
SET NLS_DATE_FORMAT = 'DD-MON-YYYY HH24:MI:SS';
ALTER SESSION SET TIME_ZONE = '-5:00';
SELECT SESSIONTIMEZONE, CURRENT_DATE FROM DUAL;
SELECT SESSIONTIMEZONE, CURRENT_TIMESTAMP FROM DUAL;
SELECT SESSIONTIMEZONE, LOCALTIMESTAMP FROM DUAL;
```

Comparing Date and Time in a Session's Time Zone

Results of queries:

alter session set succeeded.	
SESSIONTIMEZONE	CURRENT_DATE
-----	-----
-05:00	11-JUN-2007 14:12:06
1 rows selected	
SESSIONTIMEZONE	CURRENT_TIMESTAMP
-----	-----
-05:00	11-JUN-07 02.12.06.076200000 PM -05:00
1 rows selected	
SESSIONTIMEZONE	LOCALTIMESTAMP
-----	-----
-05:00	11-JUN-07 02.12.06.164828000 PM
1 rows selected	

DBTIMEZONE and SESSIONTIMEZONE

- Display the value of the database time zone:

```
SELECT DBTIMEZONE FROM DUAL;
```

DBTIMEZONE	
1	+00:00

- Display the value of the session's time zone:

```
SELECT SESSIONTIMEZONE FROM DUAL;
```

SESSIONTIMEZONE	
1	-05:00

TIMESTAMP Data Types

Data Type	Fields
TIMESTAMP	Year, Month, Day, Hour, Minute, Second with fractional seconds
TIMESTAMP WITH TIME ZONE	Same as the TIMESTAMP data type; also includes: TIMEZONE_HOUR, and TIMEZONE_MINUTE or TIMEZONE_REGION
TIMESTAMP WITH LOCAL TIME ZONE	Same as the TIMESTAMP data type; also includes a time zone offset in its value

TIMESTAMP Fields

Datetime Field	Valid Values
YEAR	–4712 to 9999 (excluding year 0)
MONTH	01 to 12
DAY	01 to 31
HOURL	00 to 23
MINUTE	00 to 59
SECOND	00 to 59.9(N) where 9(N) is precision
TIMEZONE_HOUR	–12 to 14
TIMEZONE_MINUTE	00 to 59

Difference Between DATE and TIMESTAMP

A

```
-- when hire_date is  
of type DATE
```

```
SELECT hire_date  
FROM employees;
```

	HIRE_DATE
1	17-JUN-87
2	21-SEP-89
3	13-JAN-93
4	03-JAN-90
5	21-MAY-91
6	25-JUN-97
7	05-FEB-98
8	07-FEB-99
9	17-AUG-94
10	16-AUG-94

B

```
ALTER TABLE employees  
MODIFY hire_date TIMESTAMP;
```

```
SELECT hire_date  
FROM employees;
```

	HIRE_DATE
1	17-JUN-87 12.00.00.0000000000 AM
2	21-SEP-89 12.00.00.0000000000 AM
3	13-JAN-93 12.00.00.0000000000 AM
4	03-JAN-90 12.00.00.0000000000 AM
5	21-MAY-91 12.00.00.0000000000 AM
6	25-JUN-97 12.00.00.0000000000 AM
7	05-FEB-98 12.00.00.0000000000 AM
8	07-FEB-99 12.00.00.0000000000 AM
9	17-AUG-94 12.00.00.0000000000 AM
10	16-AUG-94 12.00.00.0000000000 AM

...

Comparing TIMESTAMP Data Types

```
CREATE TABLE web_orders  
(order_date TIMESTAMP WITH TIME ZONE,  
 delivery_time TIMESTAMP WITH LOCAL TIME ZONE);
```

```
INSERT INTO web_orders values  
(current_date, current_timestamp + 2);
```

```
SELECT * FROM web_orders;
```

	ORDER_DATE	DELIVERY_TIME
1	11-JUN-07 02.44.14.000000000 PM -05:00	13-JUN-07 02.44.14.000000000 PM

Lesson Agenda

- CURRENT_DATE, CURRENT_TIMESTAMP, and LOCALTIMESTAMP
- INTERVAL data types
- Using the following functions:
 - EXTRACT
 - TZ_OFFSET
 - FROM_TZ
 - TO_TIMESTAMP
 - TO_YMINTERVAL
 - TO_DSINTERVAL

INTERVAL Data Types

- INTERVAL data types are used to store the difference between two datetime values.
- There are two classes of intervals:
 - Year-month
 - Day-time
- The precision of the interval is:
 - The actual subset of fields that constitutes an interval
 - Specified in the interval qualifier

Data Type	Fields
INTERVAL YEAR TO MONTH	Year, Month
INTERVAL DAY TO SECOND	Days, Hour, Minute, Second with fractional seconds

INTERVAL Fields

INTERVAL Field	Valid Values for Interval
YEAR	Any positive or negative integer
MONTH	00 to 11
DAY	Any positive or negative integer
HOUR	00 to 23
MINUTE	00 to 59
SECOND	00 to 59.9(N) where 9(N) is precision

INTERVAL YEAR TO MONTH: Example

```
CREATE TABLE warranty
(prod_id number,  warranty_time INTERVAL YEAR(3) TO
MONTH);

INSERT INTO warranty VALUES (123, INTERVAL '8' MONTH);
INSERT INTO warranty VALUES (155, INTERVAL '200'
YEAR(3));
INSERT INTO warranty VALUES (678, '200-11');
SELECT * FROM warranty;
```

	PROD_ID	WARRANTY_TIME
1	123	0-8
2	155	200-0
3	678	200-11

INTERVAL DAY TO SECOND

Data Type: Example

```
CREATE TABLE lab
( exp_id number, test_time INTERVAL DAY(2) TO SECOND);

INSERT INTO lab VALUES (100012, '90 00:00:00');

INSERT INTO lab VALUES (56098,
    INTERVAL '6 03:30:16' DAY TO SECOND);
```

```
SELECT * FROM lab;
```

	EXP_ID	TEST_TIME
1	10012	90 0:0:0.0
2	56098	6 3:30:16.0

Lesson Agenda

- CURRENT_DATE, CURRENT_TIMESTAMP, and LOCALTIMESTAMP
- INTERVAL data types
- Using the following functions:
 - EXTRACT
 - TZ_OFFSET
 - FROM_TZ
 - TO_TIMESTAMP
 - TO_YMINTERVAL
 - TO_DSINTERVAL

EXTRACT

- Display the YEAR component from the SYSDATE.

```
SELECT EXTRACT (YEAR FROM SYSDATE) FROM DUAL;
```

EXTRACT(YEARFROMSYSDATE)
1 2007

- Display the MONTH component from the HIRE_DATE for those employees whose MANAGER_ID is 100.

```
SELECT last name, hire date,  
       EXTRACT (MONTH FROM HIRE_DATE)  
FROM employees  
WHERE manager_id = 100;
```

	LAST_NAME	HIRE_DATE	EXTRACT(MONTHFROMHIRE_DATE)
1	Kochhar	21-SEP-89 12.00.00.0000000000 AM	9
2	De Haan	13-JAN-93 12.00.00.0000000000 AM	1
3	Raphaely	07-DEC-94 12.00.00.0000000000 AM	12
4	Weiss	18-JUL-96 12.00.00.0000000000 AM	7

TZ_OFFSET

Display the time zone offset for the 'US/Eastern',
'Canada/Yukon' and 'Europe/London' time zones:

```
SELECT TZ_OFFSET('US/Eastern'),  
       TZ_OFFSET('Canada/Yukon'),  
       TZ_OFFSET('Europe/London')  
FROM DUAL;
```

 TZ_OFFSET('US/EASTERN')	 TZ_OFFSET('CANADA/YUKON')	 TZ_OFFSET('EUROPE/LONDON')
1 -04:00	-07:00	+01:00

FROM_TZ

Display the `TIMESTAMP` value `'2000-03-28 08:00:00'` as a `TIMESTAMP WITH TIME ZONE` value for the `'Australia/North'` time zone region.

```
SELECT FROM_TZ(TIMESTAMP
               '2000-07-12 08:00:00', 'Australia/North')
FROM DUAL;
```

FROM_TZ(TIMESTAMP'2000-07-12 08:00:00','AUSTRALIA/NORTH')
1 12-JUL-00 08.00.00.000000000 AM AUSTRALIA/NORTH

TO_TIMESTAMP

Display the character string '2007-03-06 11:00:00'
as a TIMESTAMP value:

```
SELECT TO_TIMESTAMP ('2007-03-06 11:00:00',  
                     'YYYY-MM-DD HH:MI:SS')  
FROM DUAL;
```

```
TO_TIMESTAMP('2007-03-0611:00:00','YYYY-MM-DDHH:MI:SS')  
06-MAR-07 11.00.00.0000000000
```

TO_YMINTERVAL

Display a date that is one year and two months after the hire date for the employees working in the department with the DEPARTMENT_ID 20.

```
SELECT hire_date,  
       hire_date + TO_YMINTERVAL('01-02') AS  
       HIRE_DATE_YMININTERVAL  
FROM   employees  
WHERE  department_id = 20;
```

	HIRE_DATE	HIRE_DATE_YMININTERVAL
1	17-FEB-96 00.00.00.0000000000	17-APR-97 00.00.00.0000000000
2	17-AUG-97 00.00.00.0000000000	17-OCT-98 00.00.00.0000000000

TO_DSINTERVAL

Display a date that is 100 days and 10 hours after the hire date for all the employees.

```
SELECT last_name,  
       TO_CHAR(hire_date, 'mm-dd-yy:hh:mi:ss') hire_date,  
       TO_CHAR(hire_date +  
               TO_DSINTERVAL('100 10:00:00'),  
               'mm-dd-yy:hh:mi:ss') hiredate2  
FROM employees;
```

	 LAST_NAME	 HIRE_DATE	 HIREDATE2
1	King	06-17-87:12:00:00	09-25-87:10:00:00
2	Kochhar	09-21-89:12:00:00	12-30-89:10:00:00
3	De Haan	01-13-93:12:00:00	04-23-93:10:00:00
4	Hunold	01-03-90:12:00:00	04-13-90:10:00:00
5	Ernst	05-21-91:12:00:00	08-29-91:10:00:00
6	Lorentz	02-07-99:12:00:00	05-18-99:10:00:00

...

Daylight Saving Time

- First Sunday in April
 - Time jumps from 01:59:59 AM to 03:00:00 AM.
 - Values from 02:00:00 AM to 02:59:59 AM are not valid.
- Last Sunday in October
 - Time jumps from 02:00:00 AM to 01:00:01 AM.
 - Values from 01:00:01 AM to 02:00:00 AM are ambiguous because they are visited twice.

Quiz

The `TIME_ZONE` session parameter may be set to:

1. A relative offset
2. Database time zone
3. OS local time zone
4. A named region

Summary

In this lesson, you should have learned how to use the following functions:

- `CURRENT_DATE`
- `CURRENT_TIMESTAMP`
- `LOCALTIMESTAMP`
- `DBTIMEZONE`
- `SESSIONTIMEZONE`
- `EXTRACT`
- `TZ_OFFSET`
- `FROM_TZ`
- `TO_TIMESTAMP`
- `TO_YMINTERVAL`
- `TO_DSINTERVAL`

Practice 5: Overview

This practice covers using the datetime functions.

6

Retrieving Data Using Subqueries

Objectives

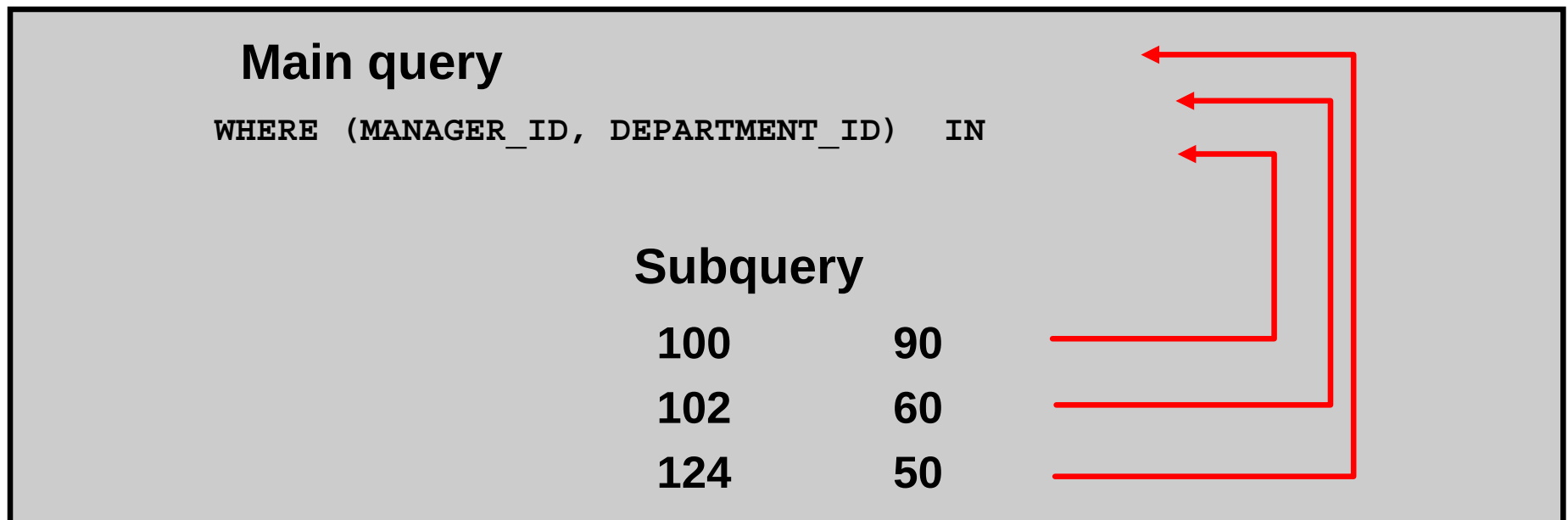
After completing this lesson, you should be able to do the following:

- Write a multiple-column subquery
- Use scalar subqueries in SQL
- Solve problems with correlated subqueries
- Update and delete rows using correlated subqueries
- Use the `EXISTS` and `NOT EXISTS` operators
- Use the `WITH` clause

Lesson Agenda

- Writing a multiple-column subquery
- Using scalar subqueries in SQL
- Solving problems with correlated subqueries
- Using the EXISTS and NOT EXISTS operators
- Using the WITH clause

Multiple-Column Subqueries



Each row of the main query is compared to values from a multiple-row and multiple-column subquery.

Column Comparisons

Multiple-column comparisons involving subqueries can be:

- Nonpairwise comparisons
- Pairwise comparisons

Pairwise Comparison Subquery

Display the details of the employees who are managed by the same manager and work in the same department as employees with the first name of “John.”

```
SELECT employee_id, manager_id, department_id
FROM   empl_demo
WHERE  (manager_id, department_id) IN
      (SELECT manager_id, department_id
       FROM empl_demo
       WHERE first_name = 'John')
AND first_name <> 'John';
```


Nonpairwise Comparison Subquery

Display the details of the employees who are managed by the same manager as the employees with the first name of “John” and work in the same department as the employees with the first name of “John.”

```
SELECT  employee_id, manager_id, department_id
FROM    empl_demo
WHERE   manager_id IN
        (SELECT manager_id
         FROM empl_demo
         WHERE first_name = 'John')
AND department_id IN
        (SELECT department_id
         FROM empl_demo
         WHERE first_name = 'John')
AND first_name <> 'John';
```

Lesson Agenda

- Writing a multiple-column subquery
- **Using scalar subqueries in SQL**
- Solving problems with correlated subqueries
- Using the EXISTS and NOT EXISTS operators
- Using the WITH clause

Scalar Subquery Expressions

- A scalar subquery expression is a subquery that returns exactly one column value from one row.
- Scalar subqueries can be used in:
 - The condition and expression part of `DECODE` and `CASE`
 - All clauses of `SELECT` except `GROUP BY`
 - The `SET` clause and `WHERE` clause of an `UPDATE` statement

Scalar Subqueries: Examples

- Scalar subqueries in CASE expressions:

```
SELECT employee_id, last_name,  
       (CASE  
         WHEN department_id = 20  
           (SELECT department_id  
            FROM departments  
            WHERE location_id = 1800)  
         THEN 'Canada' ELSE 'USA' END) location  
FROM   employees;
```

- Scalar subqueries in the ORDER BY clause:

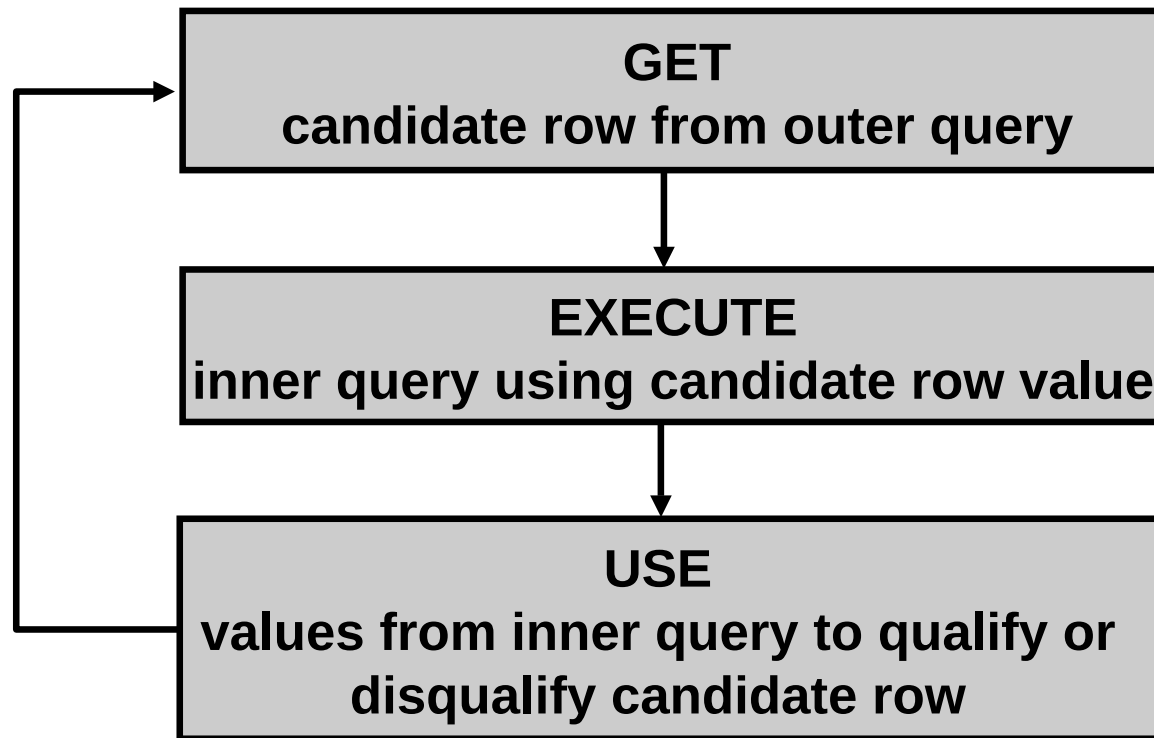
```
SELECT   employee_id, last_name  
FROM     employees e  
ORDER BY (SELECT department_name  
          FROM departments d  
          WHERE e.department_id = d.department_id);
```

Lesson Agenda

- Writing a multiple-column subquery
- Using scalar subqueries in SQL
- Solving problems with correlated subqueries
- Using the EXISTS and NOT EXISTS operators
- Using the WITH clause

Correlated Subqueries

Correlated subqueries are used for row-by-row processing. Each subquery is executed once for every row of the outer query.



Correlated Subqueries

The subquery references a column from a table in the parent query.

```
SELECT column1, column2, ...
FROM   table1  outer
WHERE  column1 operator
              (SELECT column1, column2
                 FROM   table2
                 WHERE  expr1 =
                      outer.expr2) ;
```

Using Correlated Subqueries

Find all employees who earn more than the average salary in their department.

```
SELECT last_name, salary, department_id
FROM   employees outer
WHERE  salary >
      (SELECT AVG(salary)
       FROM   employees
       WHERE  department_id =
             outer.department_id);
```



Each time a row from the outer query is processed, the inner query is evaluated.

Using Correlated Subqueries

Display details of those employees who have changed jobs at least twice.

```
SELECT e.employee_id, last_name, e.job_id
FROM   employees e
WHERE  2 <= (SELECT COUNT(*)
              FROM   job_history
              WHERE  employee_id = e.employee_id);
```

	EMPLOYEE_ID	LAST_NAME	JOB_ID
1	101	Kochhar	AD_VP
2	176	Taylor	SA_REP
3	200	Whalen	AD_ASST

Lesson Agenda

- Writing a multiple-column subquery
- Using scalar subqueries in SQL
- Solving problems with correlated subqueries
- **Using the EXISTS and NOT EXISTS operators**
- Using the WITH clause

Using the EXISTS Operator

- The EXISTS operator tests for existence of rows in the results set of the subquery.
- If a subquery row value is found:
 - The search does not continue in the inner query
 - The condition is flagged TRUE
- If a subquery row value is not found:
 - The condition is flagged FALSE
 - The search continues in the inner query

Find Employees Who Have at Least One Person Reporting to Them

```
SELECT employee_id, last_name, job_id, department_id
FROM   employees outer
WHERE  EXISTS ( SELECT 'X'
                FROM   employees
                WHERE  manager_id =
                      outer.employee_id);
```

	EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90
4	103	Hunold	IT_PROG	60
5	124	Mourgos	ST_MAN	50
6	149	Zlotkey	SA_MAN	80
7	201	Hartstein	MK_MAN	20
8	205	Higgins	AC_MGR	110

Find All Departments That Do Not Have Any Employees

```
SELECT department_id, department_name
FROM departments d
WHERE NOT EXISTS (SELECT 'X'
                  FROM employees
                  WHERE department_id = d.department_id);
```

	DEPARTMENT_ID	DEPARTMENT_NAME
1	190	Contracting

Correlated UPDATE

Use a correlated subquery to update rows in one table based on rows from another table.

```
UPDATE table1 alias1
SET    column = (SELECT expression
                     FROM    table2 alias2
                     WHERE   alias1.column =
                             alias2.column);
```

Using Correlated UPDATE

- Denormalize the EMPL6 table by adding a column to store the department name.
- Populate the table by using a correlated update.

```
ALTER TABLE empl6  
ADD(department_name VARCHAR2(25));
```

```
UPDATE empl6 e  
SET      department_name =  
          (SELECT department_name  
           FROM   departments d  
           WHERE  e.department_id = d.department_id);
```

Correlated DELETE

Use a correlated subquery to delete rows in one table based on rows from another table.

```
DELETE FROM table1 alias1
WHERE  column operator
        (SELECT expression
         FROM   table2 alias2
         WHERE  alias1.column = alias2.column);
```


Using Correlated DELETE

Use a correlated subquery to delete only those rows from the EMPL6 table that also exist in the EMP_HISTORY table.

```
DELETE FROM empl6 E
WHERE employee_id =
      (SELECT employee_id
       FROM   emp_history
       WHERE  employee_id = E.employee_id);
```

Lesson Agenda

- Writing a multiple-column subquery
- Using scalar subqueries in SQL
- Solving problems with correlated subqueries
- Using the EXISTS and NOT EXISTS operators
- **Using the WITH clause**

WITH Clause

- Using the `WITH` clause, you can use the same query block in a `SELECT` statement when it occurs more than once within a complex query.
- The `WITH` clause retrieves the results of a query block and stores it in the user's temporary tablespace.
- The `WITH` clause may improve performance.

WITH Clause: Example

Using the `WITH` clause, write a query to display the department name and total salaries for those departments whose total salary is greater than the average salary across departments.

WITH Clause: Example

```
WITH
dept_costs AS (
    SELECT d.department_name, SUM(e.salary) AS dept_total
    FROM   employees e JOIN departments d
    ON     e.department_id = d.department_id
    GROUP BY d.department_name),
avg_cost AS (
    SELECT SUM(dept_total)/COUNT(*) AS dept_avg
    FROM   dept_costs)
SELECT *
FROM   dept_costs
WHERE  dept_total >
      (SELECT dept_avg
       FROM avg_cost)
ORDER BY department_name;
```

Quiz

With a correlated subquery, the inner `SELECT` statement drives the outer `SELECT` statement.

1. True
2. False

Summary

In this lesson, you should have learned that:

- A multiple-column subquery returns more than one column
- Multiple-column comparisons can be pairwise or nonpairwise
- A multiple-column subquery can also be used in the `FROM` clause of a `SELECT` statement

Summary

- Correlated subqueries are useful whenever a subquery must return a different result for each candidate row
- The `EXISTS` operator is a Boolean operator that tests the presence of a value
- Correlated subqueries can be used with `SELECT`, `UPDATE`, and `DELETE` statements
- You can use the `WITH` clause to use the same query block in a `SELECT` statement when it occurs more than once

Practice 6: Overview

This practice covers the following topics:

- Creating multiple-column subqueries
- Writing correlated subqueries
- Using the `EXISTS` operator
- Using scalar subqueries
- Using the `WITH` clause



Regular Expression Support

Objectives

After completing this lesson, you should be able to do the following:

- List the benefits of using regular expressions
- Use regular expressions to search for, match, and replace strings

Lesson Agenda

- Introduction to regular expressions
- Using metacharacters with regular expressions
- Using the regular expressions functions
 - REGEXP_LIKE
 - REGEXP_REPLACE
 - REGEXP_INSTR
 - REGEXP_SUBSTR
- Accessing subexpressions
- Using the REGEXP_COUNT function
- Regular expressions and check constraints

What Are Regular Expressions?

- You use regular expressions to search for (and manipulate) simple and complex patterns in string data by using standard syntax conventions.
- You use a set of SQL functions and conditions to search for and manipulate strings in SQL and PL/SQL.
- You specify a regular expression by using:
 - Metacharacters, which are operators that specify the search algorithms
 - Literals, which are the characters for which you are searching

Benefits of Using Regular Expressions

Regular expressions enable you to implement complex match logic in the database with the following benefits:

- By centralizing match logic in Oracle Database, you avoid intensive string processing of SQL results sets by middle-tier applications.
- Using server-side regular expressions to enforce constraints, you eliminate the need to code data validation logic on the client.
- The built-in SQL and PL/SQL regular expression functions and conditions make string manipulations more powerful and easier than in previous releases of Oracle Database 10g.

Using the Regular Expressions Functions and Conditions in SQL and PL/SQL

Function or Condition Name	Description
REGEXP_LIKE	Is similar to the LIKE operator, but performs regular expression matching instead of simple pattern matching (condition)
REGEXP_REPLACE	Searches for a regular expression pattern and replaces it with a replacement string
REGEXP_INSTR	Searches a string for a regular expression pattern and returns the position where the match is found
REGEXP_SUBSTR	Searches for a regular expression pattern within a given string and extracts the matched substring
REGEXP_COUNT	Returns the number of times a pattern match is found in an input string

Lesson Agenda

- Introduction to regular expressions
- Using metacharacters with regular expressions
- Using the regular expressions functions
 - REGEXP_LIKE
 - REGEXP_REPLACE
 - REGEXP_INSTR
 - REGEXP_SUBSTR
- Accessing subexpressions
- Using the REGEXP_COUNT function

What Are Metacharacters?

- Metacharacters are special characters that have a special meaning such as a wildcard, a repeating character, a nonmatching character, or a range of characters.
- You can use several predefined metacharacter symbols in the pattern matching.
- For example, the `^(f|ht)tps?:$` regular expression searches for the following from the beginning of the string:
 - The literals `f` or `ht`
 - The `t` literal
 - The `p` literal, optionally followed by the `s` literal
 - The colon “`:`” literal at the end of the string

Using Metacharacters with Regular Expressions

Syntax	Description
.	Matches any character in the supported character set, except NULL
+	Matches one or more occurrences
?	Matches zero or one occurrence
*	Matches zero or more occurrences of the preceding subexpression
{ <i>m</i> }	Matches exactly <i>m</i> occurrences of the preceding expression
{ <i>m</i> , }	Matches at least <i>m</i> occurrences of the preceding subexpression
{ <i>m</i> , <i>n</i> }	Matches at least <i>m</i> , but not more than <i>n</i> , occurrences of the preceding subexpression
[...]	Matches any single character in the list within the brackets
	Matches one of the alternatives
(. . .)	Treats the enclosed expression within the parentheses as a unit. The subexpression can be a string of literals or a complex expression containing operators.

Using Metacharacters with Regular Expressions

Syntax	Description
<code>^</code>	Matches the beginning of a string
<code>\$</code>	Matches the end of a string
<code>\</code>	Treats the subsequent metacharacter in the expression as a literal
<code>\n</code>	Matches the <i>n</i> th (1–9) preceding subexpression of whatever is grouped within parentheses. The parentheses cause an expression to be remembered; a backreference refers to it.
<code>\d</code>	A digit character
<code>[:class:]</code>	Matches any character belonging to the specified POSIX character class
<code>[^:class:]</code>	Matches any single character <i>not</i> in the list within the brackets

Lesson Agenda

- Introduction to regular expressions
- Using metacharacters with regular expressions
- **Using the regular expressions functions**
 - REGEXP_LIKE
 - REGEXP_REPLACE
 - REGEXP_INSTR
 - REGEXP_SUBSTR
- Accessing subexpressions
- Using the REGEXP_COUNT function

Regular Expressions Functions and Conditions: Syntax

```
REGEXP_LIKE (source_char, pattern [,match_option]
```

```
REGEXP_INSTR (source_char, pattern [, position  
[, occurrence [, return_option  
[, match_option [, subexpr]]]])
```

```
REGEXP_SUBSTR (source_char, pattern [, position  
[, occurrence [, match_option  
[, subexpr]]]])
```

```
REGEXP_REPLACE(source_char, pattern [,replacestr  
[, position [, occurrence  
[, match_option]]]])
```

```
REGEXP_COUNT (source_char, pattern [, position  
[, occurrence [, match_option]]])
```

Performing a Basic Search Using the REGEXP_LIKE Condition

```
REGEXP_LIKE(source_char, pattern [, match_parameter ])
```

```
SELECT first_name, last_name  
FROM employees  
WHERE REGEXP_LIKE (first_name, '^Ste(v|ph)en$');
```

	 FIRST_NAME	 LAST_NAME
1	Steven	King

Replacing Patterns

Using the REGEXP_REPLACE Function

```
REGEXP_REPLACE(source_char, pattern [,replacestr  
[, position [, occurrence [, match_option]]])
```

```
SELECT REGEXP_REPLACE(phone_number, '\.','-') AS phone  
FROM employees;
```

Original

	A Z	LAST_NAME	A Z	PHONE_NUMBER
1		King		515.123.4567
2		Kochhar		515.123.4568
3		De Haan		515.123.4569
4		Hunold		590.423.4567



Partial results

	A Z	LAST_NAME	A Z	PHONE
1		King		515-123-4567
2		Kochhar		515-123-4568
3		De Haan		515-123-4569
4		Hunold		590-423-4567

Finding Patterns

Using the REGEXP_INSTR Function

```
REGEXP_INSTR (source_char, pattern [, position [,  
occurrence [, return_option [, match_option]]])
```

```
SELECT street_address,  
REGEXP_INSTR(street_address,'[[:alpha:]]') AS  
    First_Alpha_Position  
FROM locations;
```

	STREET_ADDRESS	FIRST_ALPHA_POSITION
1	2014 Jabberwocky Rd	6
2	2011 Interiors Blvd	6
3	2004 Charade Rd	6
4	460 Bloor St. W.	5
5	Magdalen Centre, The Oxford Science Park	1

Extracting Substrings

Using the REGEXP_SUBSTR Function

```
REGEXP_SUBSTR (source_char, pattern [, position  
                [, occurrence [, match_option]])
```

```
SELECT REGEXP_SUBSTR(street_address , ' [^ ]+ ') AS Road  
FROM locations;
```

	 ROAD
1	Jabberwocky
2	Interiors
3	Charade
4	Bloor
5	Centre,

Lesson Agenda

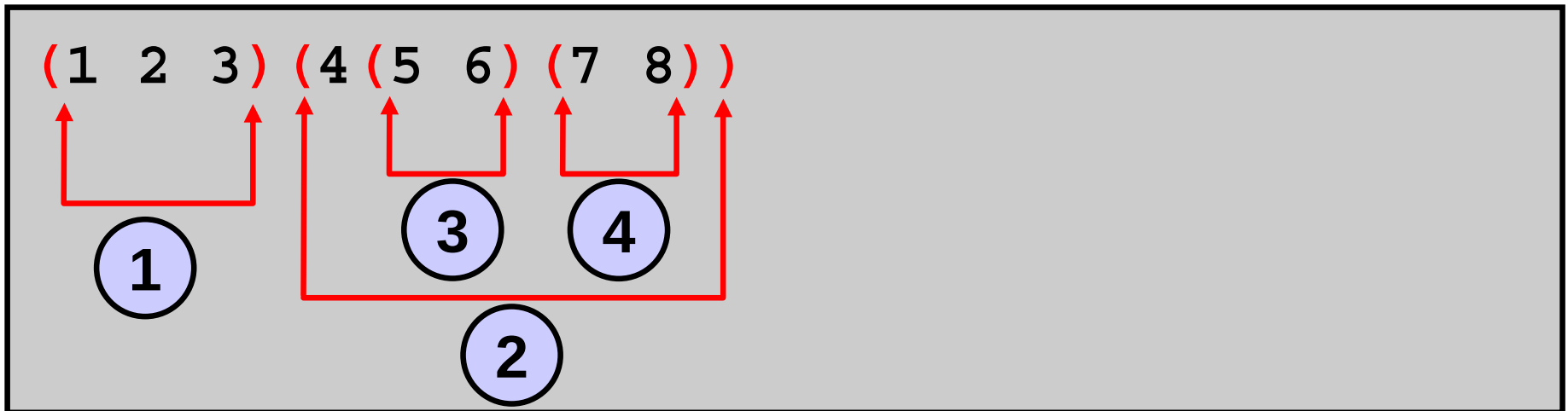
- Introduction to regular expressions
- Using metacharacters with regular expressions
- Using the regular expressions functions
 - REGEXP_LIKE
 - REGEXP_REPLACE
 - REGEXP_INSTR
 - REGEXP_SUBSTR
- **Accessing subexpressions**
- Using the REGEXP_COUNT function

Subexpressions

Examine this expression:

```
(1 2 3) (4 (5 6) (7 8))
```

The subexpressions are:



Using Subexpressions with Regular Expression Support

```
SELECT
  REGEXP_INSTR
① ('0123456789',      -- source char or search value
② '(123)(4(56)(78))', -- regular expression patterns
③ 1,                  -- position to start searching
④ 1,                  -- occurrence
⑤ 0,                  -- return option
⑥ 'i',                -- match option (case insensitive)
⑦ 1)                  -- sub-expression on which to search
  "Position"
FROM dual;
```

Position	
1	2

Why Access the *n*th Subexpression?

- A more realistic use: DNA sequencing
- You may need to find a specific subpattern that identifies a protein needed for immunity in the mouse DNA.

```
SELECT
  REGEXP_INSTR('ccacctttccctccactcctcacgttctcacctgtaaagcgtccctc
cctcatccccatgcccccttaccctgcagggtagagtaggctagaaaccagagagctccaagc
tccatctgtggagaggtgccatccttgggctgcagagagaggagaatttgccccaagctgcc
tgcagagcttcaccacccttagtctcacaagccttgagttcatagcatttcttgagttttca
ccctgcccagcaggacactgcagcacccaaagggcttcccaggagtaggggttgccctcaagag
gctcttgggtctgatggccacatcctggaattgttttcaagttgatgggtcacagccctgaggc
atgtagggggcgtggggatgcgctctgctctgctctcctctcctgaaccctgaaccctctggc
taccagagcacttagagccag',
               '(gtc(tcac)(aaag))',
               1, 1, 0, 'i',
               1) "Position"

FROM dual;
```

Position	
1	195

REGEXP_SUBSTR: Example

```
SELECT
  REGEXP_SUBSTR
    ① ('acgctgcactgca', -- source char or search value
    ② 'acg(.*)gca',      -- regular expression pattern
    ③ 1,                  -- position to start searching
    ④ 1,                  -- occurrence
    ⑤ 'i',                -- match option (case insensitive)
    ⑥ 1)                 -- sub-expression
    "Value"
FROM dual;
```

	Value
1	ctgcact

Lesson Agenda

- Introduction to regular expressions
- Using metacharacters with regular expressions
- Using the regular expressions functions
 - REGEXP_LIKE
 - REGEXP_REPLACE
 - REGEXP_INSTR
 - REGEXP_SUBSTR
- Accessing subexpressions
- Using the REGEXP_COUNT function

Using the REGEXP_COUNT Function

```
REGEXP_COUNT (source_char, pattern [, position  
               [, occurrence [, match_option]])
```

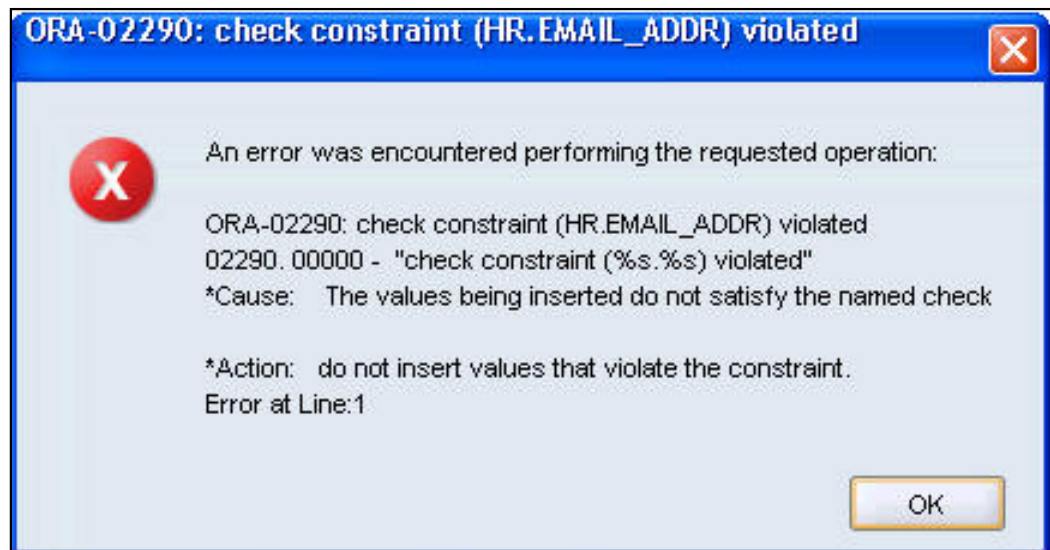
```
SELECT REGEXP_COUNT(  
    'ccacctttccctccactcctcacgttctcacctgtaaagcgtccctccctcatccccatgcccccttaccctgcag  
    ggtagagtaggctagaaaccagagagctccaagctccatctgtggagaggtgccatccttgggctgcagagagaggag  
    aatttgcccaaagctgcctgcagagcttcaccacccttagtctcaciaaagccttgagttcatagcatttcttgagtt  
    ttcaccctgcccagcaggacactgcagcacccaaagggcttcccaggagtagggttgccctcaagaggctcttgggct  
    tgatggccacatcctggaattgttttcaagttgatggtcacagccctgaggcatgtaggggcgtggggatgcgctctg  
    ctctgctctcctctcctgaaccctgaaccctctggctaccccagagcacttagagccag',  
    'gtc') AS Count  
FROM dual;
```

	COUNT
1	4

Regular Expressions and Check Constraints: Examples

```
ALTER TABLE emp8  
ADD CONSTRAINT email_addr  
CHECK(REGEXP_LIKE(email, '@')) NOVALIDATE;
```

```
INSERT INTO emp8 VALUES  
(500, 'Christian', 'Patel', 'ChrisP2creme.com',  
1234567890, '12-Jan-2004', 'HR_REP', 2000, null, 102, 40);
```



Quiz

With the use of regular expressions in SQL and PL/SQL, you can:

1. Avoid intensive string processing of SQL result sets by middle-tier applications
2. Avoid data validation logic on the client
3. Enforce constraints on the server

Summary

In this lesson, you should have learned how to use regular expressions to search for, match, and replace strings.

Practice 7: Overview

This practice covers using regular expressions functions to do the following:

- Searching for, replacing, and manipulating data
- Creating a new `CONTACTS` table and adding a `CHECK` constraint to the `p_number` column to ensure that phone numbers are entered into the database in a specific standard format
- Testing the adding of some phone numbers into the `p_number` column by using various formats

Table Descriptions



Using SQL Developer

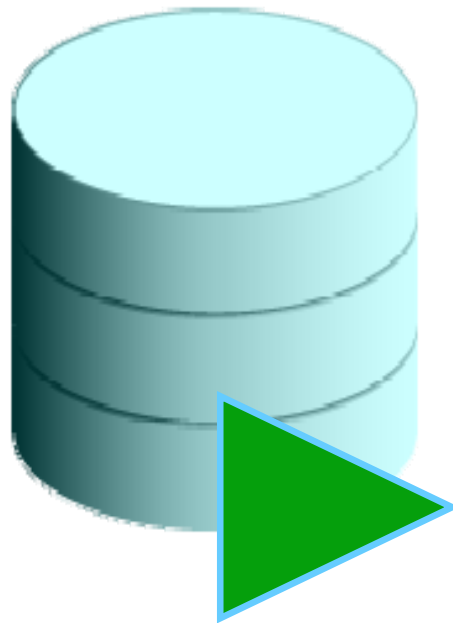
Objectives

After completing this appendix, you should be able to do the following:

- List the key features of Oracle SQL Developer
- Install Oracle SQL Developer 1.2.1
- Identify menu items of Oracle SQL Developer
- Create a database connection
- Manage database objects
- Use SQL Worksheet
- Save and run SQL scripts
- Create and save reports
- Install and use Oracle SQL Developer 1.5.3

What Is Oracle SQL Developer?

- Oracle SQL Developer is a graphical tool that enhances productivity and simplifies database development tasks.
- You can connect to any target Oracle database schema by using standard Oracle database authentication.



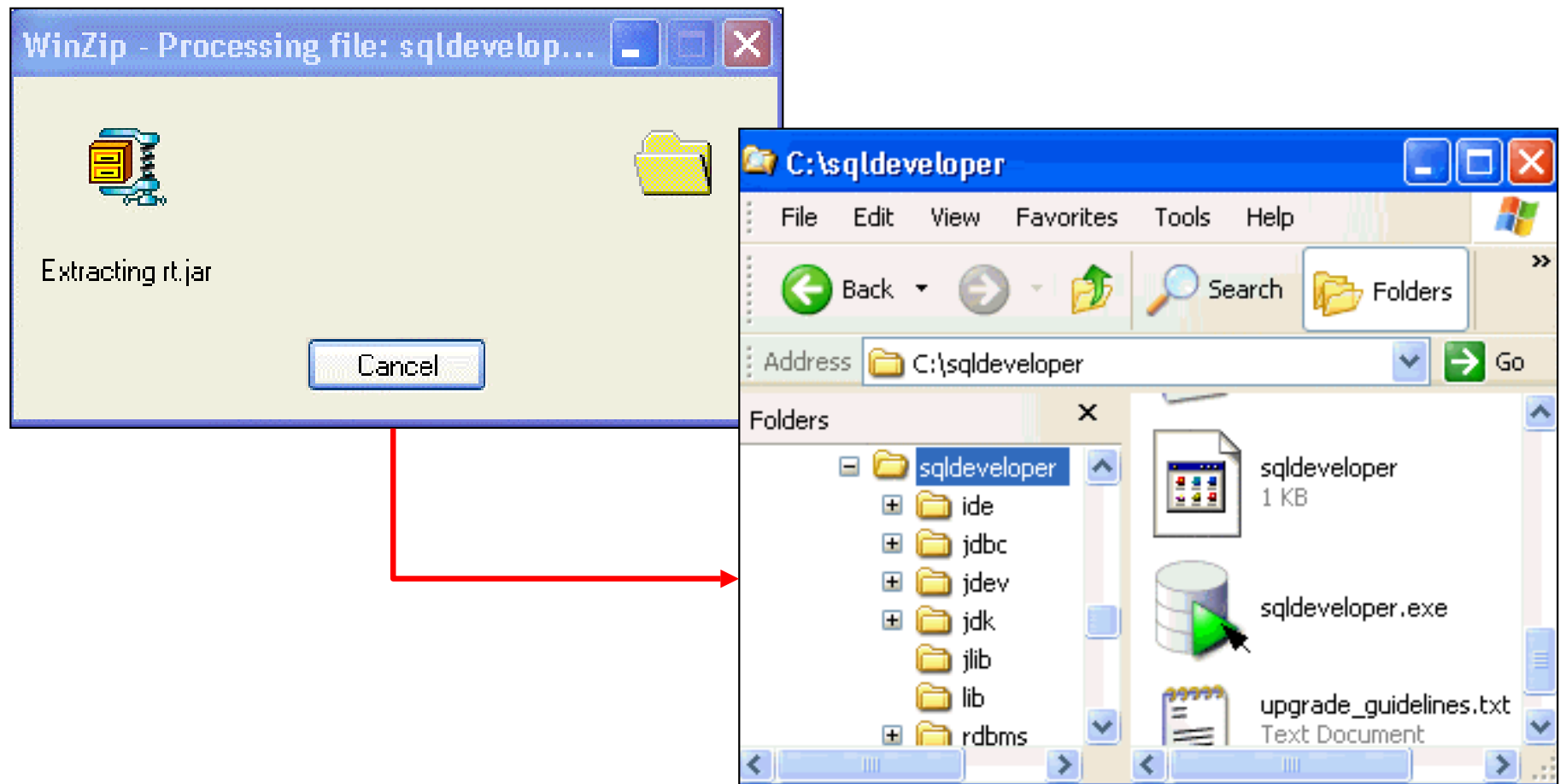
SQL Developer

Specifications of SQL Developer

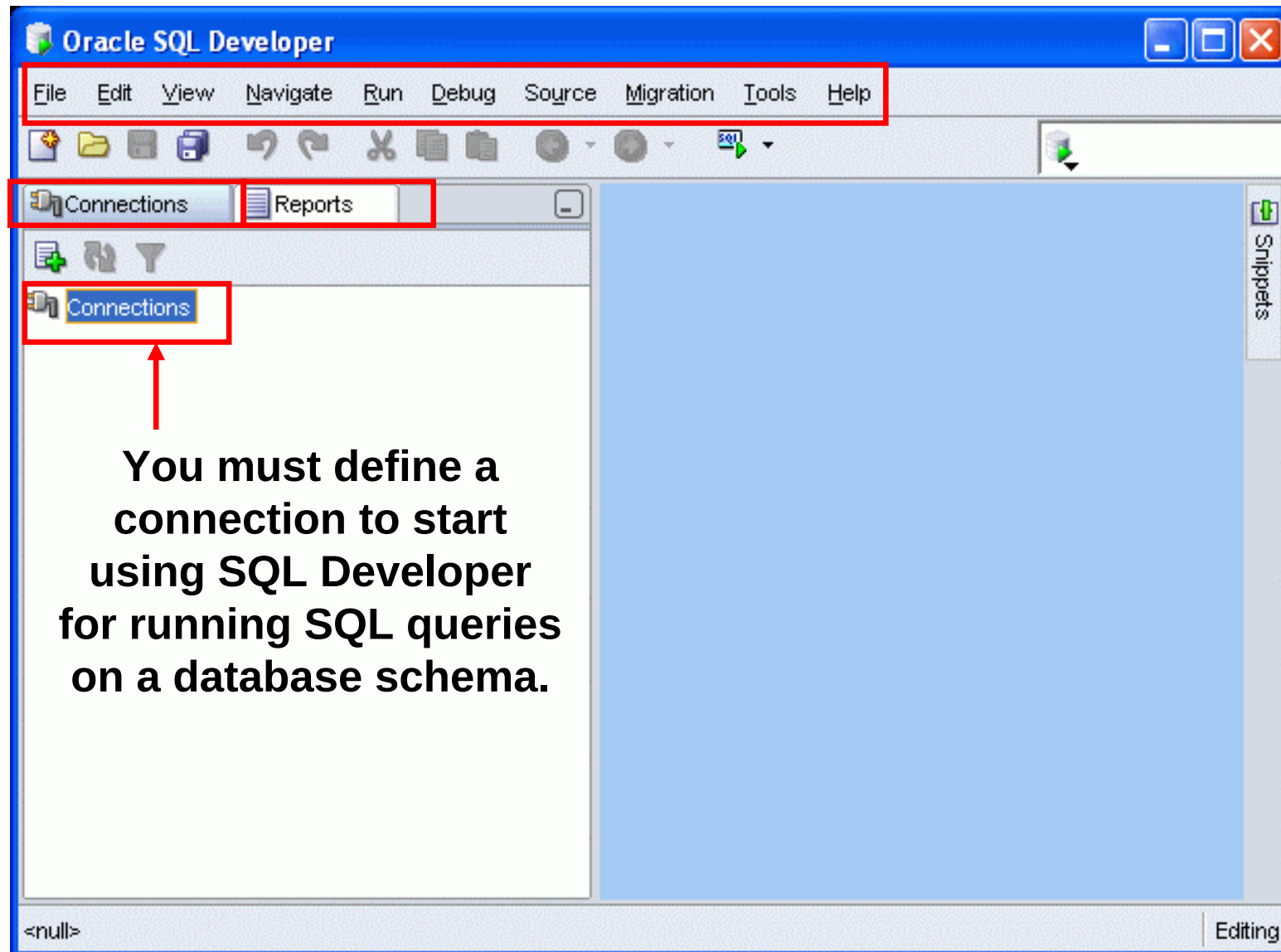
- Developed in Java
- Supports Windows, Linux, and Mac OS X platforms
- Default connectivity by using the JDBC Thin driver
- Does not require an installer
 - Unzip the downloaded SQL Developer kit and double-click `sqldeveloper.exe` to start SQL Developer.
- Connects to Oracle Database version 9.2.0.1 and later
- Freely downloadable from the following link:
 - http://www.oracle.com/technology/products/database/sql_developer/index.html
- Needs JDK 1.5 installed on your system that can be downloaded from the following link:
 - http://java.sun.com/javase/downloads/index_jdk5.jsp

Installing SQL Developer

Download the Oracle SQL Developer kit and unzip it into any directory on your machine.



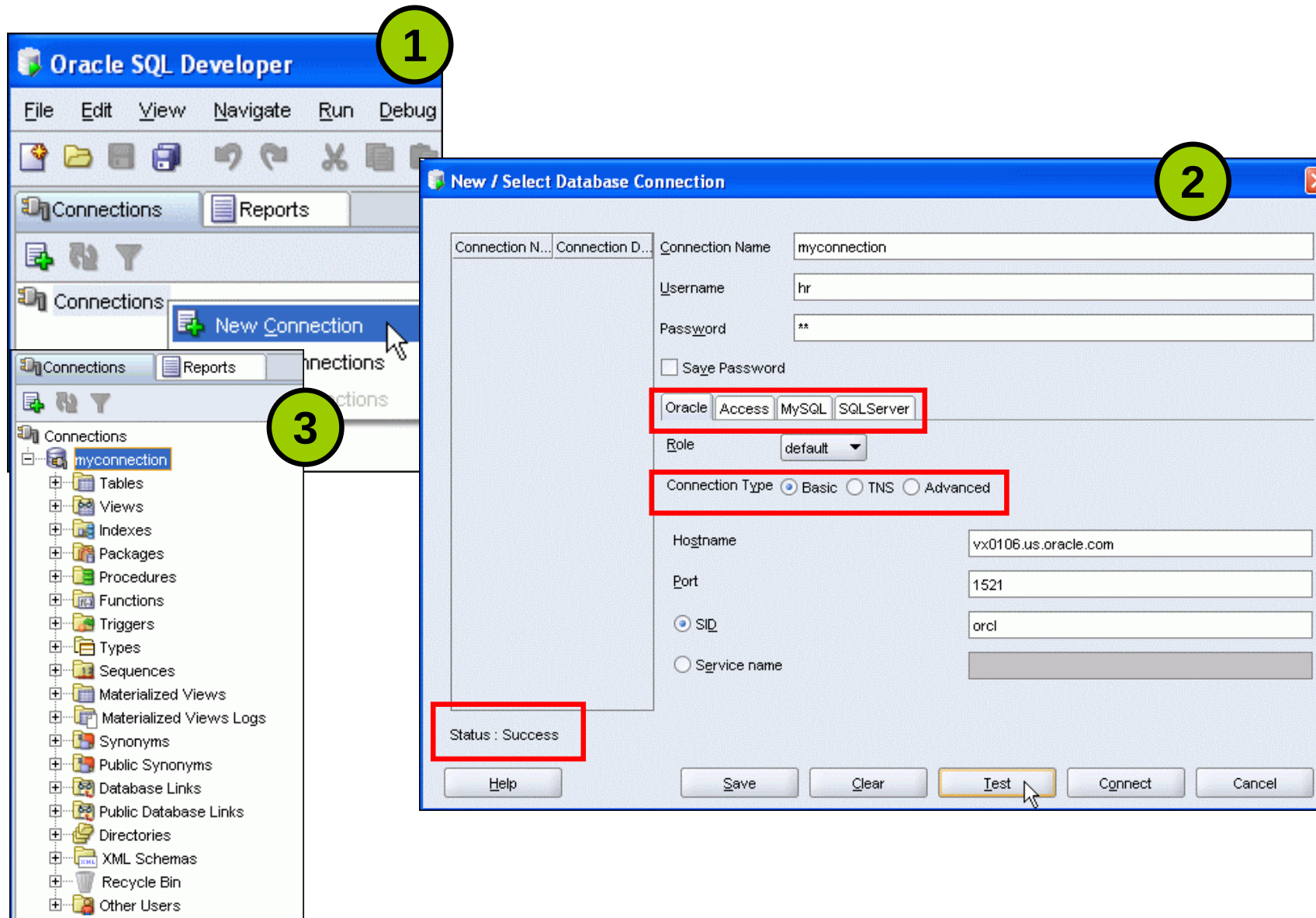
SQL Developer 1.2 Interface



Creating a Database Connection

- You must have at least one database connection to use SQL Developer.
- You can create and test connections for:
 - Multiple databases
 - Multiple schemas
- SQL Developer automatically imports any connections defined in the `tnsnames.ora` file on your system.
- You can export connections to an Extensible Markup Language (XML) file.
- Each additional database connection created is listed in the Connections Navigator hierarchy.

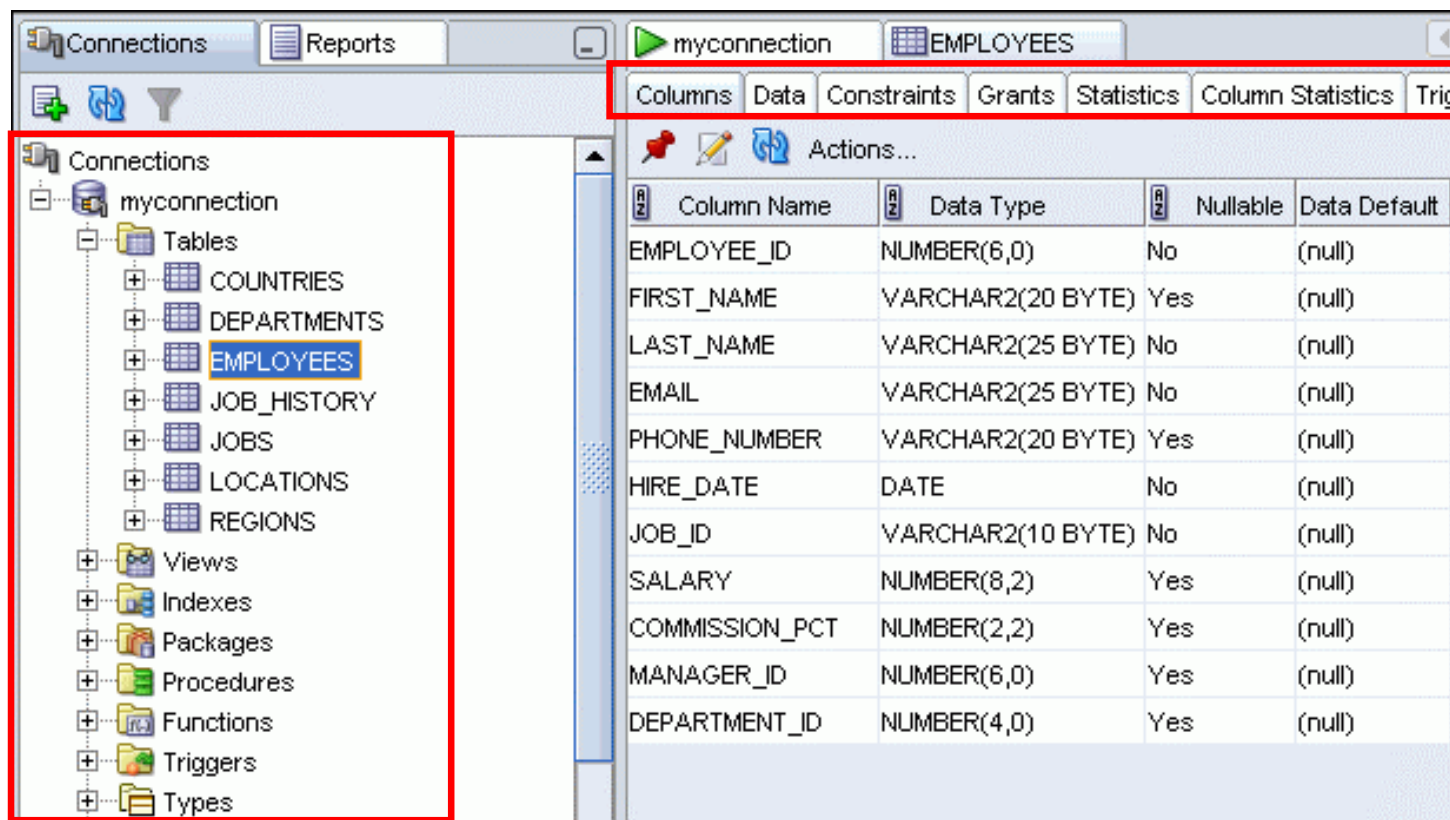
Creating a Database Connection



Browsing Database Objects

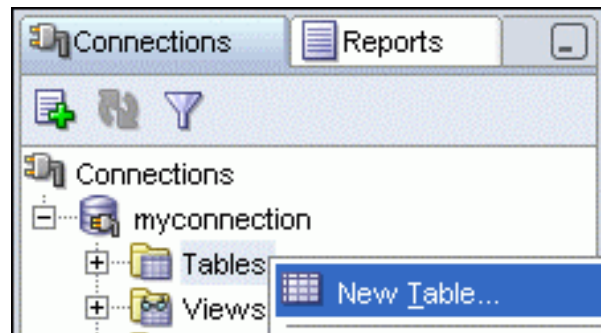
Use the Connections Navigator to:

- Browse through many objects in a database schema
- Review the definitions of objects at a glance



Creating a Schema Object

- SQL Developer supports the creation of any schema object by:
 - Executing a SQL statement in SQL Worksheet
 - Using the context menu
- Edit the objects by using an edit dialog box or one of the many context-sensitive menus.
- View the data definition language (DDL) for adjustments such as creating a new object or editing an existing schema object.



Creating a New Table: Example

Create Table

Schema:

Name:

Table Type: ☒ Normal ☐ External ☐ Index Organized ☐ Temporary (Transaction) ☐ Temporary (Session)

☒ Advanced

Columns

- Primary Key
- Unique Constraints
- Foreign Keys
- Check Constraints
- Indexes
- Column Sequences
- Storage Options
- Lob Parameters
- Partitioning
 - Partition Definitions
 - Subpartition Templates
- Comment
- DDL

Columns:

- ID
- FIRST_NAME
- LAST_NAME
- RELATION
- BIRTHDATE

Column Properties

Name:

Datatype: ☒ Simple ☐ Complex

Type:

Precision:

Scale:

Default:

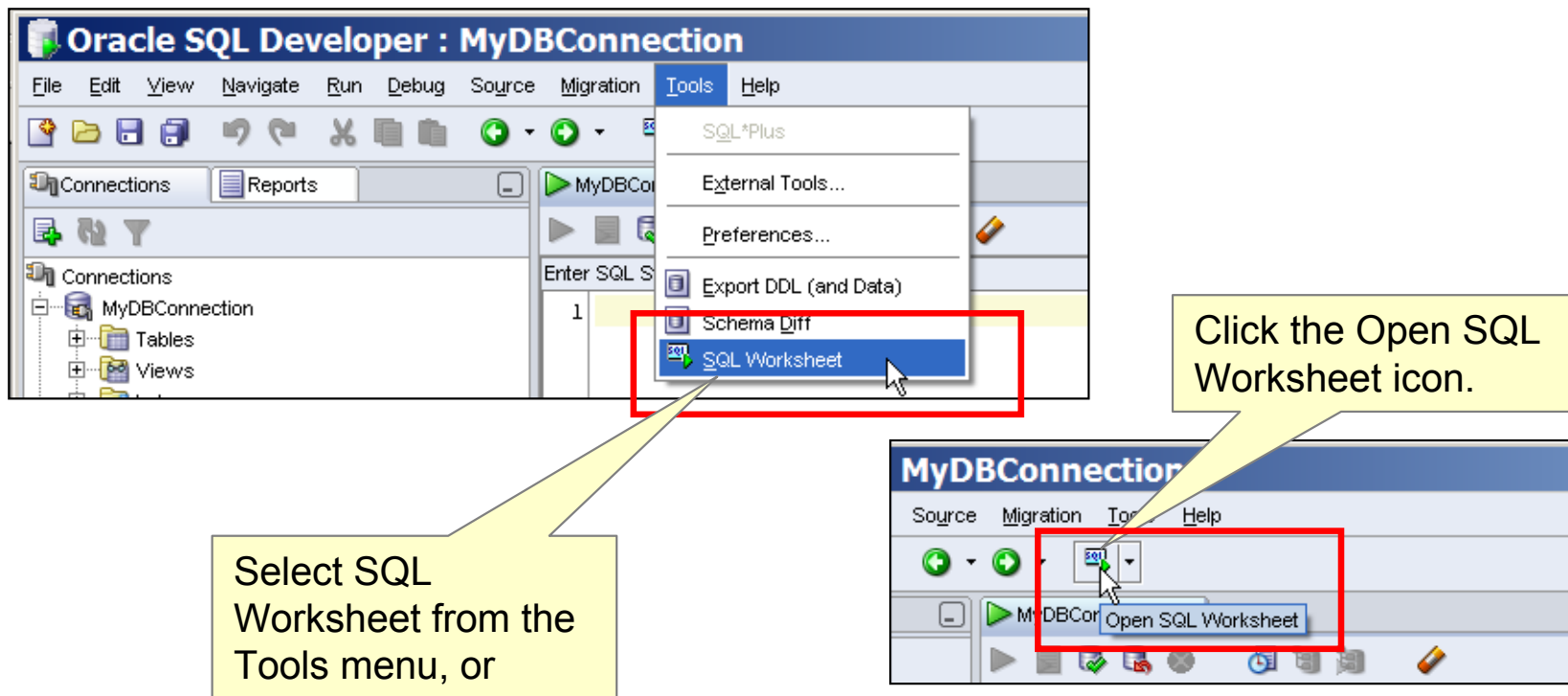
☒ Cannot be NULL

Comment:

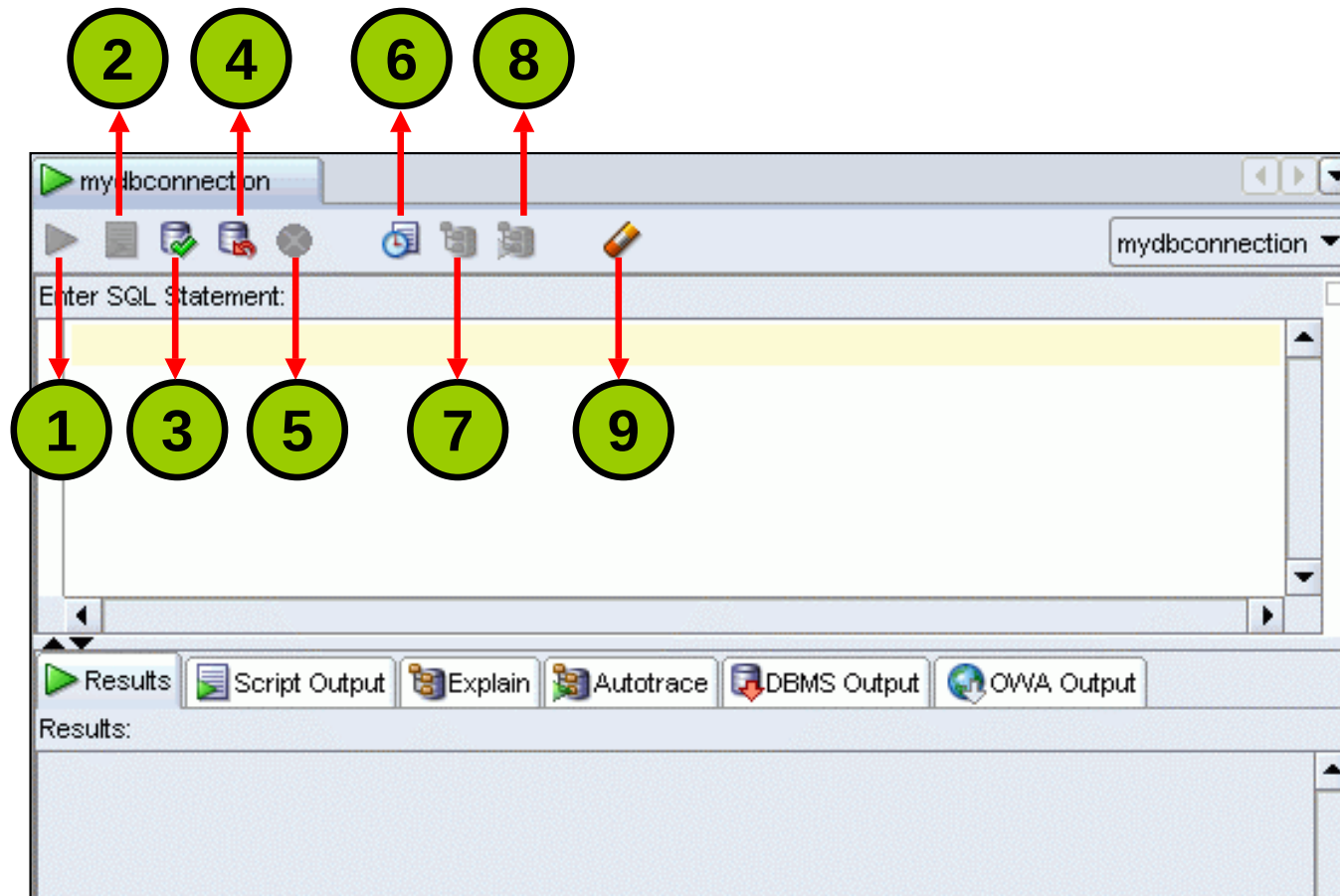
Help OK Cancel

Using the SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL *Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.

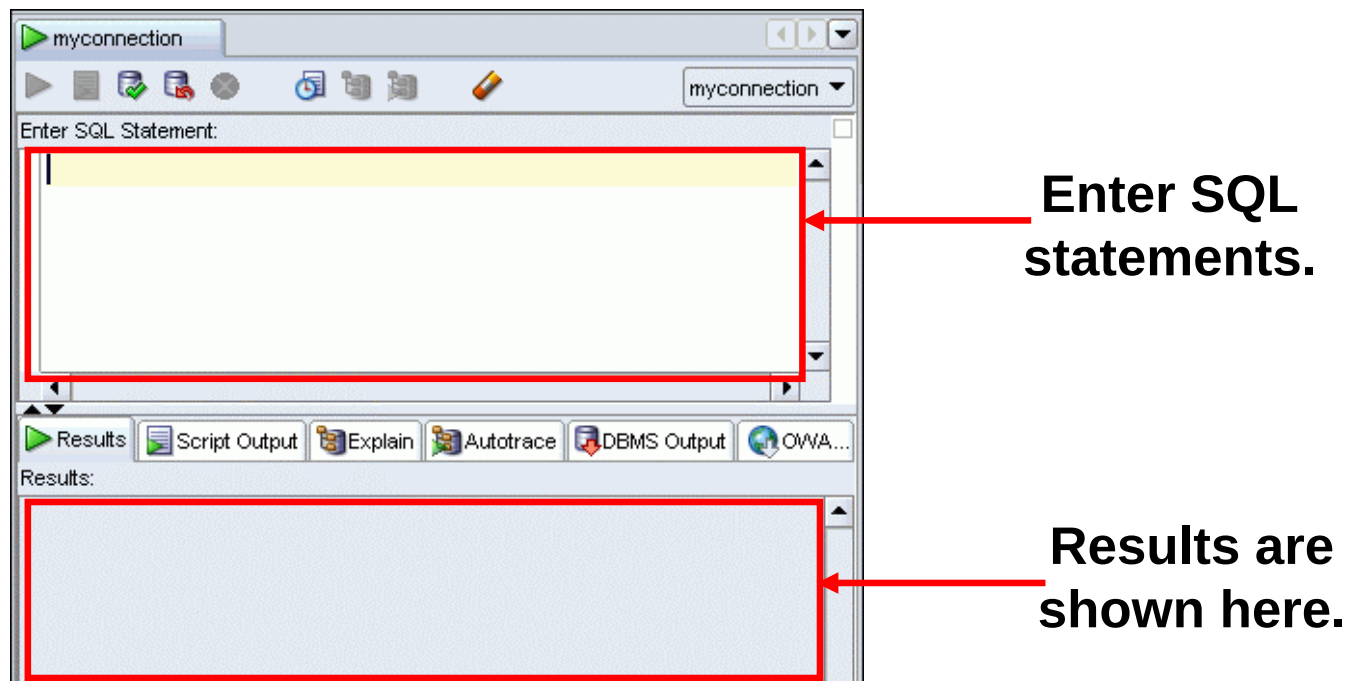


Using the SQL Worksheet



Using the SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL*Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.



Executing SQL Statements

Use the Enter SQL Statement box to enter single or multiple SQL statements.

Use the Enter SQL Statement box to enter single or multiple SQL statements.

View the results on the Script Output tabbed page.

The screenshot shows the Oracle SQL Developer interface. At the top, a tab labeled 'MyDBConnection' is active. Below it, a toolbar contains icons for running, saving, and other database operations. A red box highlights the 'Enter SQL Statement:' text box, which contains the following SQL code:

```
1 SELECT last_name, salary
2 FROM employees
3 WHERE salary > 10000;
4
5 SELECT last_name "Name", salary*12 "Annual Salary"
6 FROM employees;
```

Below the SQL editor, another toolbar shows tabs for 'Results', 'Script Output', 'Explain', 'Autotrace', 'DBMS Output', and 'OWA Output'. A red box highlights the 'Script Output' tab. The 'Script Output' tab is active and displays the results of the SQL execution. It shows two tables of results. The first table has two columns: 'last_name' and 'salary'. The second table has two columns: 'Name' and 'Annual Salary'. The results are as follows:

last_name	salary
Ozer	11500
Abel	11000

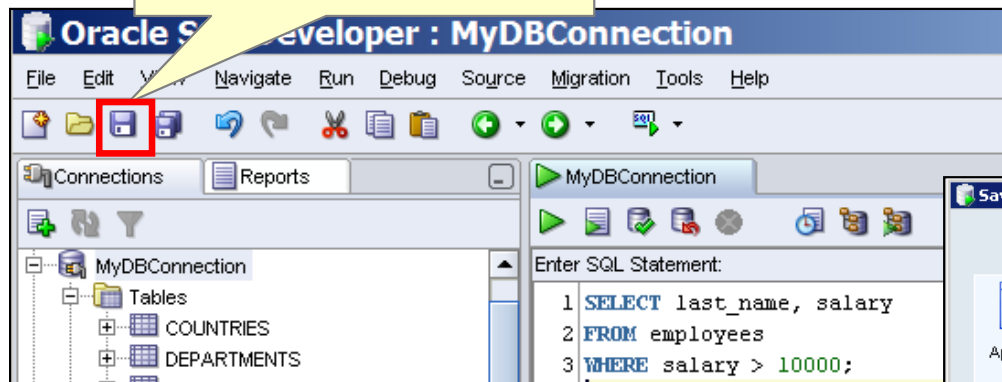
15 rows selected

Name	Annual Salary
OConnell	31200
Grant	31200
Whalen	52800

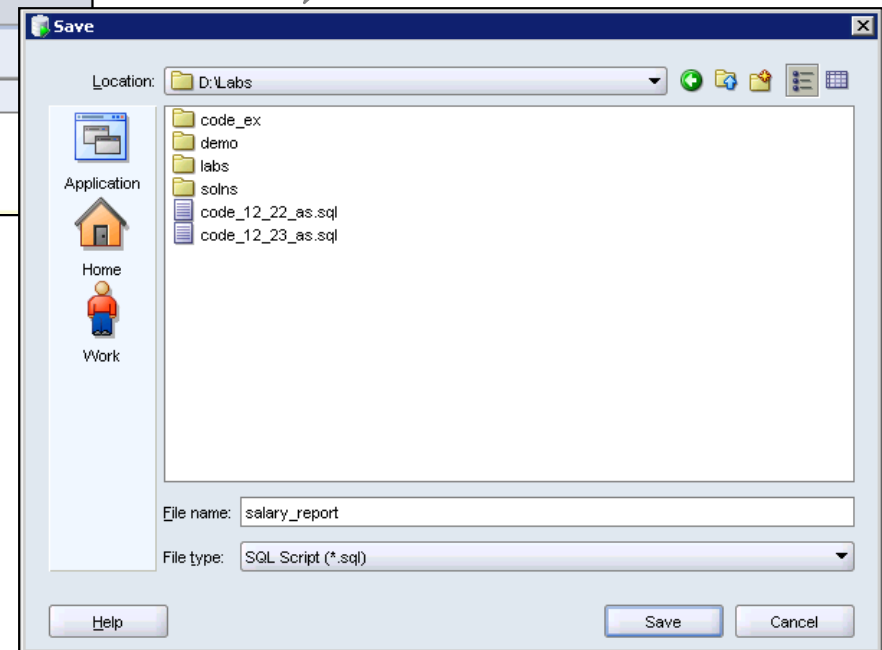
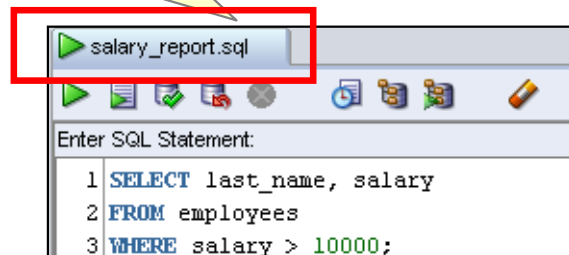
Saving SQL Scripts

Click the Save icon to save your SQL statement to a file.

Enter a file name and identify a location to save the file, and click Save.



The contents of the saved file are visible and editable in your SQL Worksheet window.



Executing Saved Script Files: Method 1

Right-click in the SQL Worksheet area, and select Open File from the shortcut menu.

Select (or navigate to) the script file that you want to open.

Click Open.

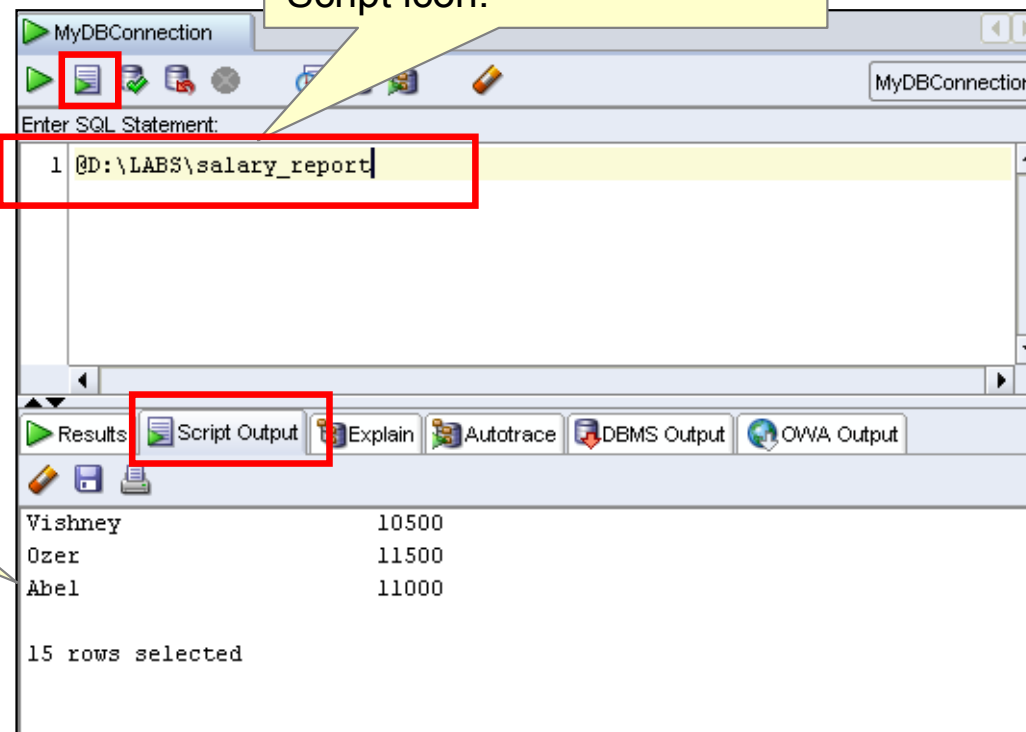
To run the code, click the Run Script (F5) icon.

```
CREATE PROCEDURE getemp IS -- header
emp_id employees.employee_id%type;
lname employees.last_name%type;
BEGIN
emp_id := 100;
SELECT last_name INTO lname
FROM EMPLOYEES
WHERE employee_id = emp_id;
DBMS_OUTPUT.PUT_LINE('Last name: '||lname);
END;
```

Executing Saved Script Files: Method 2

Use the @ command followed by the location and name of the file you want to execute, and click the Run Script icon.

The output from the script is displayed on the Script Output tabbed page.



Executing SQL Statements

Use the Enter SQL Statement box to enter single or multiple SQL statements.

The screenshot illustrates the workflow for executing an SQL statement in Oracle SQL Developer. It features three main components: the 'Enter SQL Statement' window, the 'Results' window, and the 'Script Output' window. Red boxes and green circles with function key labels (F9, F5) highlight the execution steps.

Enter SQL Statement Window: The 'myconnection' toolbar contains a green play button (F9) and a document icon (F5). The text area contains the SQL statement: `SELECT employee_id, last_name FROM employees;`

Results Window: The 'Results' tab is active, showing a table of results. The 'F9' label points to the 'Results' tab button.

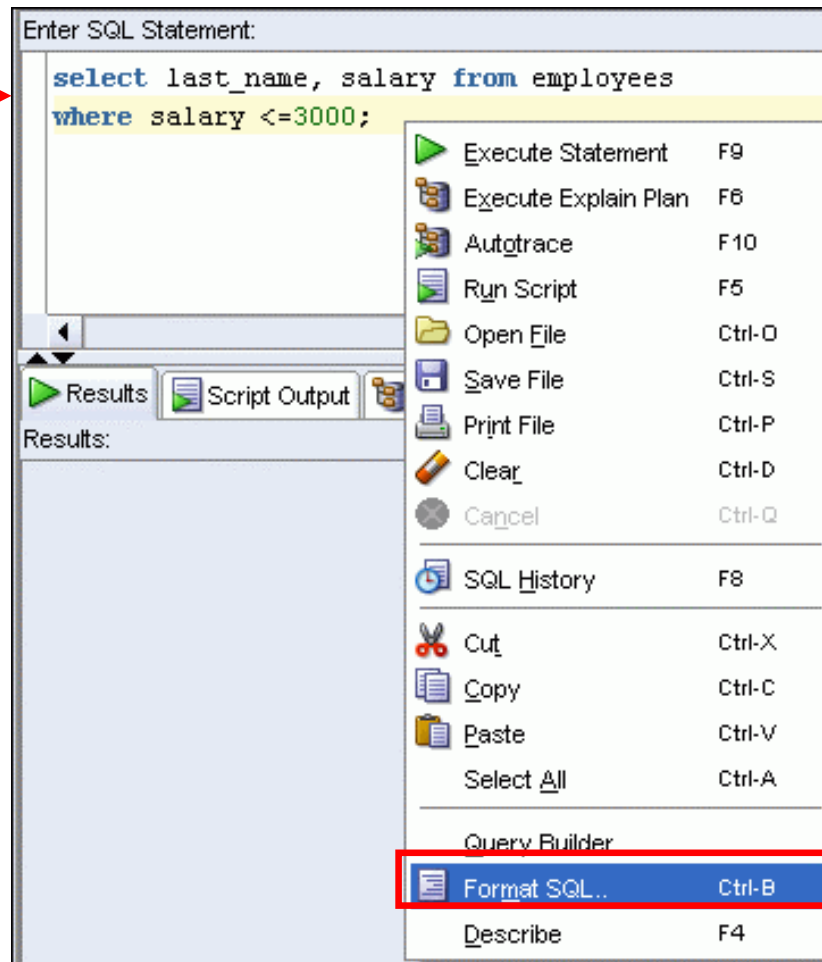
	EMPLOYEE_ID	LAST_NAME
1	100	King
2	101	Kochhar
3	102	De Haan
4	103	Hunold
5	104	Ernst

Script Output Window: The 'Script Output' tab is active, showing the execution results. The 'F5' label points to the 'Script Output' tab button.

EMPLOYEE_ID	LAST_NAME
100	King
101	Kochhar
102	De Haan
103	Hunold
104	Ernst
105	Austin

Formatting the SQL Code

**Before
formatting**

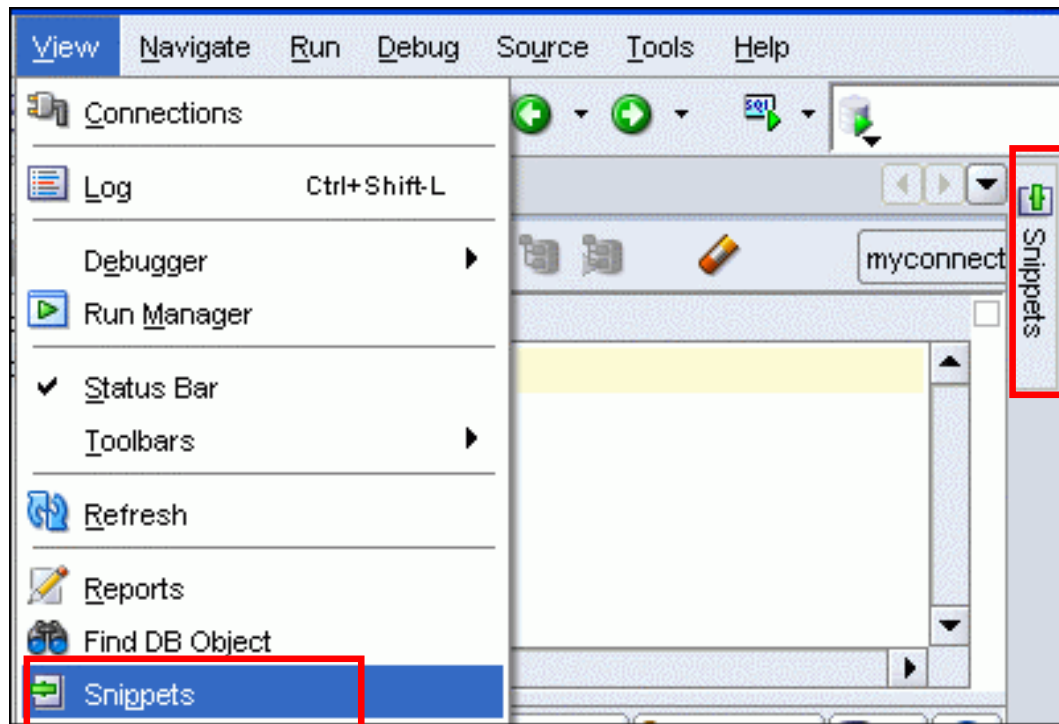


**After
formatting**

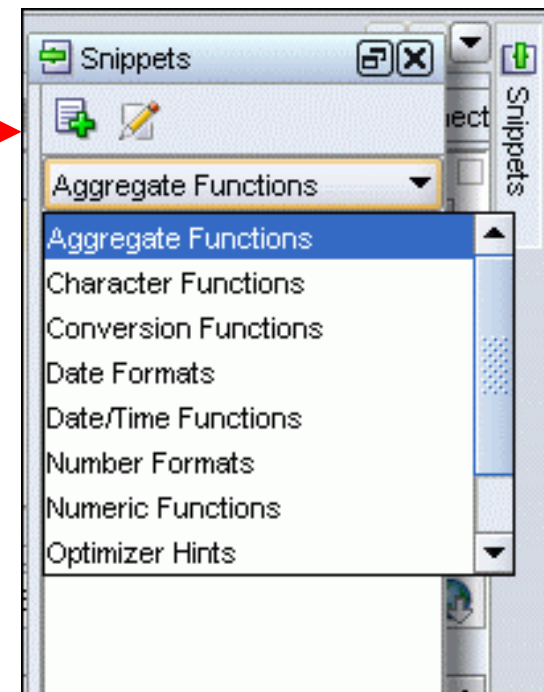
The screenshot shows the same SQL code after formatting: `SELECT last_name,
salary
FROM employees
WHERE salary <= 3000;`. The code is now properly indented and uses uppercase keywords.

Using Snippets

Snippets are code fragments that may be just syntax or examples.

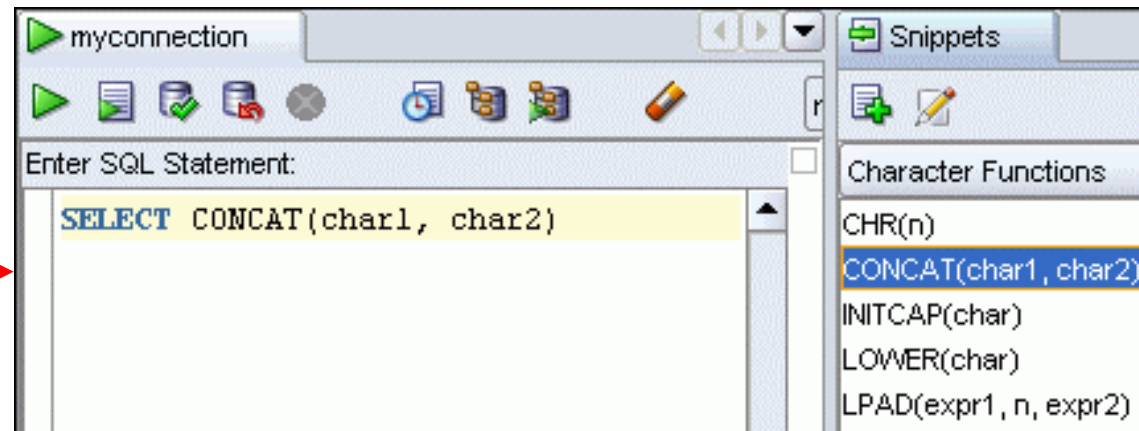


When you place your cursor here, it shows the Snippets window. From the drop-down list, you can select the functions category that you want.

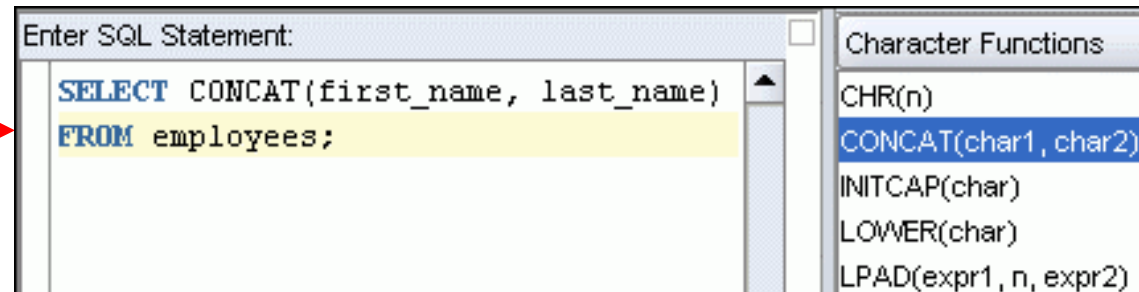


Using Snippets: Example

Inserting a snippet

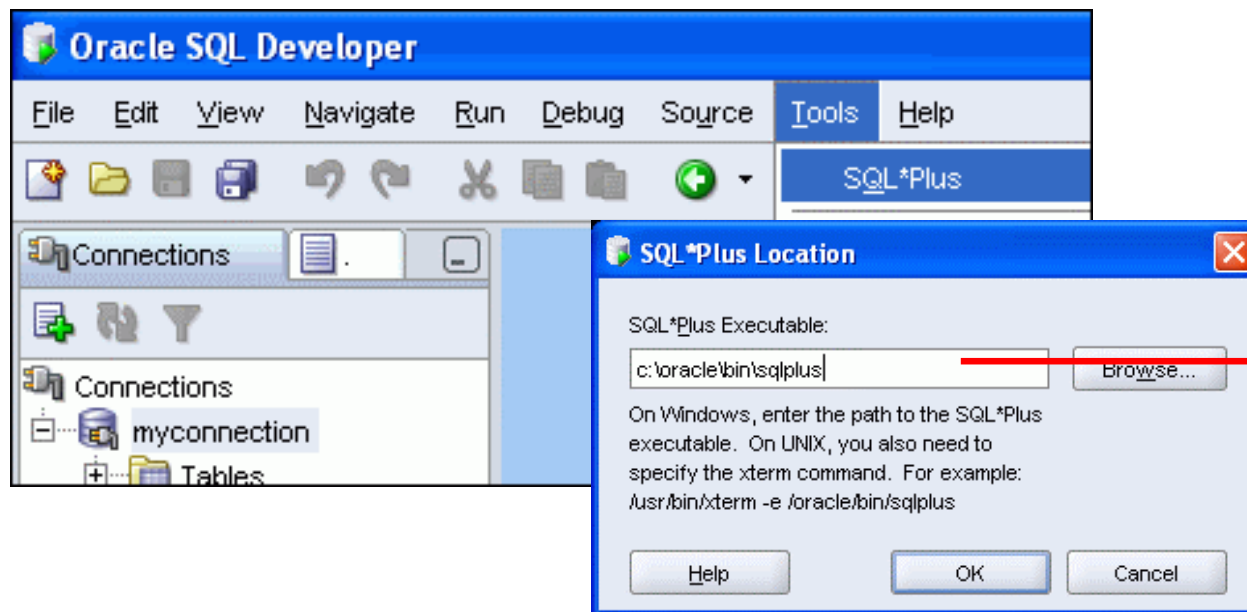


Editing the snippet



Using SQL*Plus

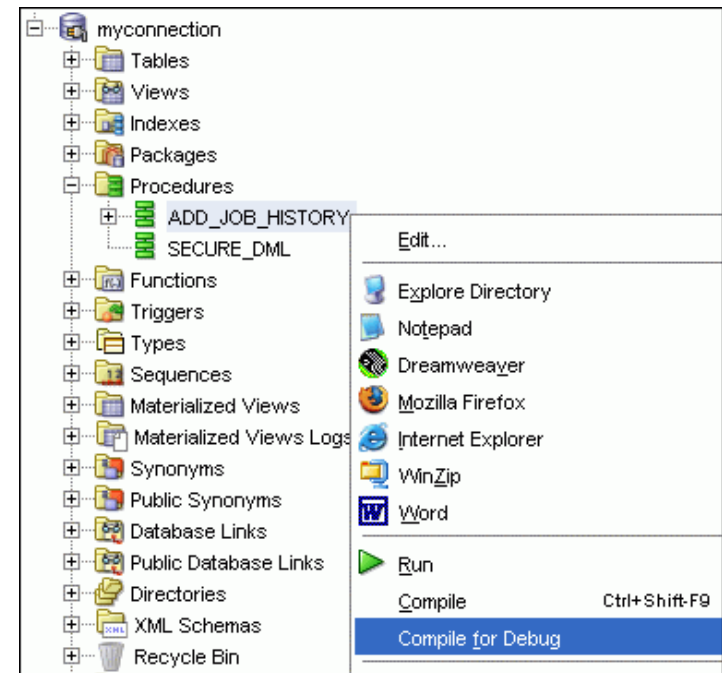
- You can invoke the SQL*Plus command-line interface from SQL Developer.
- Close all the SQL Worksheets to enable the SQL*Plus menu option.



Provide the location of the sqlplus.exe file only the first time you invoke SQL*Plus.

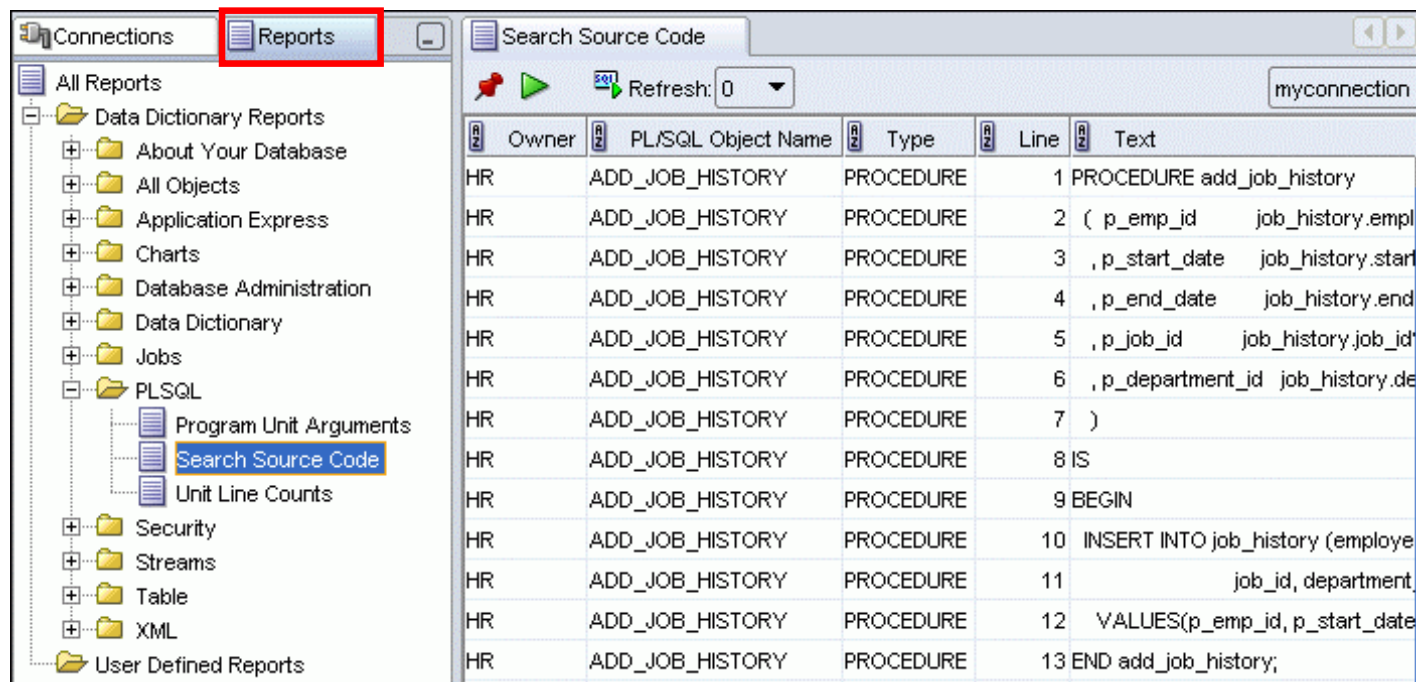
Debugging Procedures and Functions

- Use SQL Developer to debug PL/SQL functions and procedures.
- Use the Compile for Debug option to perform a PL/SQL compilation so that the procedure can be debugged.
- Use Debug menu options to set breakpoints, and to perform step into, step over tasks.



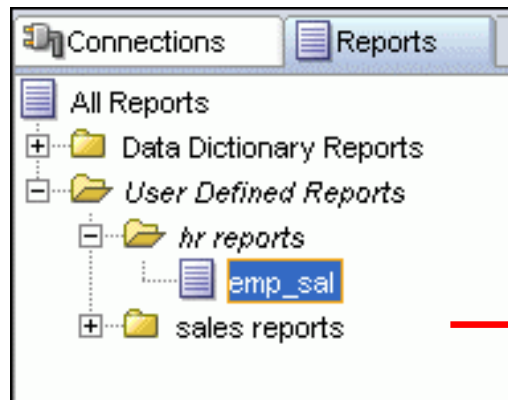
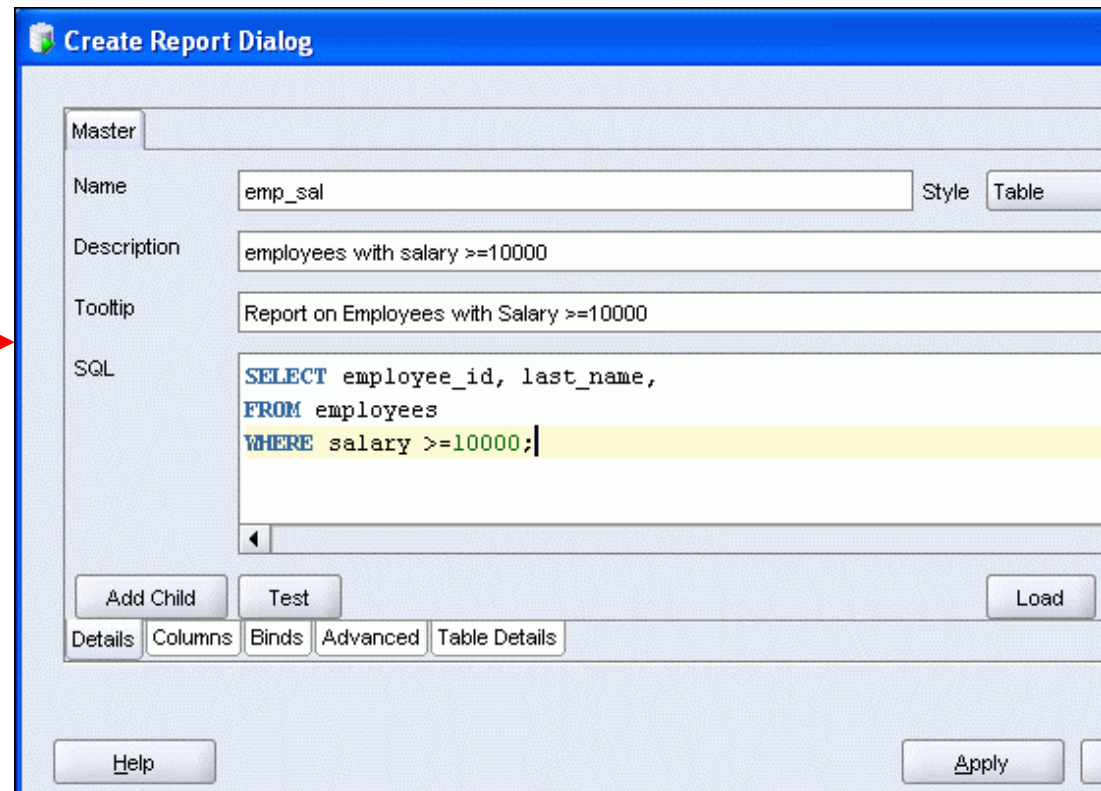
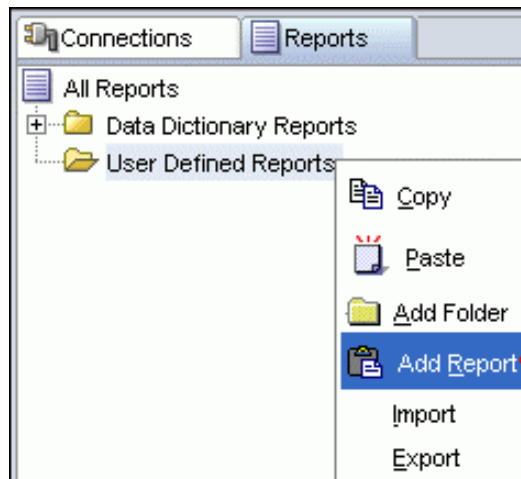
Database Reporting

SQL Developer provides a number of predefined reports about the database and its objects.



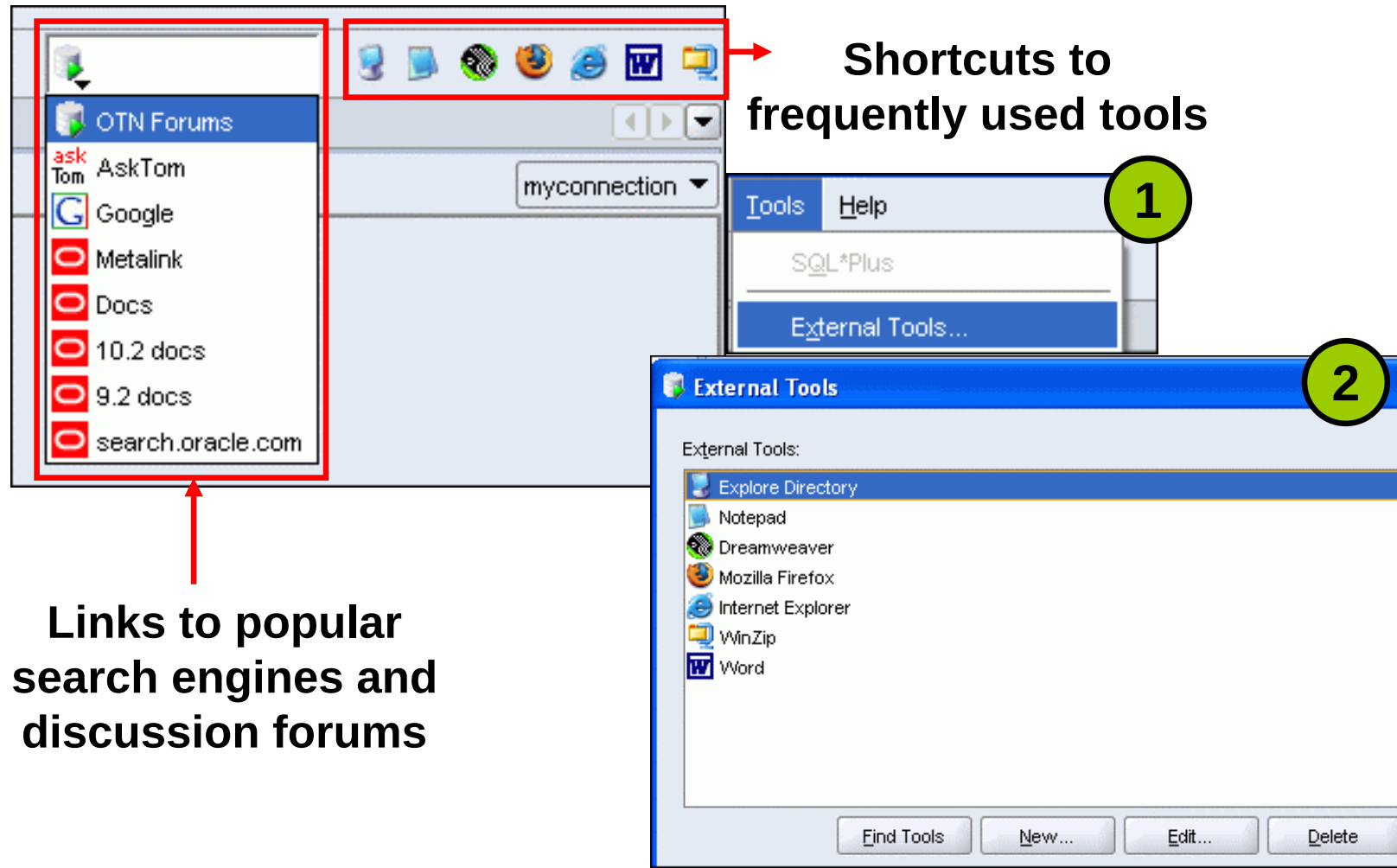
Creating a User-Defined Report

Create and save user-defined reports for repeated use.



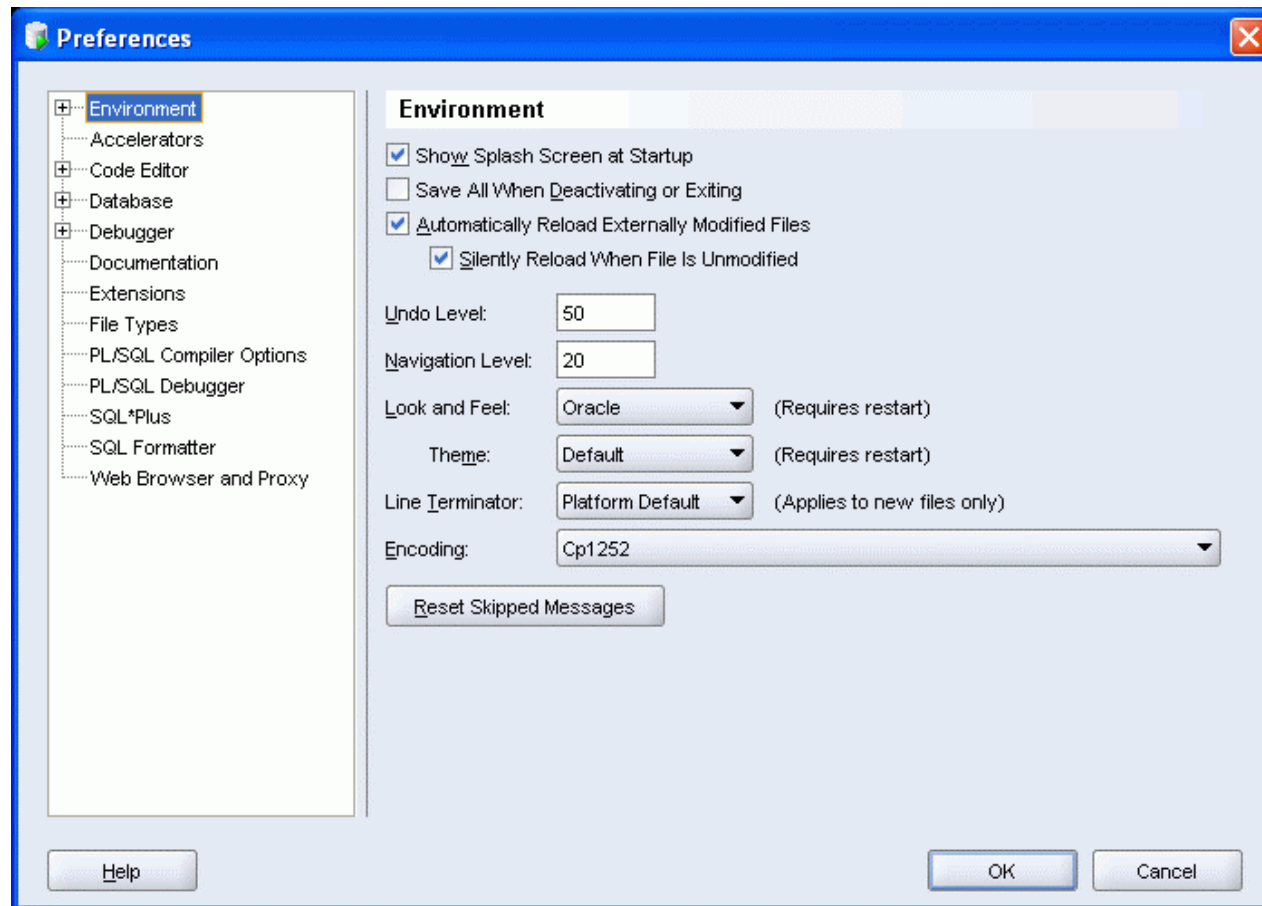
Organize reports in folders.

Search Engines and External Tools



Setting Preferences

- Customize the SQL Developer interface and environment.
- In the Tools menu, select Preferences.

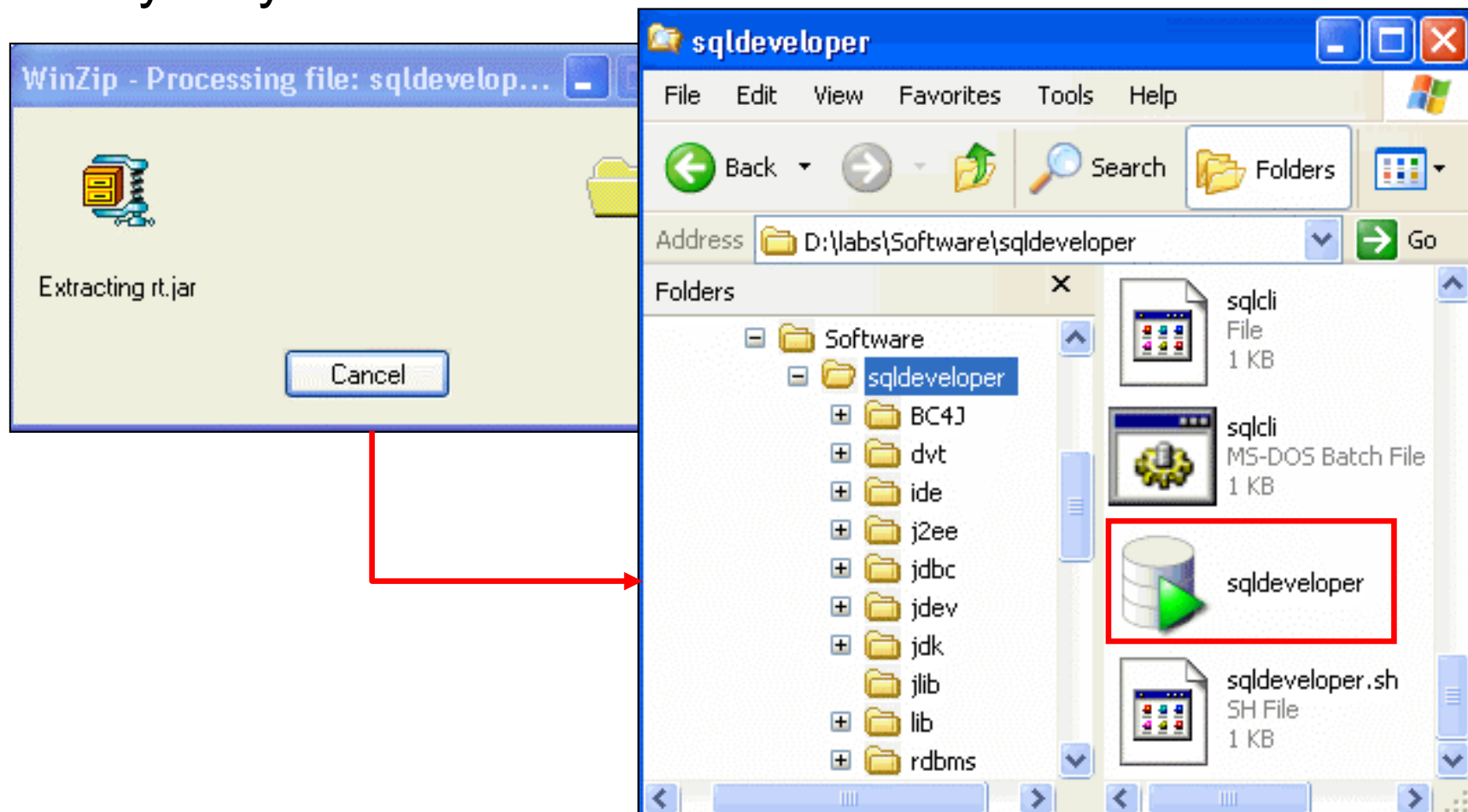


Specifications of SQL Developer 1.5.3

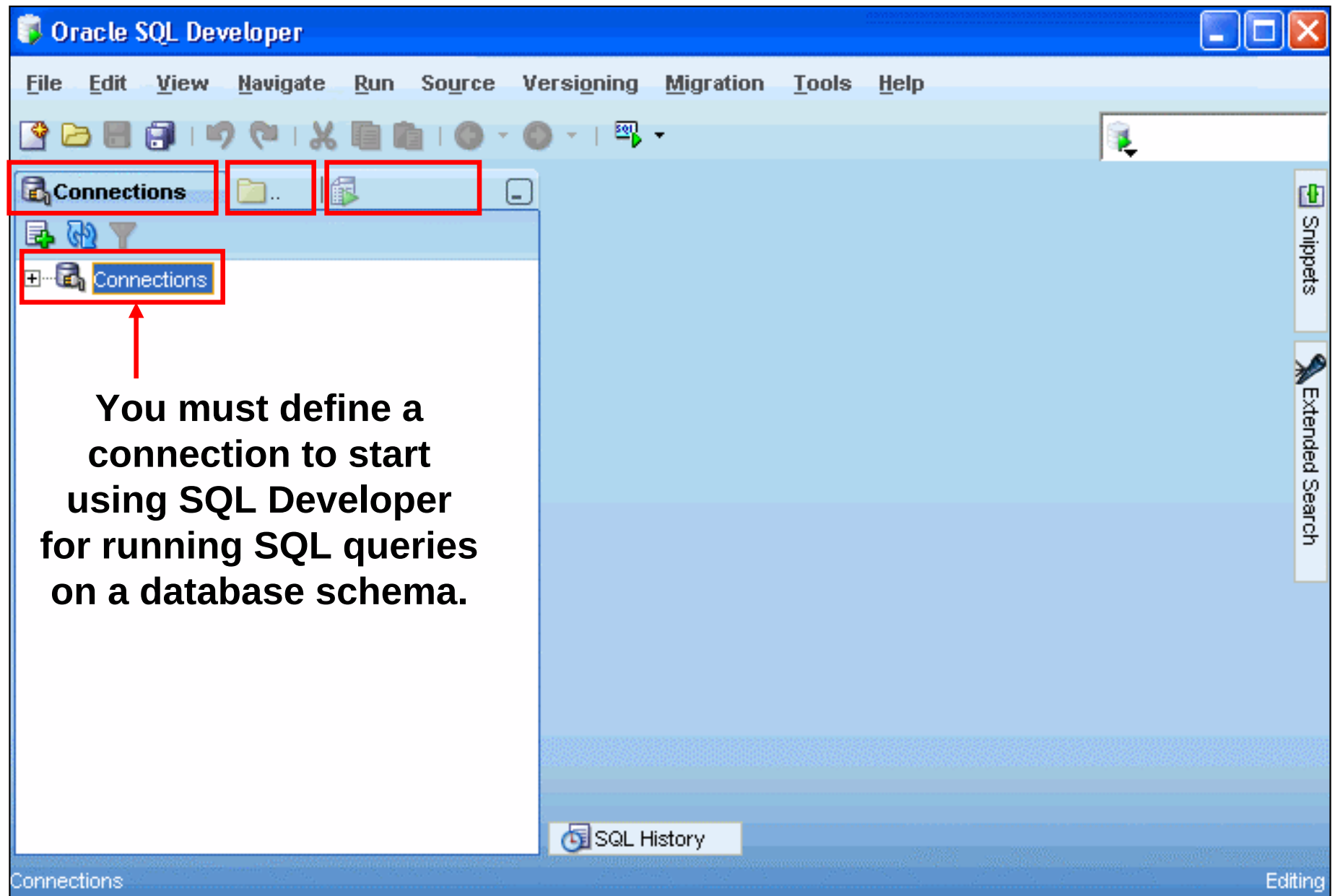
- SQL Developer 1.5.3 is the first translation release and a patch to Oracle SQL Developer 1.5.
- The new feature list is available at:
 - http://www.oracle.com/technology/products/database/sql_developer/files/newFeatures_v15.html
- It supports Windows, Linux, and Mac OS X platforms.
- To install, unzip the downloaded SQL Developer kit, which includes the required minimum JDK (JDK1.5.0_06).
- To start, double-click `sqldeveloper.exe`.
- It connects to Oracle Database version 9.2.0.1 and later.
- It is freely downloadable from the following link:
 - http://www.oracle.com/technology/products/database/sql_developer/index.html

Installing SQL Developer 1.5.3

Download the Oracle SQL Developer kit and unzip it into any directory on your machine.



SQL Developer 1.5.3 Interface



Summary

In this appendix, you should have learned how to use SQL Developer to do the following:

- Browse, create, and edit database objects
- Execute SQL statements and scripts in SQL Worksheet
- Create and save custom reports

D

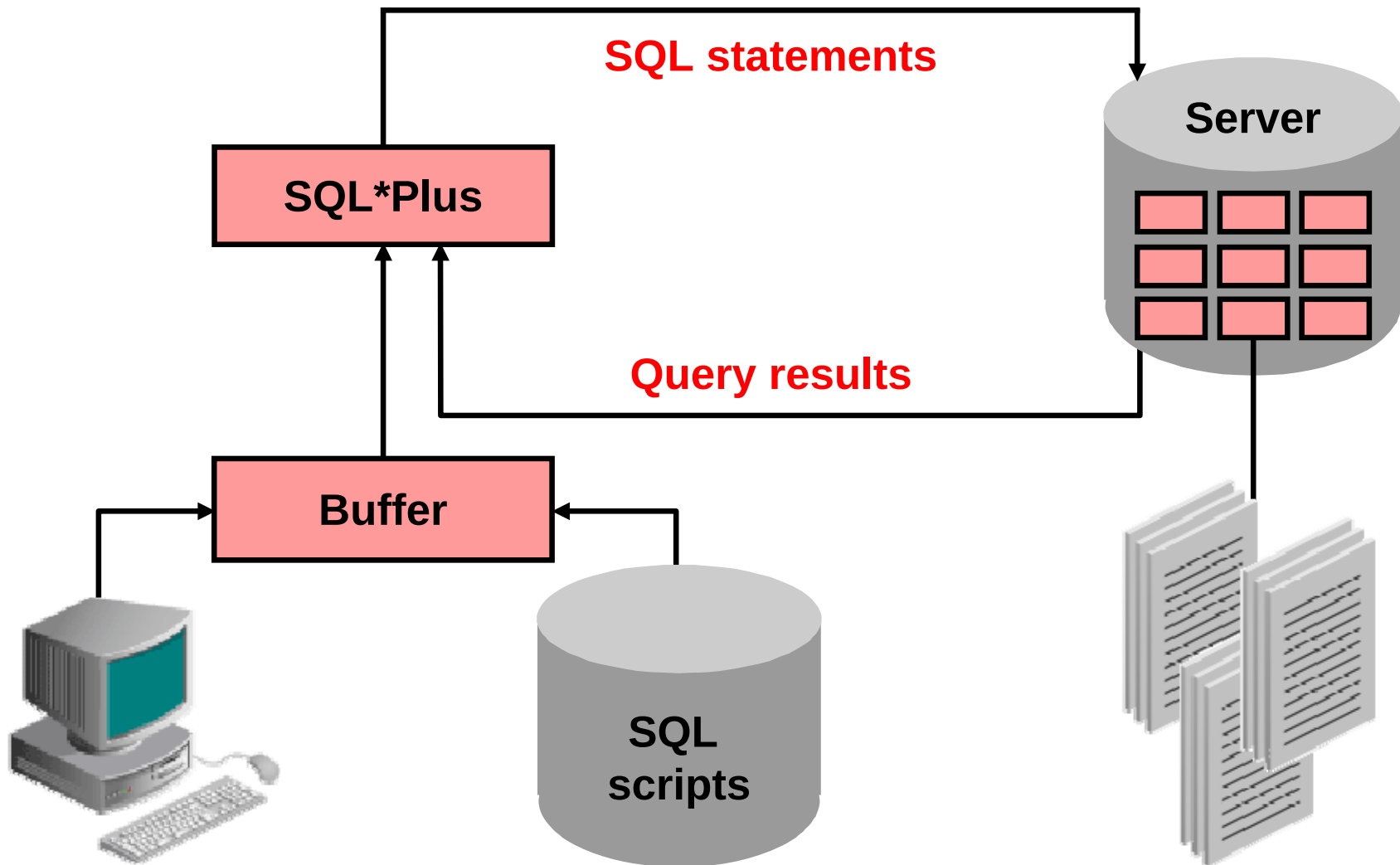
Using SQL*Plus

Objectives

After completing this appendix, you should be able to do the following:

- Log in to SQL*Plus
- Edit SQL commands
- Format the output using SQL*Plus commands
- Interact with script files

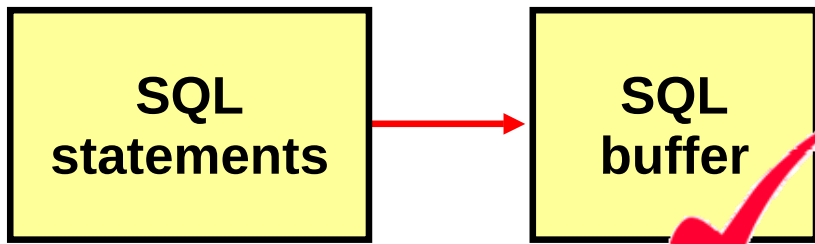
SQL and SQL*Plus Interaction



SQL Statements Versus SQL*Plus Commands

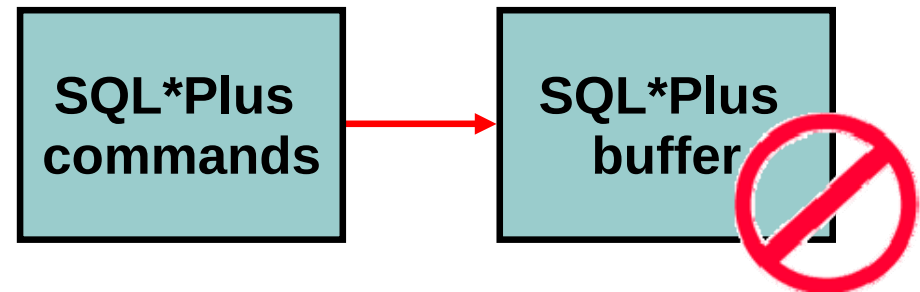
SQL

- A language
- ANSI-standard
- Keywords cannot be abbreviated.
- Statements manipulate data and table definitions in the database.



SQL*Plus

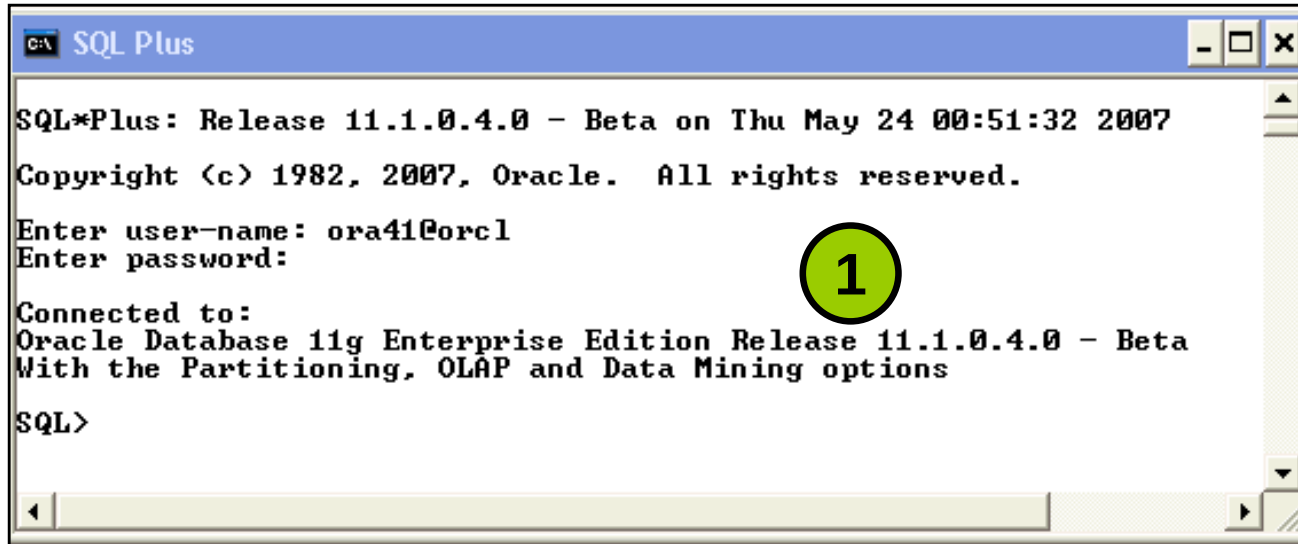
- An environment
- Oracle-proprietary
- Keywords can be abbreviated.
- Commands do not allow manipulation of values in the database.



Overview of SQL*Plus

- Log in to SQL*Plus.
- Describe the table structure.
- Edit your SQL statement.
- Execute SQL from SQL*Plus.
- Save SQL statements to files and append SQL statements to files.
- Execute saved files.
- Load commands from the file to buffer to edit.

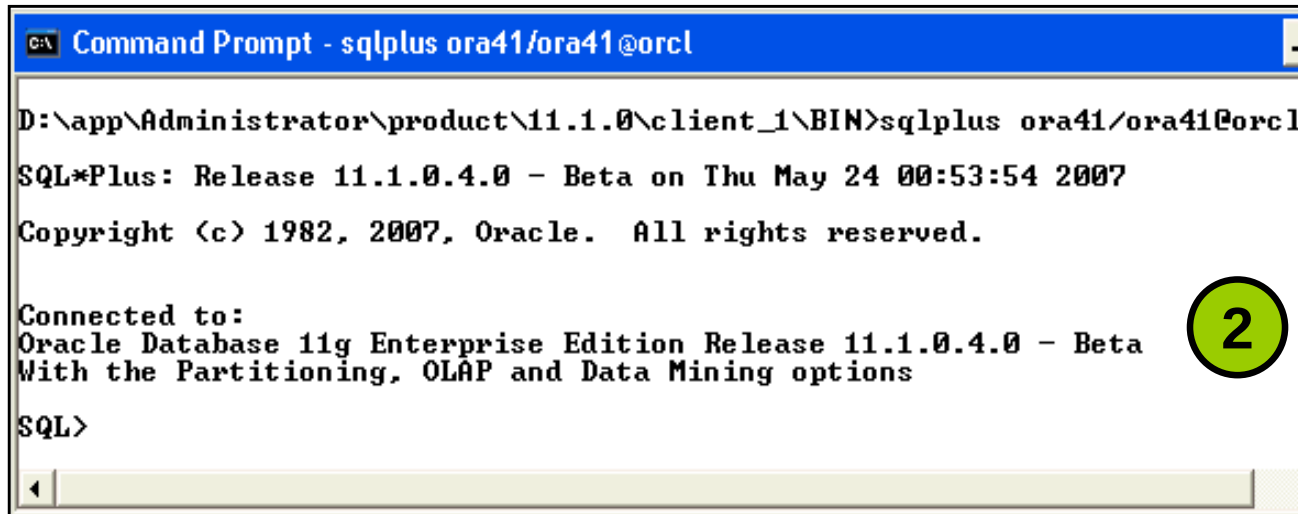
Logging In to SQL*Plus

A screenshot of a Windows-style window titled "SQL Plus". The window contains the following text:

```
SQL*Plus: Release 11.1.0.4.0 - Beta on Thu May 24 00:51:32 2007
Copyright (c) 1982, 2007, Oracle. All rights reserved.
Enter user-name: ora41@orcl
Enter password:
Connected to:
Oracle Database 11g Enterprise Edition Release 11.1.0.4.0 - Beta
With the Partitioning, OLAP and Data Mining options
SQL>
```

A green circle with the number "1" is positioned to the right of the password prompt line.

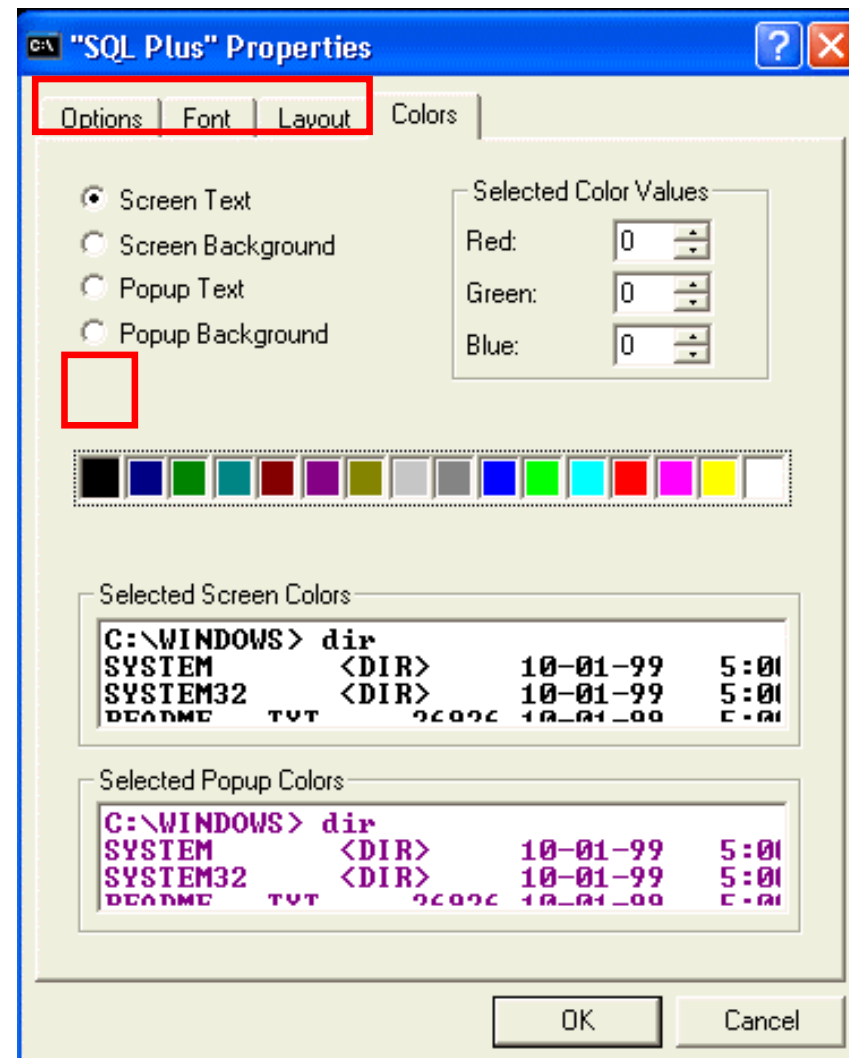
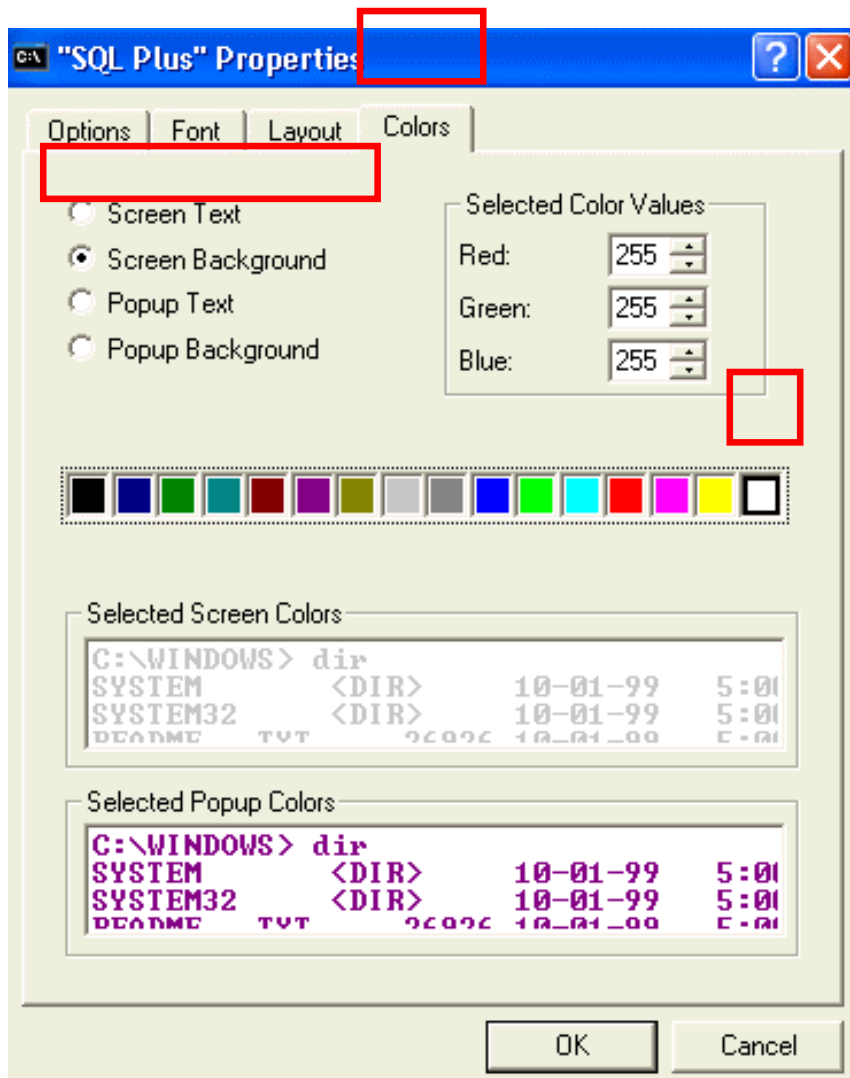
```
sqlplus [username[/password[@database]]]
```

A screenshot of a Windows-style window titled "Command Prompt - sqlplus ora41/ora41@orcl". The window shows the execution of the sqlplus command from a directory path. The text inside is:

```
D:\app\Administrator\product\11.1.0\client_1\BIN>sqlplus ora41/ora41@orcl
SQL*Plus: Release 11.1.0.4.0 - Beta on Thu May 24 00:53:54 2007
Copyright (c) 1982, 2007, Oracle. All rights reserved.
Connected to:
Oracle Database 11g Enterprise Edition Release 11.1.0.4.0 - Beta
With the Partitioning, OLAP and Data Mining options
SQL>
```

A green circle with the number "2" is positioned to the right of the "Connected to:" line.

Changing the Settings of the SQL*Plus Environment



Displaying the Table Structure

Use the SQL*Plus DESCRIBE command to display the structure of a table:

```
DESC[RIBE] tablename
```

Displaying the Table Structure

```
DESCRIBE departments
```

Name	Null?	Type
-----	-----	-----
DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

SQL*Plus Editing Commands

- `A[PPEND] text`
- `C[HANGE] / old / new`
- `C[HANGE] / text /`
- `CL[EAR] BUFF[ER]`
- `DEL`
- `DEL n`
- `DEL m n`

SQL*Plus Editing Commands

- I [NPUT]
- I [NPUT] *text*
- L [IST]
- L [IST] *n*
- L [IST] *m n*
- R [UN]
- *n*
- *n text*
- 0 *text*

Using LIST, n, and APPEND

LIST

```
1  SELECT last_name  
2* FROM    employees
```

1

```
1* SELECT last_name
```

A , job_id

```
1* SELECT last_name, job_id
```

LIST

```
1  SELECT last_name, job_id  
2* FROM    employees
```

Using the CHANGE Command

```
LIST  
1* SELECT * from employees
```

```
c/employees/departments  
1* SELECT * from departments
```

```
LIST  
1* SELECT * from departments
```

SQL*Plus File Commands

- `SAVE filename`
- `GET filename`
- `START filename`
- `@ filename`
- `EDIT filename`
- `SPOOL filename`
- `EXIT`

Using the SAVE, START, and EDIT Commands

```
LIST
```

```
1  SELECT last_name, manager_id, department_id
2*  FROM employees
```

```
SAVE my_query
```

```
Created file my_query
```

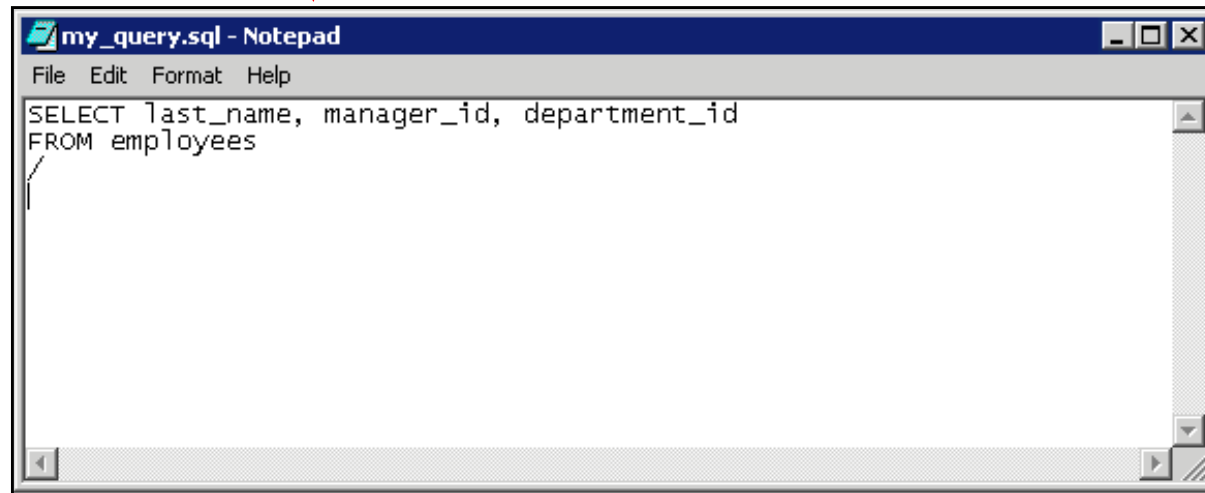
```
START my_query
```

LAST_NAME	MANAGER_ID	DEPARTMENT_ID
-----	-----	-----
King		90
Kochhar	100	90
...		

107 rows selected.

Using the SAVE, START, and EDIT Commands

EDIT my_query



SERVEROUTPUT Command

- Use the `SET SERVEROUT [PUT]` command to control whether to display the output of stored procedures or PL/SQL blocks in SQL*Plus.
- The `DBMS_OUTPUT` line length limit is increased from 255 bytes to 32767 bytes.
- The default size is now unlimited.
- Resources are not preallocated when `SERVEROUTPUT` is set.
- Because there is no performance penalty, use `UNLIMITED` unless you want to conserve physical memory.

```
SET SERVEROUT [PUT] {ON | OFF} [SIZE {n | UNL[IMITED]}]  
[FOR [MAT] {WRA[PPED] | WOR[D_WAPPED] | TRU[NCATED]}]
```

Using the SQL*Plus SPOOL Command

```
SPO[OL] [file_name[.ext] [CRE[ATE] | REP[LACE] |  
APP[END]] | OFF | OUT]
```

Option	Description
file_name[.ext]	Spools output to the specified file name
CRE[ATE]	Creates a new file with the name specified
REP[LACE]	Replaces the contents of an existing file. If the file does not exist, REPLACE creates the file.
APP[END]	Adds the contents of the buffer to the end of the file you specify
OFF	Stops spooling
OUT	Stops spooling and sends the file to your computer's standard (default) printer

Using the AUTOTRACE Command

- It displays a report after the successful execution of SQL DML statements such as SELECT, INSERT, UPDATE, or DELETE.
- The report can now include execution statistics and the query execution path.

```
SET AUTOT[RACE] {ON | OFF | TRACE[ONLY]} [EXP[LAIN]]  
[STAT[ISTICS]]
```

```
SET AUTOTRACE ON
```

```
-- The AUTOTRACE report includes both the optimizer  
-- execution path and the SQL statement execution  
-- statistics
```

Summary

In this appendix, you should have learned how to use SQL*Plus as an environment to do the following:

- Execute SQL statements
- Edit SQL statements
- Format the output
- Interact with script files



Generating Reports by Grouping Related Data

Objectives

After completing this appendix, you should be able to do the following:

- Use the `ROLLUP` operation to produce subtotal values
- Use the `CUBE` operation to produce cross-tabulation values
- Use the `GROUPING` function to identify the row values created by `ROLLUP` or `CUBE`
- Use `GROUPING SETS` to produce a single result set

Review of Group Functions

- Group functions operate on sets of rows to give one result per group.

```
SELECT      [column,] group_function(column) . . .
FROM        table
[WHERE      condition]
[GROUP BY   group_by_expression]
[ORDER BY   column];
```

- Example:

```
SELECT AVG(salary), STDDEV(salary),
       COUNT(commission_pct), MAX(hire_date)
FROM   employees
WHERE  job_id LIKE 'SA%';
```

Review of the GROUP BY Clause

- Syntax:

```
SELECT      [column,] group_function(column) . . .  
FROM        table  
[WHERE      condition]  
[GROUP BY   group_by_expression]  
[ORDER BY   column];
```

- Example:

```
SELECT      department_id, job_id, SUM(salary),  
            COUNT(employee_id)  
FROM        employees  
GROUP BY    department_id, job_id ;
```

Review of the HAVING Clause

- Use the HAVING clause to specify which groups are to be displayed.
- You further restrict the groups on the basis of a limiting condition.

```
SELECT      [column,] group_function(column) ...  
FROM        table  
[WHERE      condition]  
[GROUP BY   group_by_expression]  
[HAVING     having_expression]  
[ORDER BY   column];
```

GROUP BY with ROLLUP and CUBE Operators

- Use ROLLUP or CUBE with GROUP BY to produce superaggregate rows by cross-referencing columns.
- ROLLUP grouping produces a result set containing the regular grouped rows and the subtotal values.
- CUBE grouping produces a result set containing the rows from ROLLUP and cross-tabulation rows.

ROLLUP Operator

- ROLLUP is an extension to the GROUP BY clause.
- Use the ROLLUP operation to produce cumulative aggregates, such as subtotals.

```
SELECT      [column,] group_function(column). . .  
FROM        table  
[WHERE      condition]  
[GROUP BY   [ROLLUP] group_by_expression]  
[HAVING     having_expression];  
[ORDER BY   column];
```

ROLLUP Operator: Example

```
SELECT  department_id, job_id, SUM(salary)
FROM    employees
WHERE   department_id < 60
GROUP BY ROLLUP(department_id, job_id);
```

	DEPARTMENT_ID	JOB_ID	SUM(SALARY)	
1	10	AD_ASST	4400	1
2	10		4400	
3	20	MK_MAN	13000	
4	20	MK_REP	6000	
5	20		19000	
6	30	PU_MAN	11000	
7	30	PU_CLERK	13900	
8	30		24900	
9	40	HR_REP	6500	
10	40		6500	2
11	50	ST_MAN	36400	
12	50	SH_CLERK	64300	
13	50	ST_CLERK	55700	
14	50		156400	
15			211200	3

CUBE Operator

- CUBE is an extension to the GROUP BY clause.
- You can use the CUBE operator to produce cross-tabulation values with a single SELECT statement.

```
SELECT      [column,] group_function(column) ...  
FROM        table  
[WHERE      condition]  
[GROUP BY   [CUBE] group_by_expression]  
[HAVING     having_expression]  
[ORDER BY   column];
```

CUBE Operator: Example

```
SELECT  department_id, job_id, SUM(salary)
FROM    employees
WHERE   department_id < 60
GROUP BY CUBE (department_id, job_id) ;
```

	DEPARTMENT_ID	JOB_ID	SUM(SALARY)
1			211200
2		HR_REP	6500
3		MK_MAN	13000
4		MK_REP	6000
5		PU_MAN	11000
6		ST_MAN	36400
7		AD_ASST	4400
8		PU_CLERK	13900
9		SH_CLERK	64300
10		ST_CLERK	55700
11	10		4400
12	10	AD_ASST	4400
13	20		19000
14	20	MK_MAN	13000
15	20	MK_REP	6000
16	30		24900
17	30	PU_MAN	11000

1

2

3

4

GROUPING Function

The GROUPING function:

- Is used with either the CUBE or ROLLUP operator
- Is used to find the groups forming the subtotal in a row
- Is used to differentiate stored NULL values from NULL values created by ROLLUP or CUBE
- Returns 0 or 1

```
SELECT      [column,] group_function(column) .. ,  
            GROUPING(expr)  
FROM        table  
[WHERE      condition]  
[GROUP BY  [ROLLUP] [CUBE] group_by_expression]  
[HAVING    having_expression]  
[ORDER BY  column];
```

GROUPING Function: Example

```
SELECT    department_id DEPTID, job_id JOB,
          SUM(salary),
          GROUPING(department_id) GRP_DEPT,
          GROUPING(job_id) GRP_JOB
FROM      employees
WHERE     department_id < 50
GROUP BY  ROLLUP(department_id, job_id);
```

	DEPTID	JOB	SUM(SALARY)	GRP_DEPT	GRP_JOB
1	10	AD_ASST	4400	0	0
2	10		4400	0	1
3	20	MK_MAN	13000	0	0
4	20	MK_REP	6000	0	0
5	20		19000	0	1
6	30	PU_MAN	11000	0	0
7	30	PU_CLERK	13900	0	0
8	30		24900	0	1
9	40	HR_REP	6500	0	0
10	40		6500	0	1
11			54800	1	1

GROUPING SETS

- The GROUPING SETS syntax is used to define multiple groupings in the same query.
- All groupings specified in the GROUPING SETS clause are computed and the results of individual groupings are combined with a UNION ALL operation.
- Grouping set efficiency:
 - Only one pass over the base table is required.
 - There is no need to write complex UNION statements.
 - The more elements GROUPING SETS has, the greater is the performance benefit.

GROUPING SETS: Example

```
SELECT    department_id, job_id,  
          manager_id,AVG(salary)  
FROM      employees  
GROUP BY GROUPING SETS  
        ((department_id,job_id), (job_id,manager_id));
```

DEPARTMENT_ID	JOB_ID	MANAGER_ID	AVG(SALARY)
	SH_CLERK	122	3200
	AC_MGR	101	12000
	ST_MAN	100	7280
...	ST_CLERK	121	2675

1

DEPARTMENT_ID	JOB_ID	MANAGER_ID	AVG(SALARY)
110	AC_MGR		12000
90	AD_PRES		24000
60	IT_PROG		5760
100	FI_MGR		12000
30	PU_CLERK		2780
...	50 SH_CLERK		3215

2

Composite Columns

- A composite column is a collection of columns that are treated as a unit.

```
ROLLUP (a, (b, c), d)
```

- Use parentheses within the GROUP BY clause to group columns, so that they are treated as a unit while computing ROLLUP or CUBE operations.
- When used with ROLLUP or CUBE, composite columns would require skipping aggregation across certain levels.

Composite Columns: Example

```
SELECT    department_id, job_id, manager_id,  
          SUM(salary)  
FROM      employees  
GROUP BY ROLLUP( department_id, (job_id, manager_id));
```

1	DEPARTMENT_ID	JOB_ID	MANAGER_ID	SUM(SALARY)	2
		SA_REP	149	7000	
				7000	
	10	AD_ASST	101	4400	
	10			4400	
	20	MK_MAN	100	13000	
	20	MK_REP	201	6000	
	20			19000	
...					
	100	FI_MGR	101	12000	
	100	FI_ACCOUNT	108	39600	
	100			51600	3
	110	AC_MGR	101	12000	
	110	AC_ACCOUNT	205	8300	
	110			20300	
				691400	4

Concatenated Groupings

- Concatenated groupings offer a concise way to generate useful combinations of groupings.
- To specify concatenated grouping sets, you separate multiple grouping sets, ROLLUP, and CUBE operations with commas so that the Oracle server combines them into a single GROUP BY clause.
- The result is a cross-product of groupings from each GROUPING SET.

```
GROUP BY GROUPING SETS (a, b), GROUPING SETS (c, d)
```

Concatenated Groupings: Example

```
SELECT  department_id, job_id, manager_id,
        SUM(salary)
FROM    employees
GROUP BY department_id,
        ROLLUP (job_id),
        CUBE (manager_id);
```

	DEPARTMENT_ID	JOB_ID	MANAGER_ID	SUM(SALARY)
		SA_REP	149	7000
	10	AD_ASST	101	4400
	20	MK_MAN	100	13000
	20	MK_REP	201	6000
	...			
1	90	AD_VP		34000
	90	AD_PRES		24000
	...			
		SA_REP		7000
	10	AD_ASST		4400
	...			
2	110		101	12000
	110		205	8300
	110			20300
				3

Summary

In this appendix, you should have learned how to use the:

- ROLLUP operation to produce subtotal values
- CUBE operation to produce cross-tabulation values
- GROUPING function to identify the row values created by ROLLUP or CUBE
- GROUPING SETS syntax to define multiple groupings in the same query
- GROUP BY clause to combine expressions in various ways:
 - Composite columns
 - Concatenated grouping sets



Hierarchical Retrieval

Objectives

After completing this lesson, you should be able to do the following:

- Interpret the concept of a hierarchical query
- Create a tree-structured report
- Format hierarchical data
- Exclude branches from the tree structure

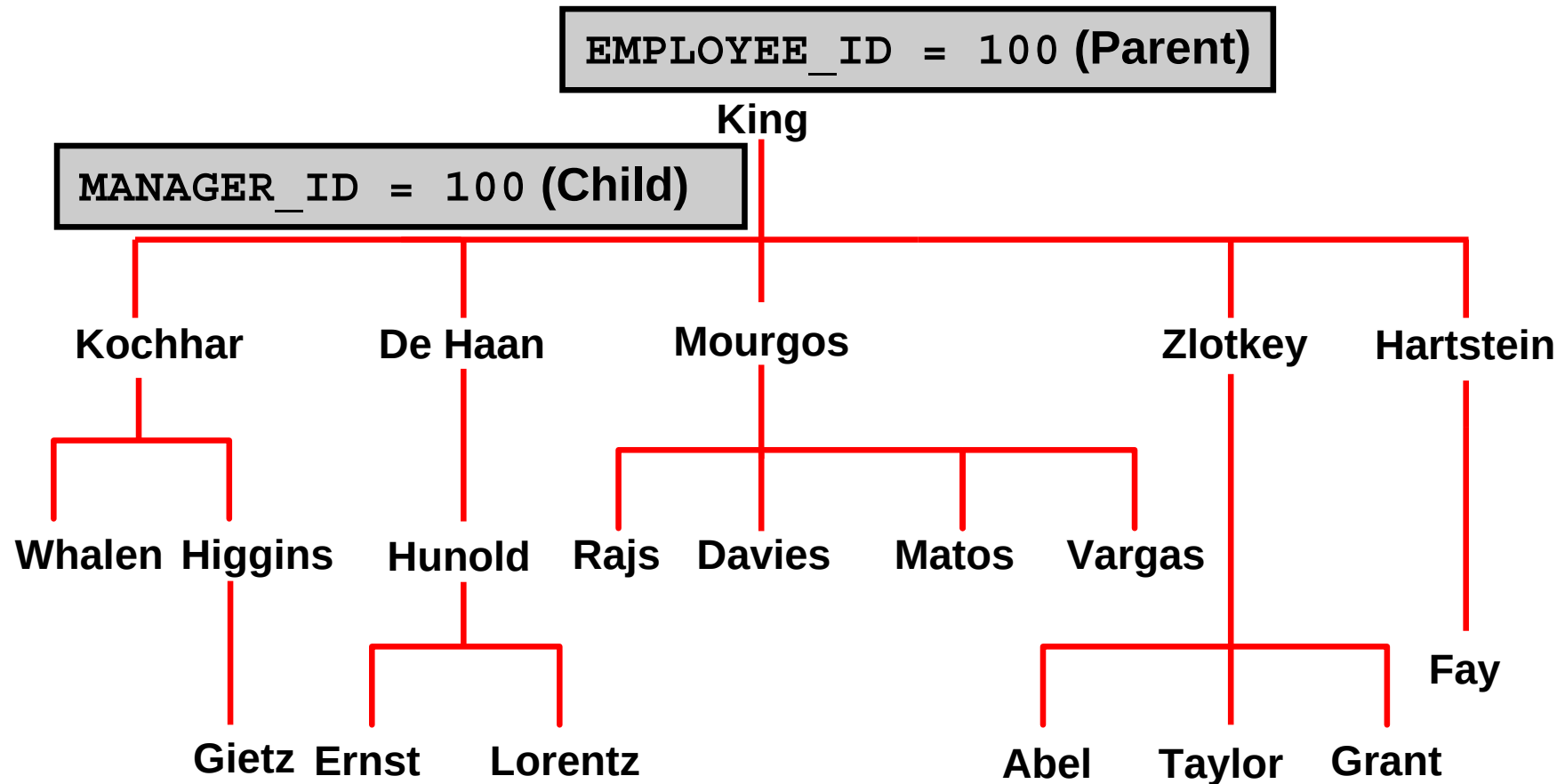
Sample Data from the EMPLOYEES Table

	EMPLOYEE_ID	LAST_NAME	JOB_ID	MANAGER_ID
1	100	King	AD_PRES	(null)
2	101	Kochhar	AD_VP	100
3	102	De Haan	AD_VP	100
4	103	Hunold	IT_PROG	102
5	104	Ernst	IT_PROG	103
6	107	Lorentz	IT_PROG	103

...

16	200	Whalen	AD_ASST	101
17	201	Hartstein	MK_MAN	100
18	202	Fay	MK_REP	201
19	205	Higgins	AC_MGR	101
20	206	Gietz	AC_ACCOUNT	205

Natural Tree Structure



Hierarchical Queries

```
SELECT [LEVEL], column, expr...  
FROM   table  
[WHERE condition(s)]  
[START WITH condition(s)]  
[CONNECT BY PRIOR condition(s)] ;
```

condition:

```
expr comparison_operator expr
```

Walking the Tree

Starting Point

- Specifies the condition that must be met
- Accepts any valid condition

```
START WITH column1 = value
```

Using the EMPLOYEES table, start with the employee whose last name is Kochhar.

```
...START WITH last_name = 'Kochhar'
```

Walking the Tree

CONNECT BY PRIOR *column1* = *column2*

Walk from the top down, using the `EMPLOYEES` table.

```
... CONNECT BY PRIOR employee id = manager id
```

Direction

Top down $\longrightarrow$ Column1 = Parent Key
 Column2 = Child Key

Bottom up $\longrightarrow$ Column1 = Child Key
Column2 = Parent Key

Walking the Tree: From the Bottom Up

```
SELECT employee_id, last_name, job_id, manager_id
FROM   employees
START WITH employee_id = 101
CONNECT BY PRIOR manager_id = employee_id ;
```

	EMPLOYEE_ID	LAST_NAME	JOB_ID	MANAGER_ID
1	101	Kochhar	AD_VP	100
2	100	King	AD_PRES	(null)

Walking the Tree: From the Top Down

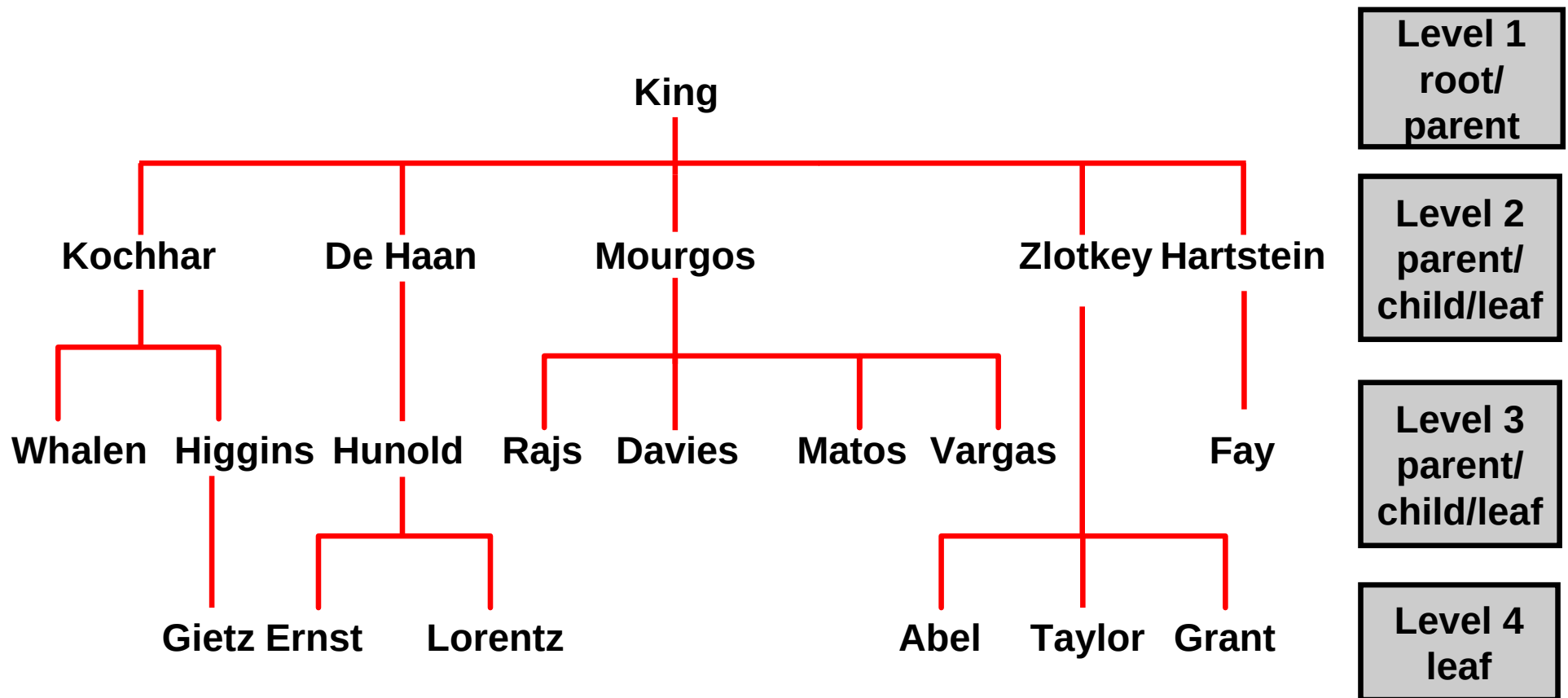
```
SELECT last_name || ' reports to ' ||  
PRIOR last_name "Walk Top Down"  
FROM employees  
START WITH last_name = 'King'  
CONNECT BY PRIOR employee_id = manager_id ;
```

Walk Top Down	
1	King reports to
2	Kochhar reports to King
3	Whalen reports to Kochhar
4	Higgins reports to Kochhar

...

18	Grant reports to Zlotkey
19	Hartstein reports to King
20	Fay reports to Hartstein

Ranking Rows with the `LEVEL` Pseudocolumn



Formatting Hierarchical Reports Using LEVEL and LPAD

Create a report displaying company management levels, beginning with the highest level and indenting each of the following levels.

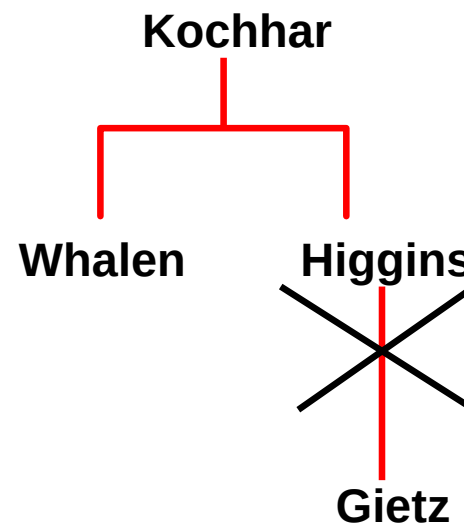
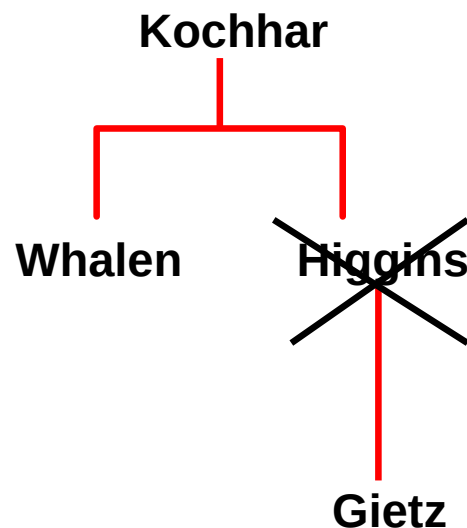
```
COLUMN org_chart FORMAT A12
SELECT LPAD(last_name, LENGTH(last_name) + (LEVEL*2) - 2, '_')
        AS org_chart
FROM    employees
START WITH first_name='Steven' AND last_name='King'
CONNECT BY PRIOR employee_id=manager_id
```


Pruning Branches

Use the **WHERE** clause
to eliminate a node.

Use the **CONNECT BY** clause
to eliminate a branch.

```
WHERE last_name != 'Higgins' CONNECT BY PRIOR  
employee_id = manager_id  
AND last_name != 'Higgins'
```



Summary

In this lesson, you should have learned that:

- You can use hierarchical queries to view a hierarchical relationship between rows in a table
- You specify the direction and starting point of the query
- You can eliminate nodes or branches by pruning



Writing Advanced Scripts

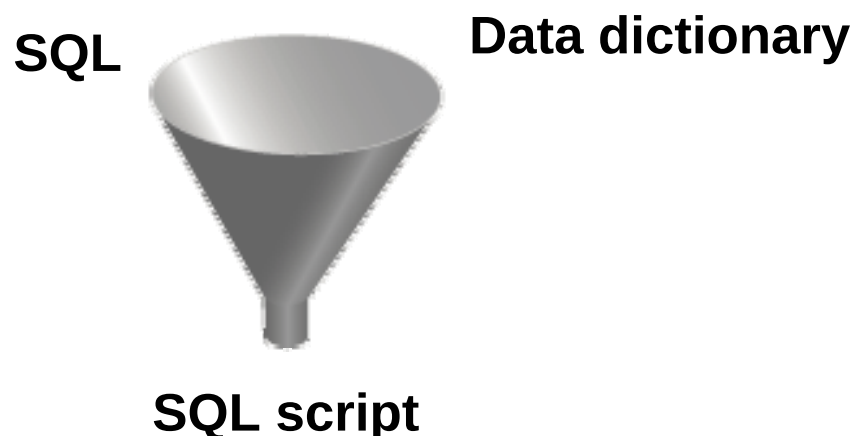
Objectives

After completing this appendix, you should be able to do the following:

- Describe the type of problems that are solved by using SQL to generate SQL
- Write a script that generates a script of `DROP TABLE` statements
- Write a script that generates a script of `INSERT INTO` statements

Using SQL to Generate SQL

- SQL can be used to generate scripts in SQL.
- The data dictionary is:
 - A collection of tables and views that contain database information
 - Created and maintained by the Oracle server



Creating a Basic Script

```
SELECT 'CREATE TABLE ' || table_name ||  
      '_test ' || 'AS SELECT * FROM '  
      || table_name || ' WHERE 1=2;'  
      AS "Create Table Script"  
FROM   user tables;
```

	Create Table Script
1	CREATE TABLE REGIONS_test AS SELECT * FROM REGIONS WHERE 1=2;
2	CREATE TABLE LOCATIONS_test AS SELECT * FROM LOCATIONS WHERE 1=2;
3	CREATE TABLE DEPARTMENTS_test AS SELECT * FROM DEPARTMENTS WHERE 1=2;
4	CREATE TABLE JOBS_test AS SELECT * FROM JOBS WHERE 1=2;
5	CREATE TABLE EMPLOYEES_test AS SELECT * FROM EMPLOYEES WHERE 1=2;
6	CREATE TABLE JOB_HISTORY_test AS SELECT * FROM JOB_HISTORY WHERE 1=2;
7	CREATE TABLE JOB_GRADES_test AS SELECT * FROM JOB_GRADES WHERE 1=2;
8	CREATE TABLE COUNTRIES_test AS SELECT * FROM COUNTRIES WHERE 1=2;

Controlling the Environment

```
SET ECHO OFF  
SET FEEDBACK OFF  
SET PAGESIZE 0
```

Set system variables
to appropriate values.

SQL statement

```
SET FEEDBACK ON  
SET PAGESIZE 24  
SET ECHO ON
```

Set system variables
back to the default
value.

The Complete Picture

```
SET ECHO OFF
SET FEEDBACK OFF
SET PAGESIZE 0

SELECT 'DROP TABLE ' || object_name || ';'
FROM   user_objects
WHERE  object_type = 'TABLE'
/

SET FEEDBACK ON
SET PAGESIZE 24
SET ECHO ON
```


Dumping the Contents of a Table to a File

```
SET HEADING OFF ECHO OFF FEEDBACK OFF
SET PAGESIZE 0

SELECT
  'INSERT INTO departments_test VALUES
    (' || department_id || ', ''' || department_name ||
    ''', ''' || location_id || ''');'
  AS "Insert Statements Script"
FROM   departments
/

SET PAGESIZE 24
SET HEADING ON ECHO ON FEEDBACK ON
```

Dumping the Contents of a Table to a File

Source	Result
<pre>' ' 'X' ' '</pre>	<pre>'X'</pre>
<pre>' ' ' ' '</pre>	<pre>' '</pre>
<pre>' ' ' department_name ' ' ' '</pre>	<pre>'Administration'</pre>
<pre>' ' ', ' ' ' '</pre>	<pre>' , '</pre>
<pre>' ' ') ; '</pre>	<pre>') ;'</pre>

Generating a Dynamic Predicate

```
COLUMN my_col NEW_VALUE dyn_where_clause

SELECT DECODE('&&deptno', null,
DECODE ('&&hiredate', null, ' ',
'WHERE hire_date=TO_DATE('' || '&&hiredate'', ''DD-MON-YYYY'')'),
DECODE ('&&hiredate', null,
'WHERE department_id = ' || '&&deptno',
'WHERE department_id = ' || '&&deptno' ||
' AND hire_date = TO_DATE('' || '&&hiredate'', ''DD-MON-YYYY'')'))
AS my_col FROM dual;
```

```
SELECT last_name FROM employees &dyn_where_clause;
```

Summary

In this appendix, you should have learned that:

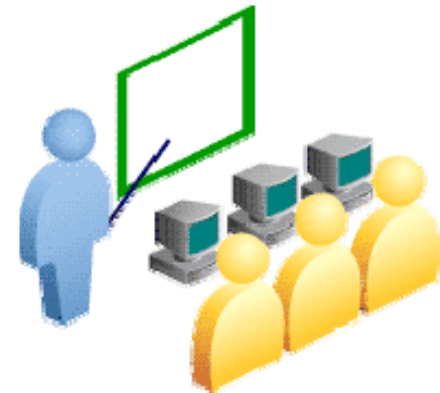
- You can write a SQL script to generate another SQL script
- Script files often use the data dictionary
- You can capture the output in a file

Oracle Database Architectural Components

Objectives

After completing this lesson, you should be able to do the following:

- List the major database architectural components
- Describe the background processes
- Explain the memory structures
- Correlate the logical and physical storage structures

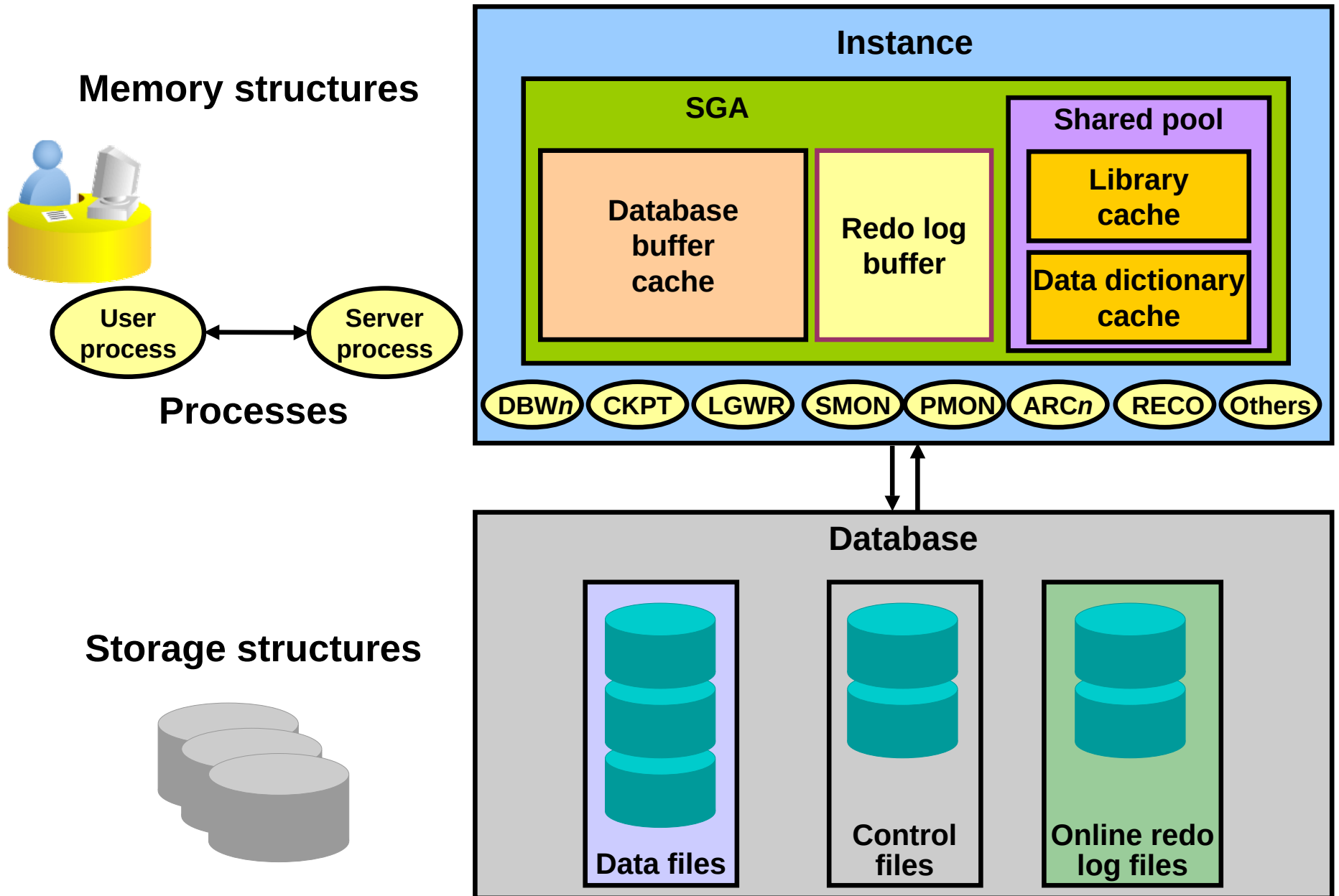


Oracle Database Architecture: Overview

The Oracle Relational Database Management System (RDBMS) is a database management system that provides an open, comprehensive, integrated approach to information management.

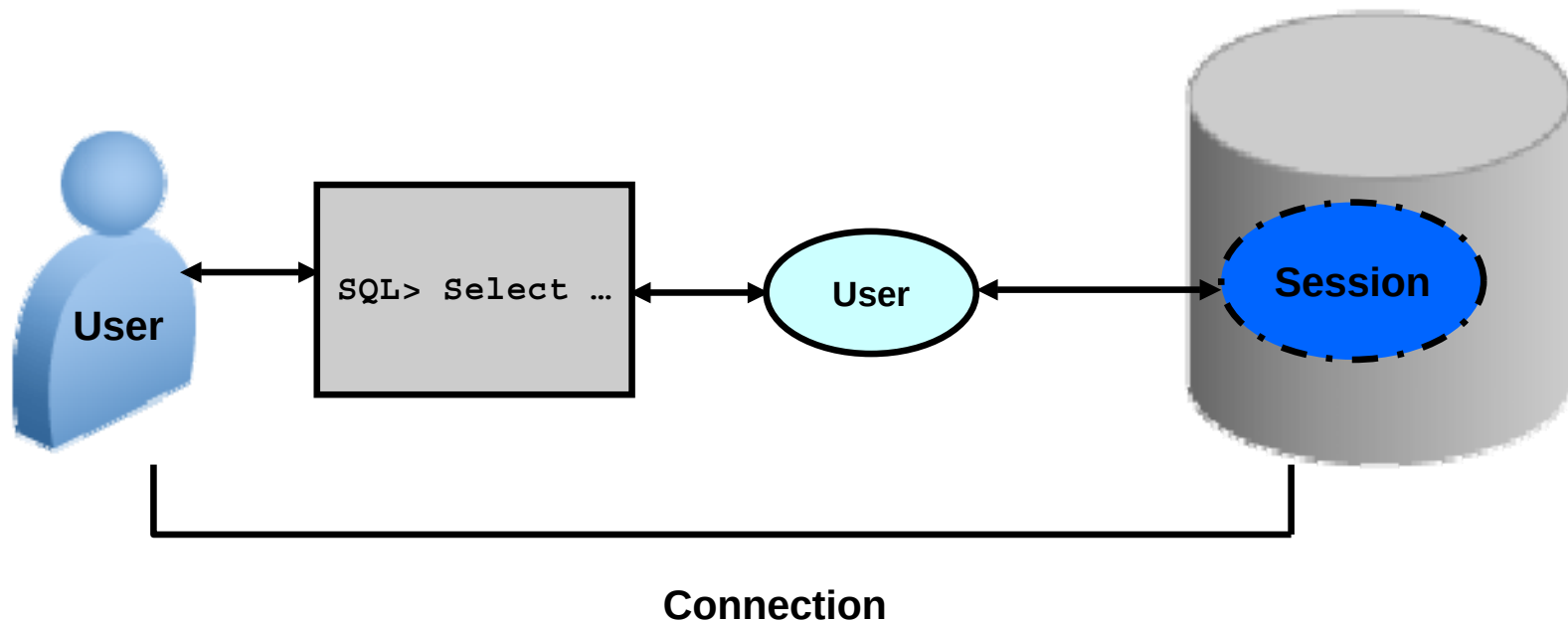


Oracle Database Server Structures

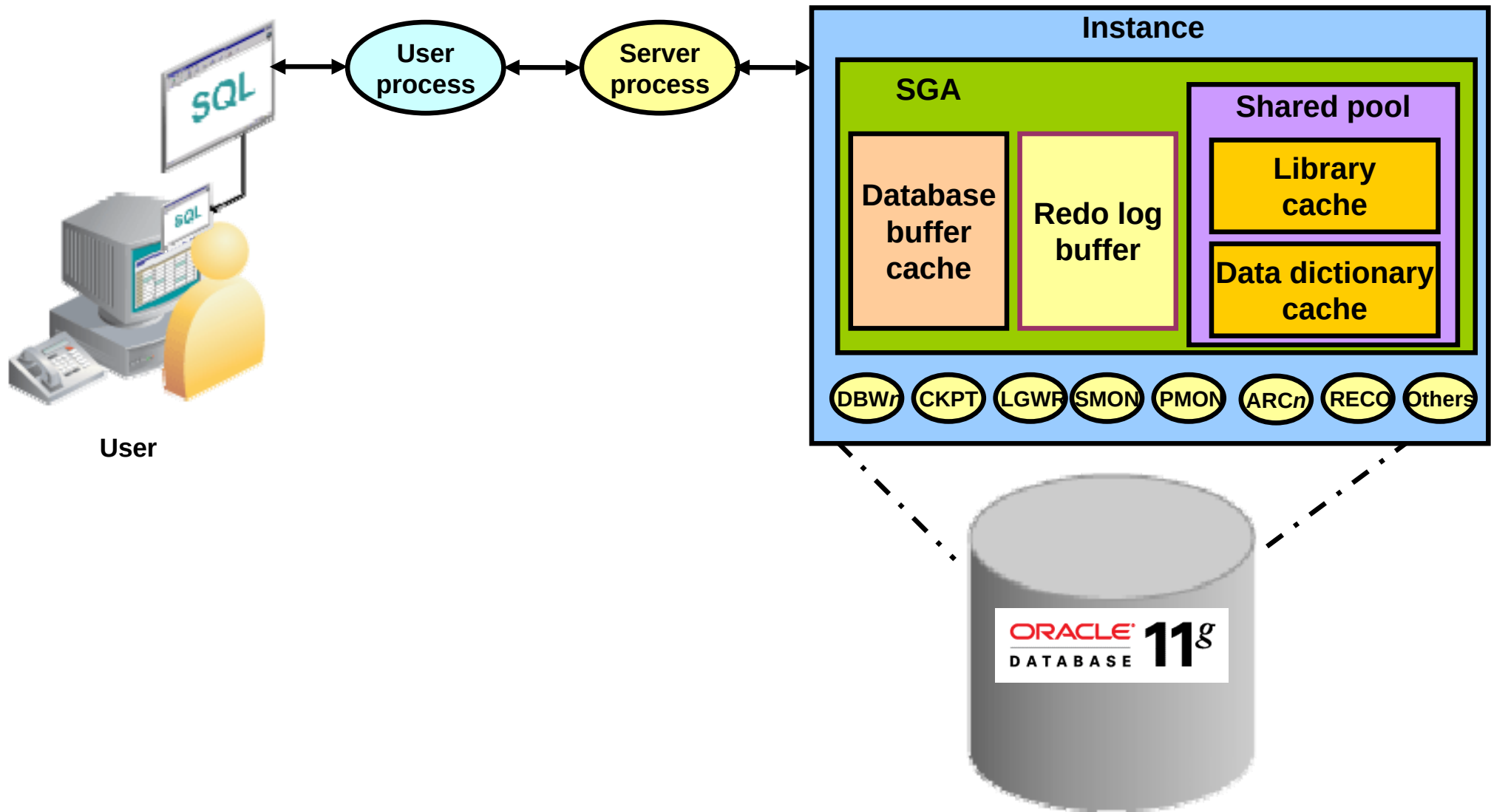


Connecting to the Database

- Connection: Communication pathway between a user process and a database instance
- Session: A specific connection of a user to a database instance through a user process



Interacting with an Oracle Database



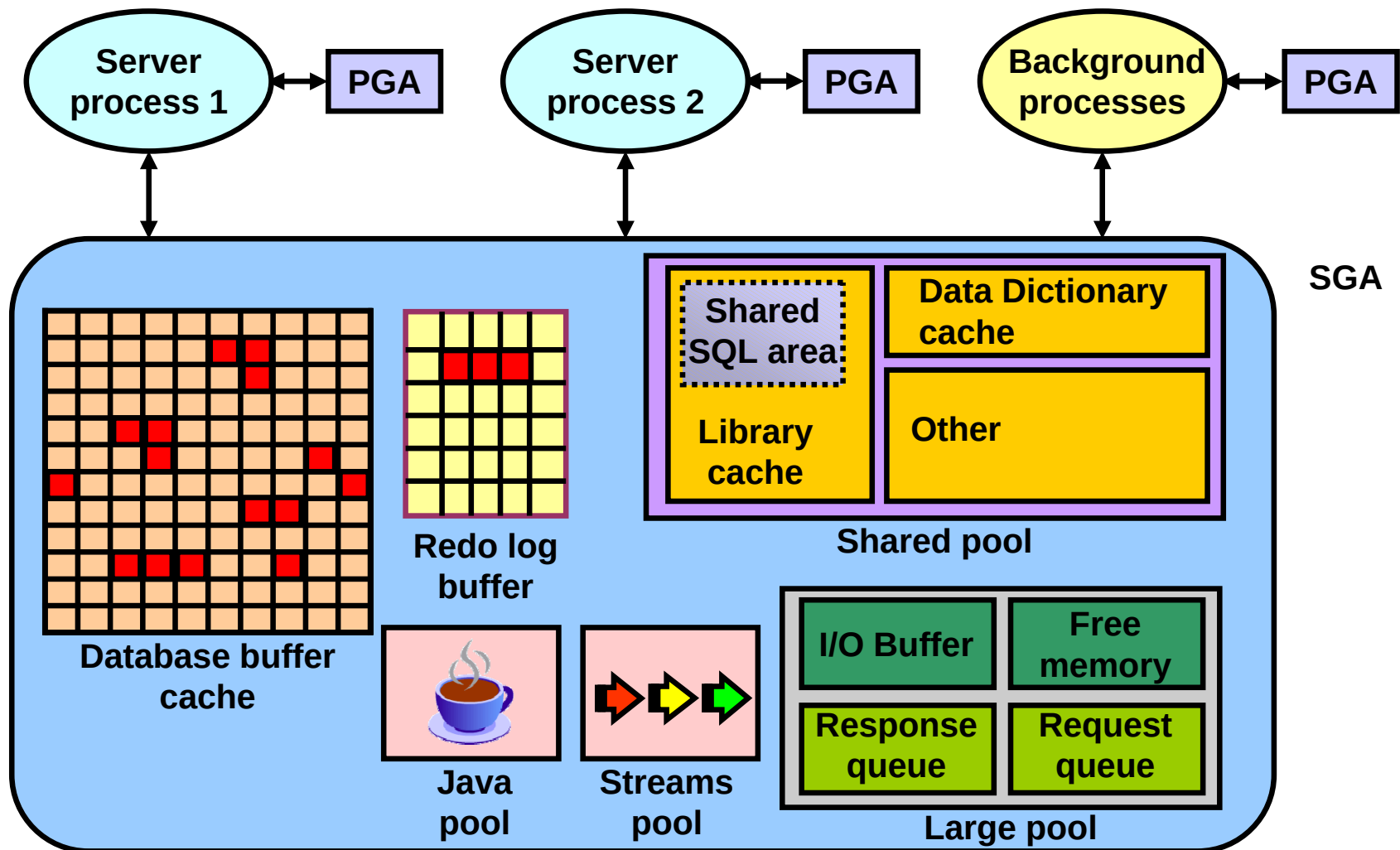
Oracle Memory Architecture

DB structures

→ Memory

- Process

- Storage

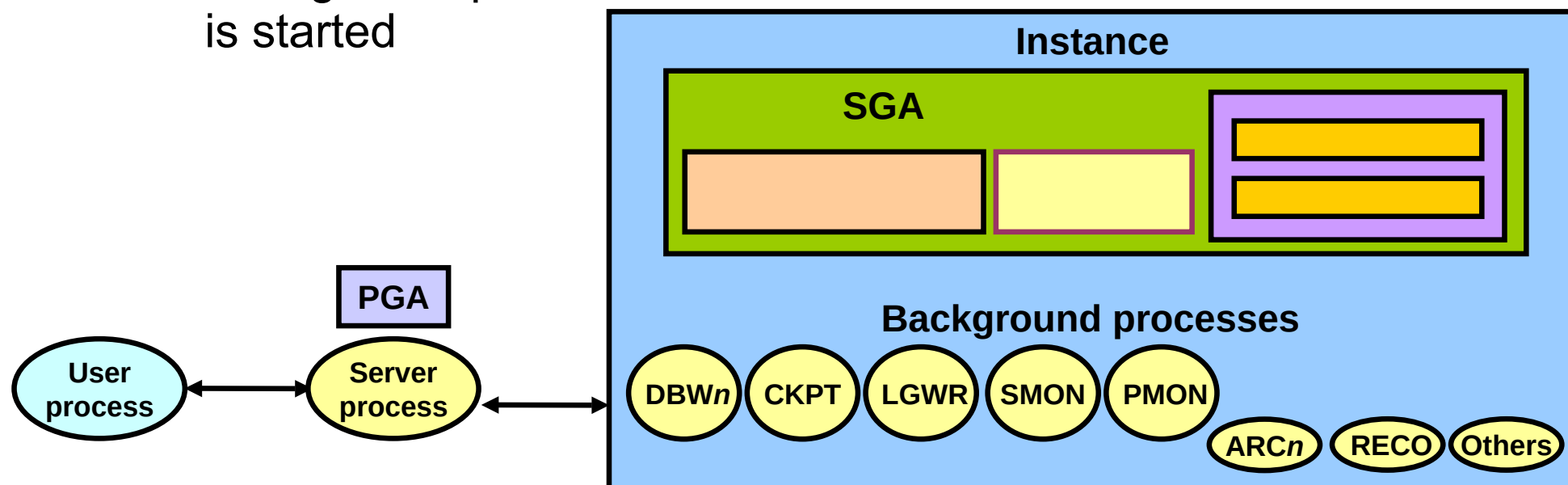


Process Architecture

DB structures

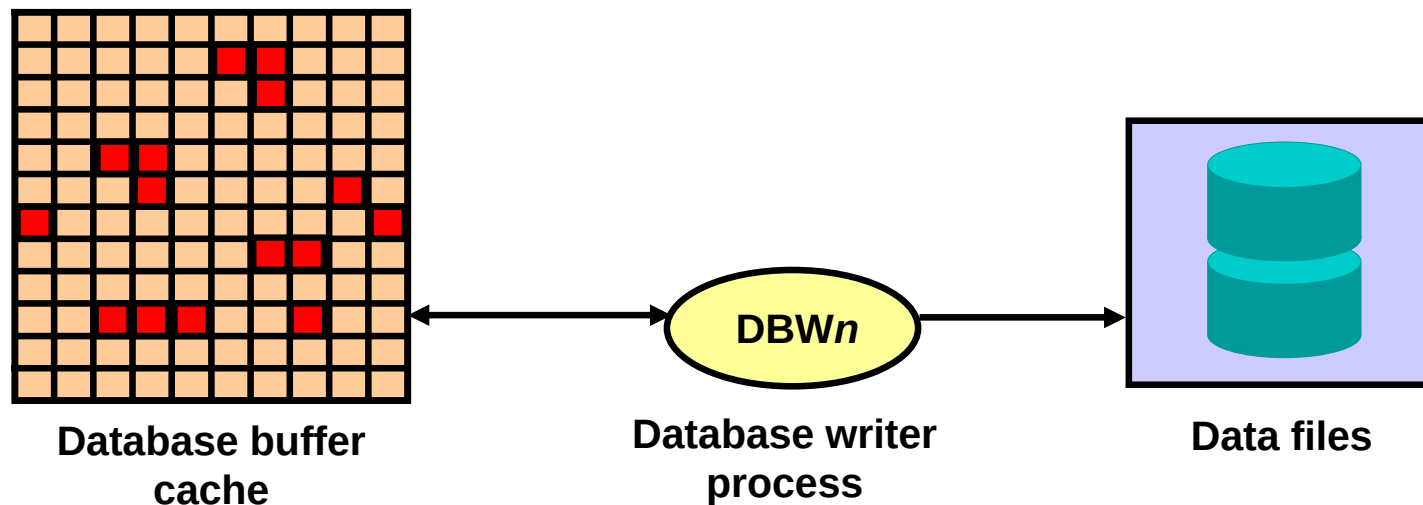
- Memory
- **Process**
- Storage

- User process:
 - Is started when a database user or a batch process connects to the Oracle Database
- Database processes:
 - Server process: Connects to the Oracle instance and is started when a user establishes a session
 - Background processes: Are started when an Oracle instance is started



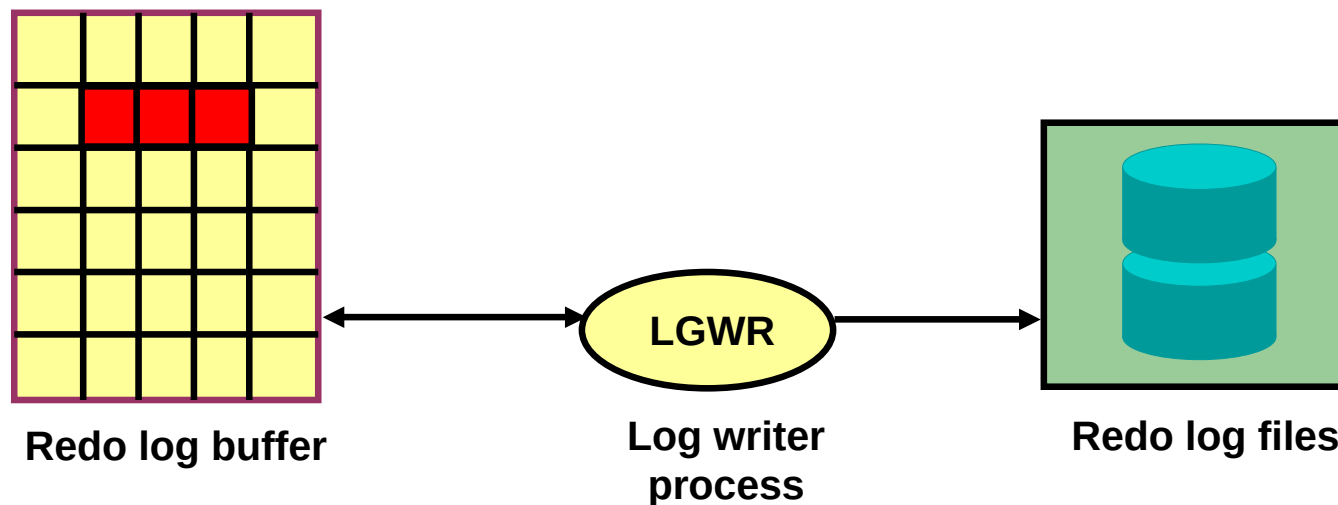
Database Writer Process

- Writes modified (dirty) buffers in the database buffer cache to disk:
 - Asynchronously while performing other processing
 - Periodically to advance the checkpoint



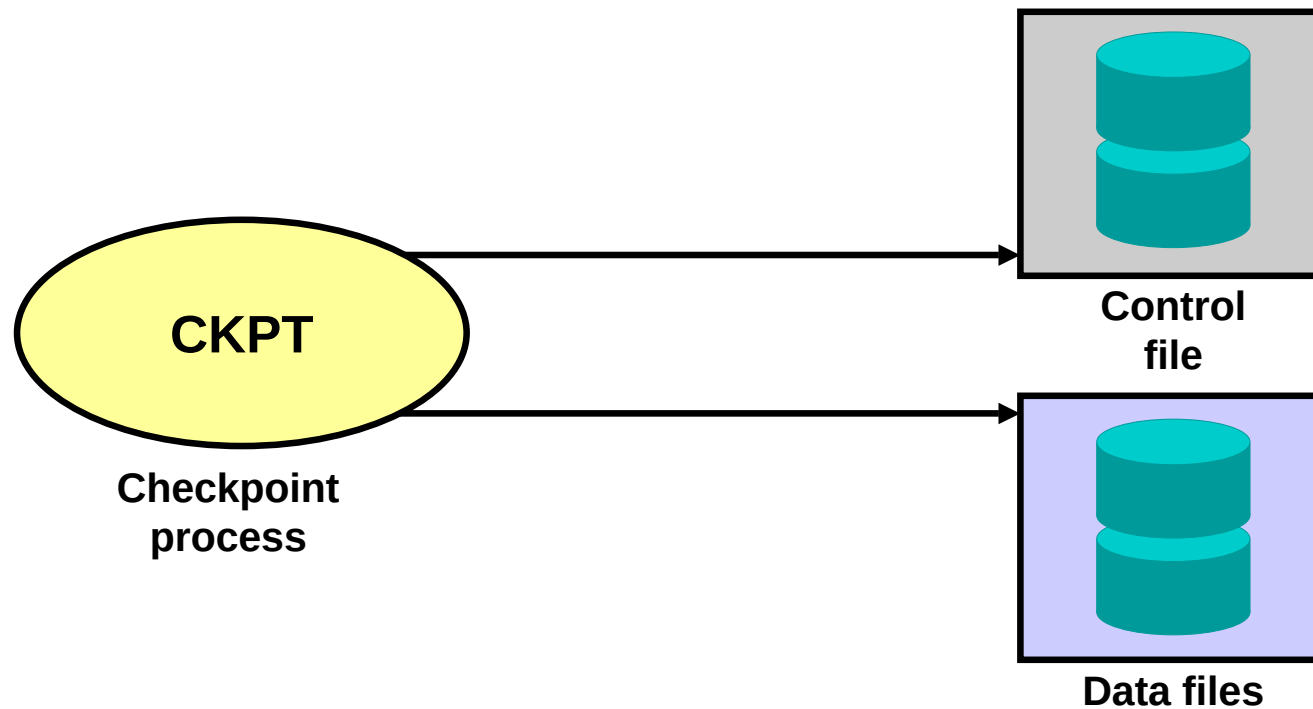
Log Writer Process

- Writes the redo log buffer to a redo log file on disk
- LGWR writes:
 - A process commits a transaction
 - When the redo log buffer is one-third full
 - Before a DBWn process writes modified buffers to disk



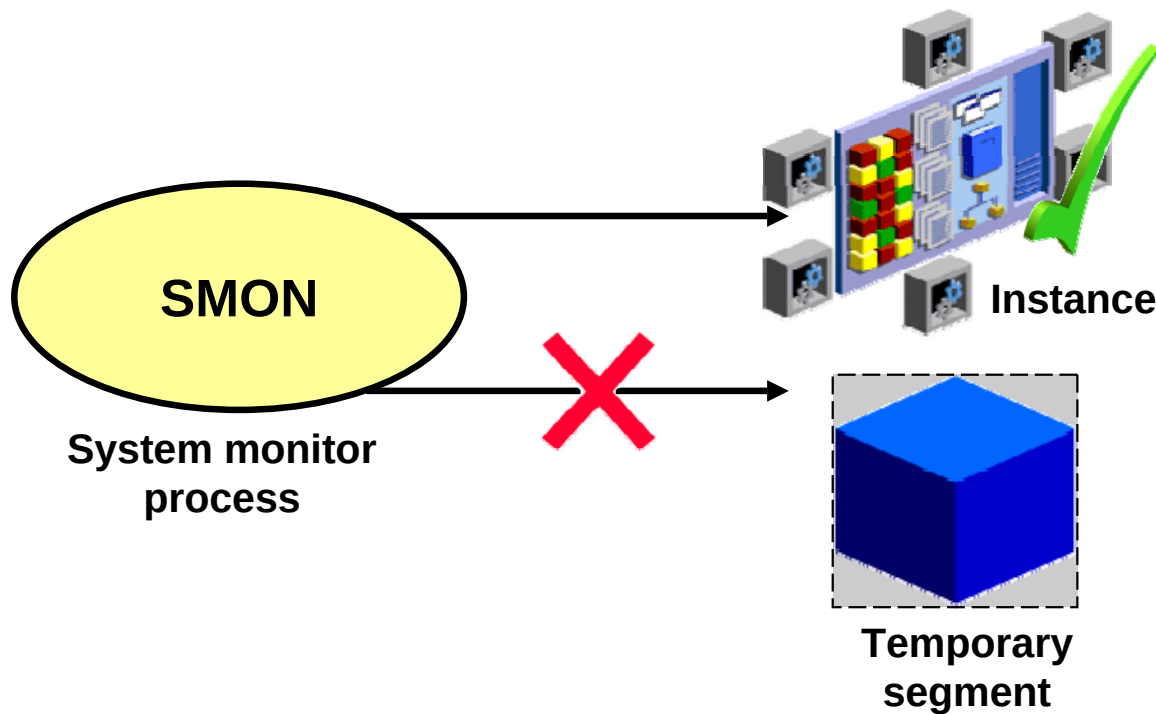
Checkpoint Process

- Records checkpoint information in:
 - The control file
 - Each datafile header



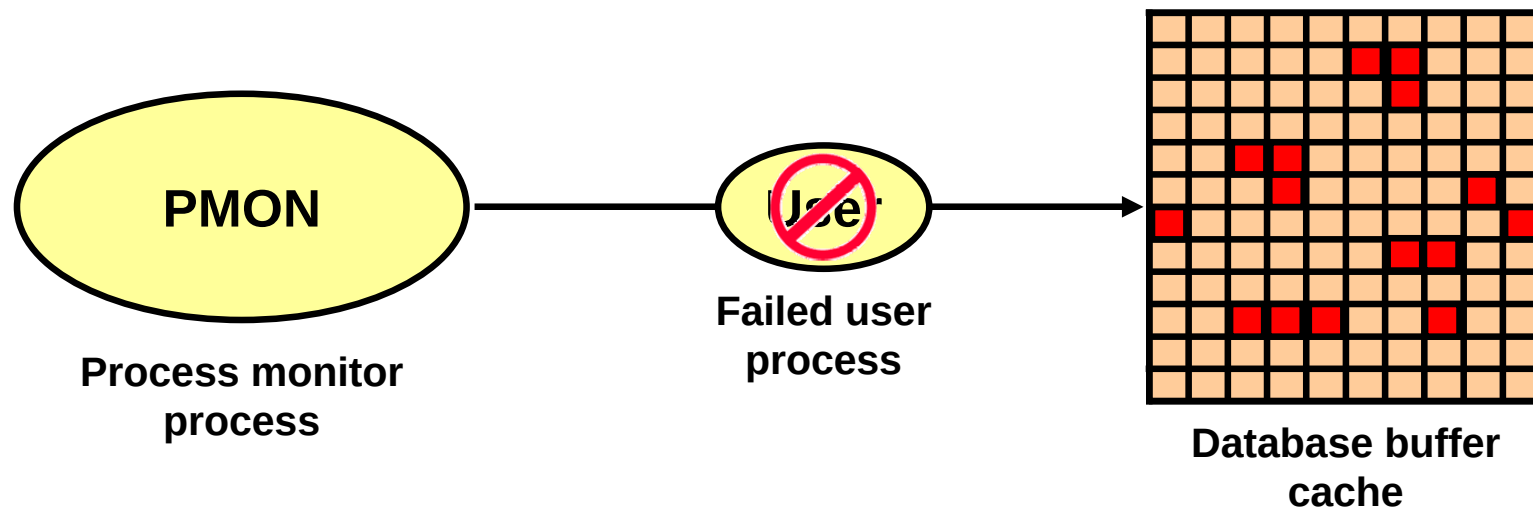
System Monitor Process

- Performs recovery at instance startup
- Cleans up unused temporary segments



Process Monitor Process

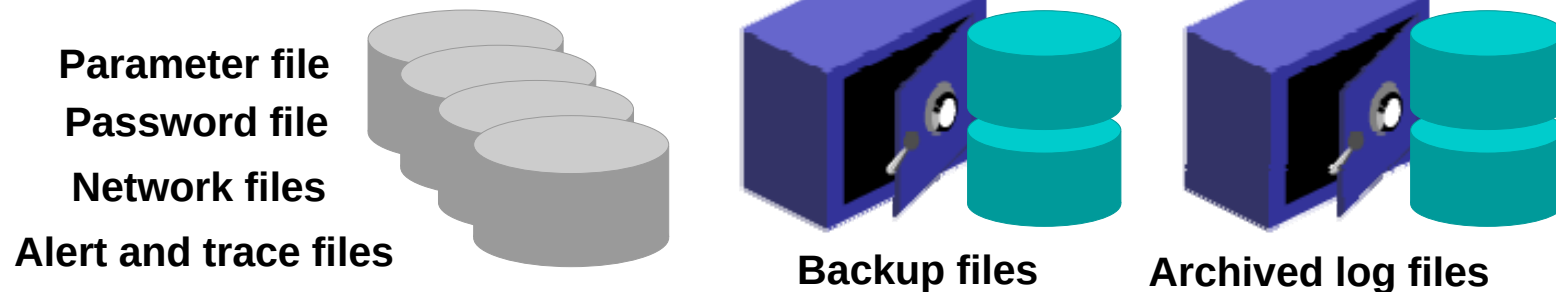
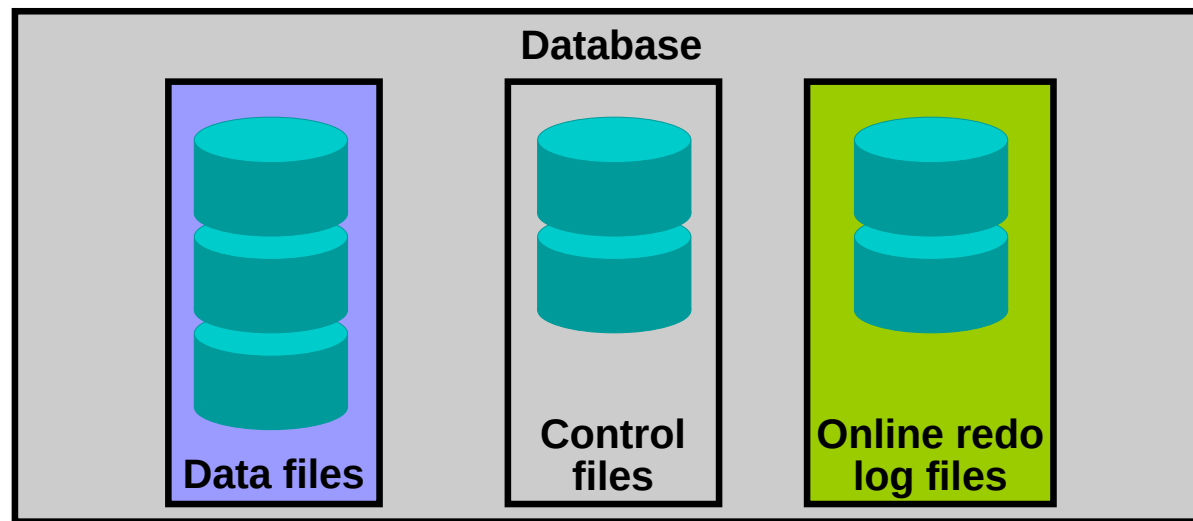
- Performs process recovery when a user process fails
 - Cleans up the database buffer cache
 - Frees resources used by the user process
- Monitors sessions for idle session timeout
- Dynamically registers database services with listeners



Oracle Database Storage Architecture

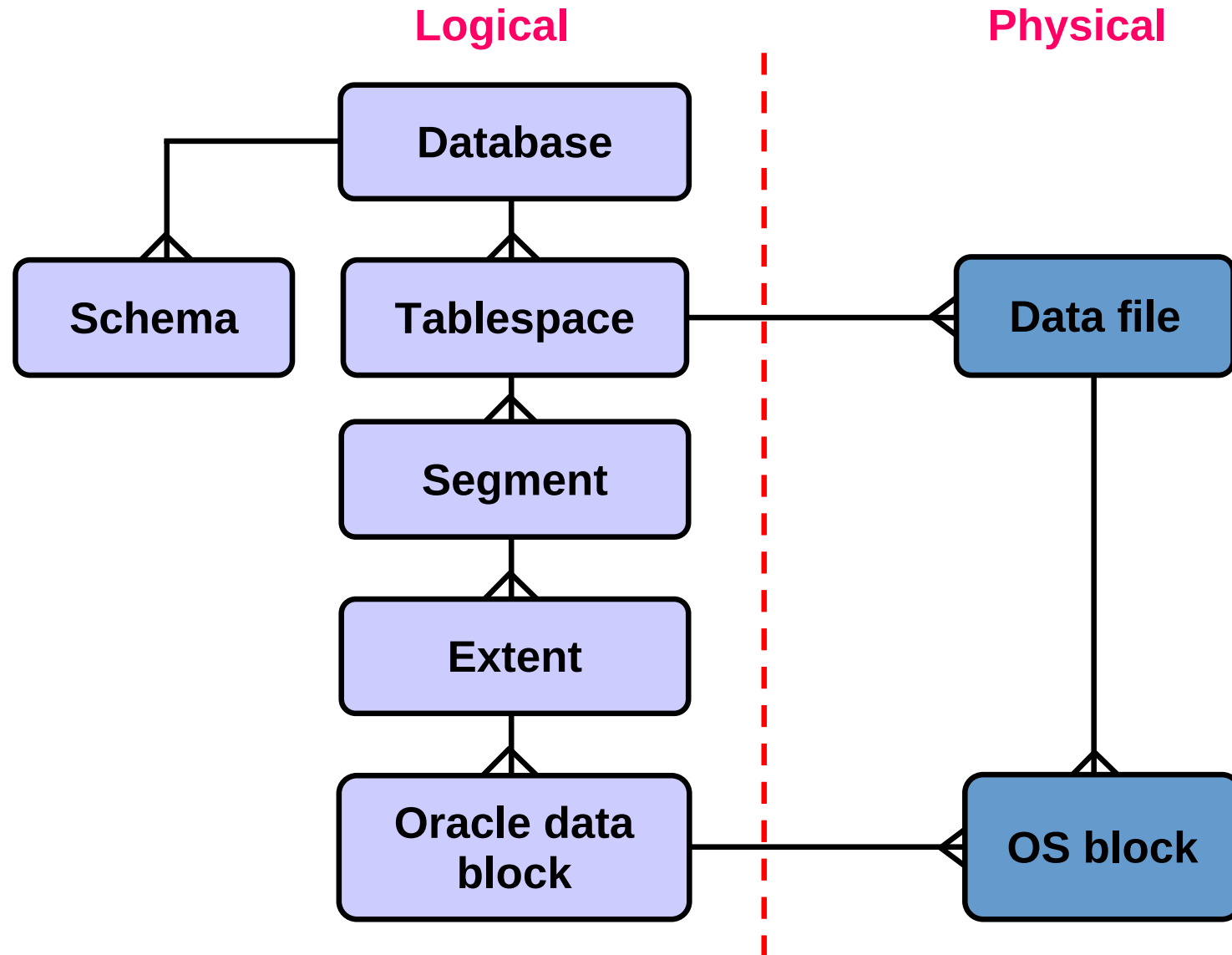
DB structures

- Memory
- Process
- **Storage**



ORACLE

Logical and Physical Database Structures



Processing a SQL Statement

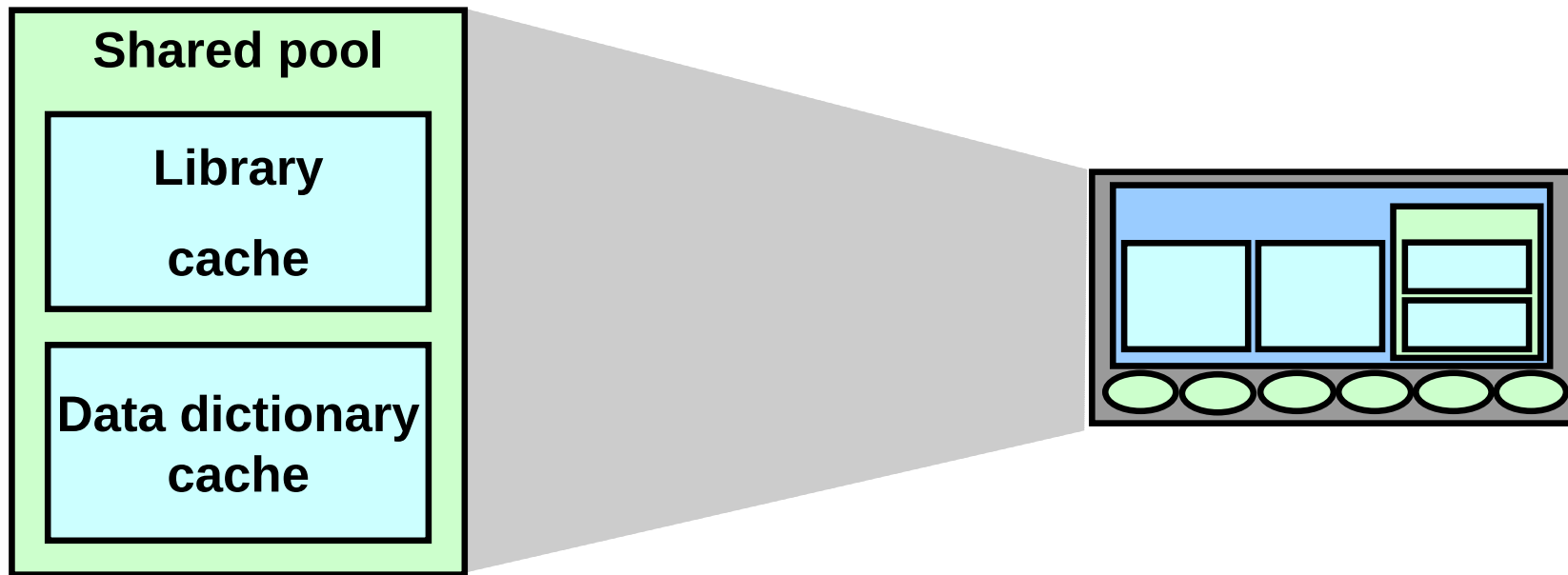
- Connect to an instance using:
 - The user process
 - The server process
- The Oracle server components that are used depend on the type of SQL statement:
 - Queries return rows.
 - Data manipulation language (DML) statements log changes.
 - Commit ensures transaction recovery.
- Some Oracle server components do not participate in SQL statement processing.

Processing a Query

- Parse:
 - Search for an identical statement.
 - Check the syntax, object names, and privileges.
 - Lock the objects used during parse.
 - Create and store the execution plan.
- Execute: Identify the rows selected.
- Fetch: Return the rows to the user process.

Shared Pool

- The library cache contains the SQL statement text, parsed code, and execution plan.
- The data dictionary cache contains table, column, and other object definitions and privileges.
- The shared pool is sized by `SHARED_POOL_SIZE`.



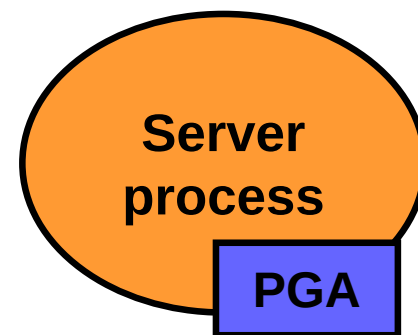
Database Buffer Cache

- The database buffer cache stores the most recently used blocks.
- The size of a buffer is based on `DB_BLOCK_SIZE`.
- The number of buffers is defined by `DB_BLOCK_BUFFERS`.

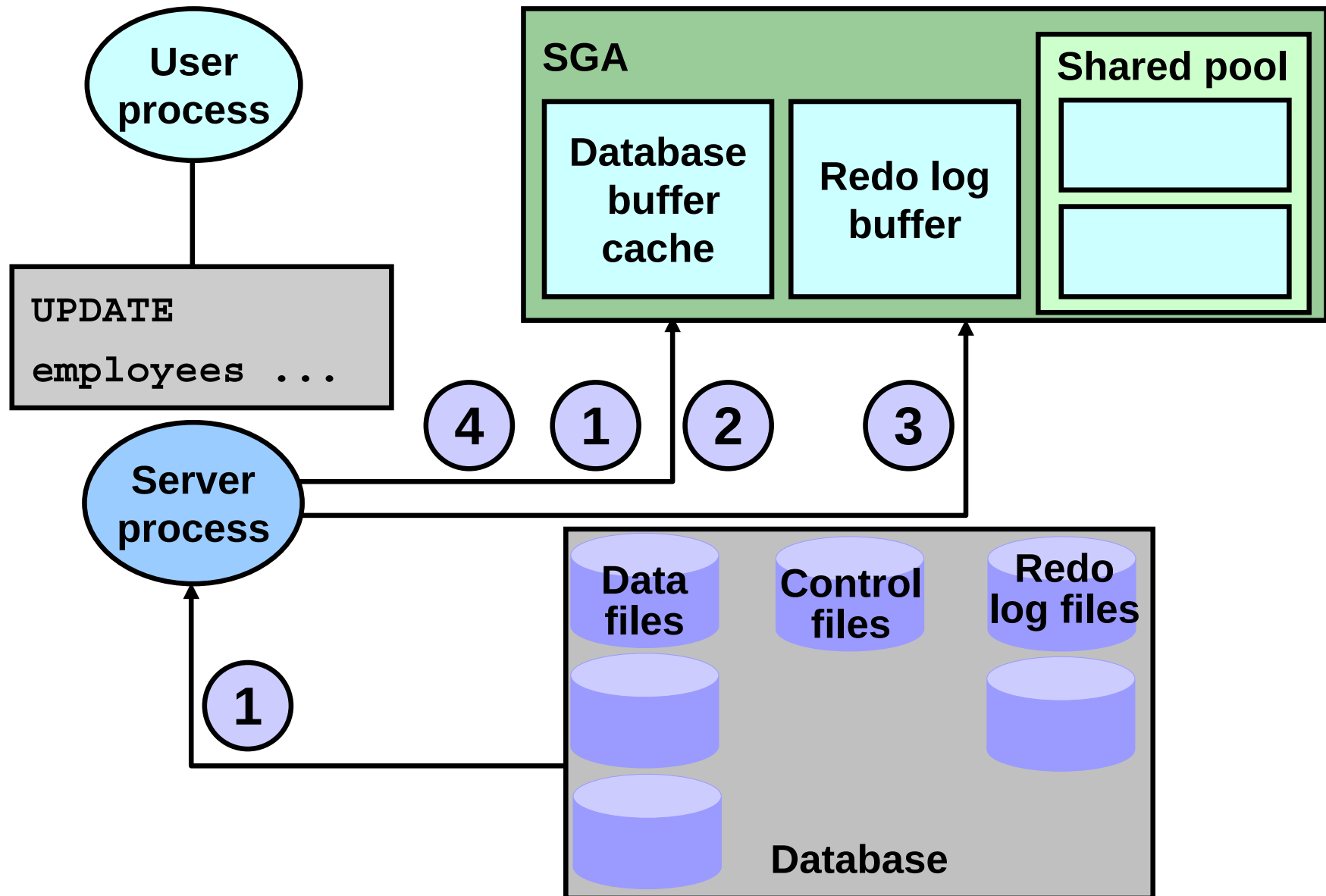


Program Global Area (PGA)

- Is not shared
- Is writable only by the server process
- Contains:
 - Sort area
 - Session information
 - Cursor state
 - Stack space



Processing a DML Statement

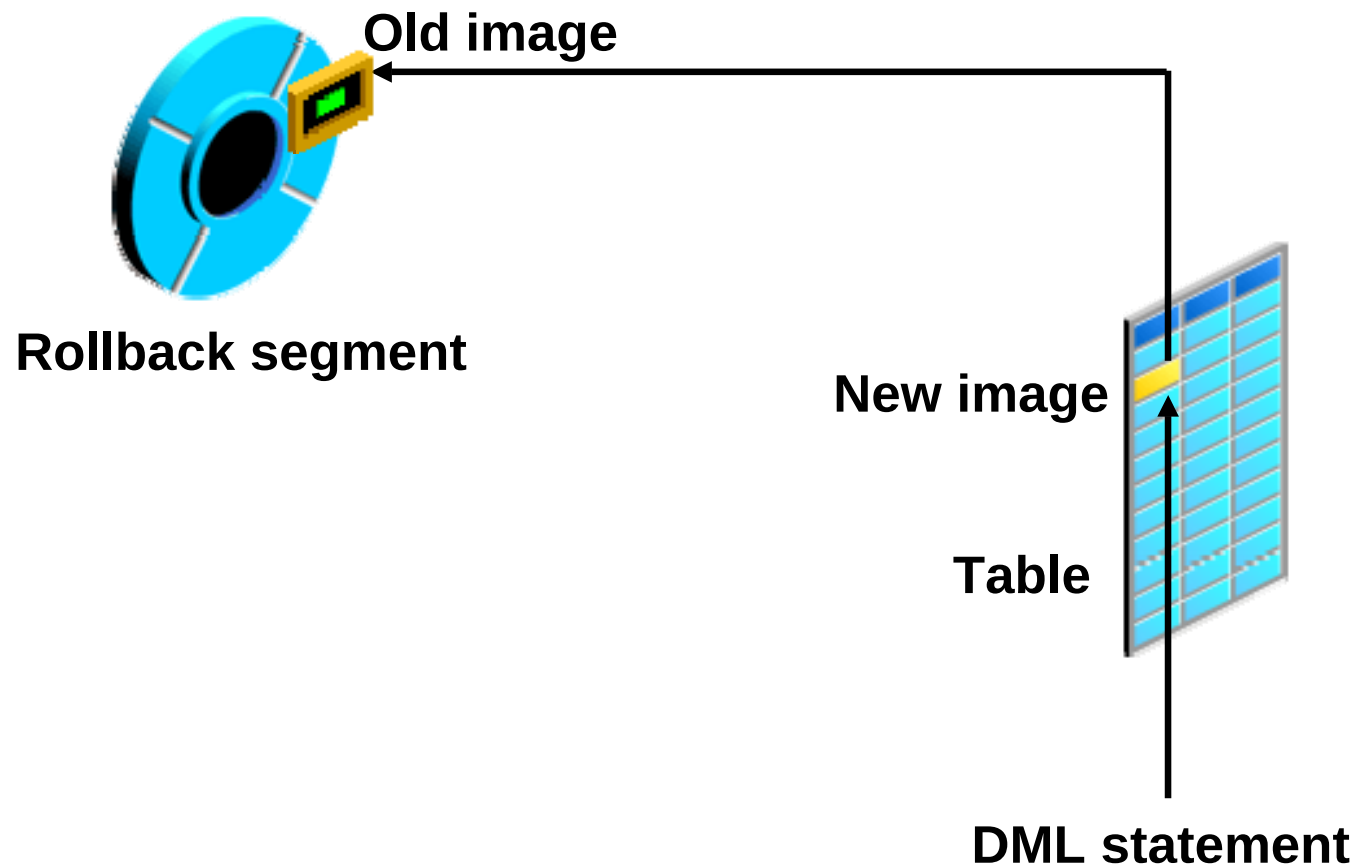


Redo Log Buffer

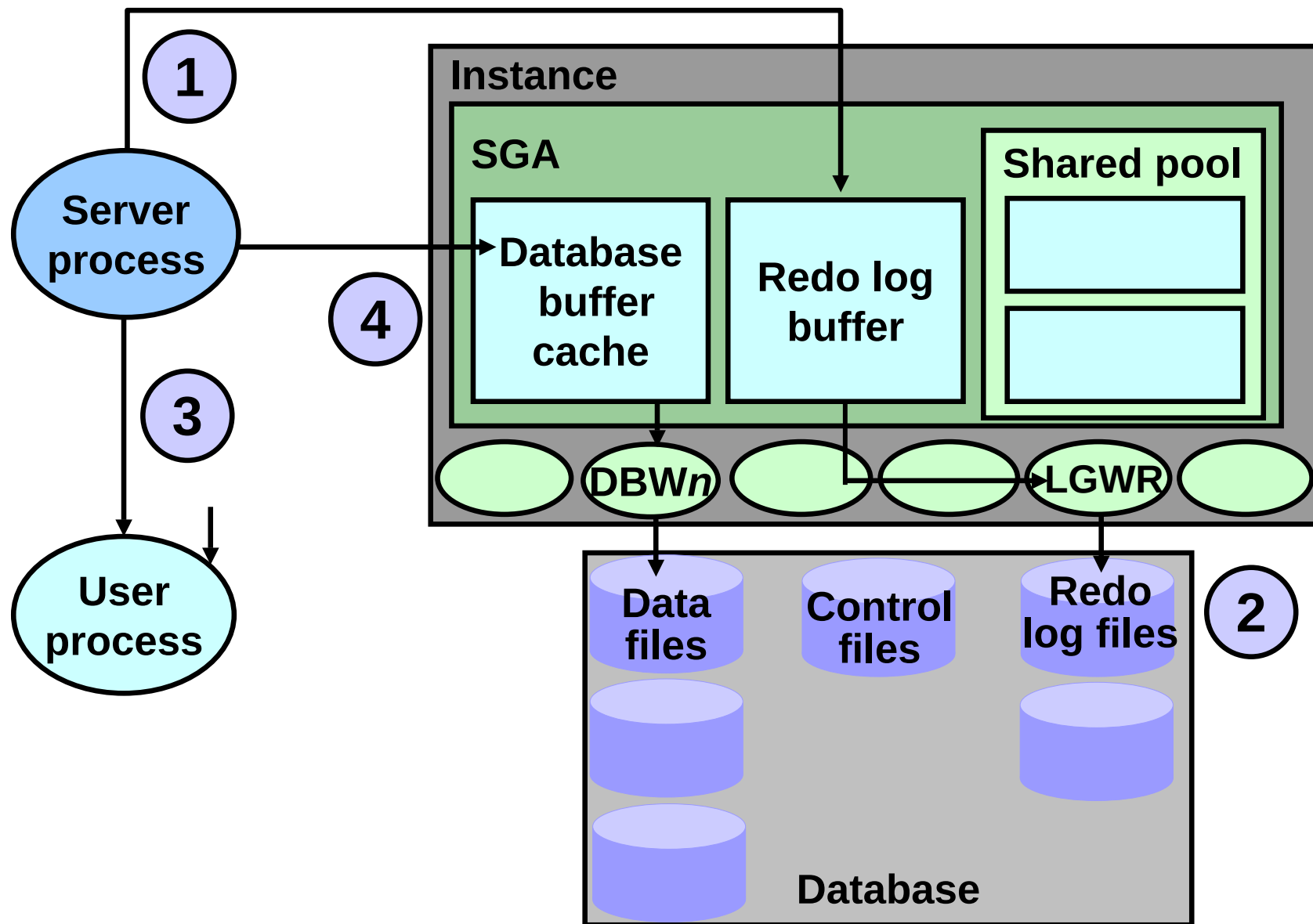
- Has its size defined by `LOG_BUFFER`
- Records changes made through the instance
- Is used sequentially
- Is a circular buffer



Rollback Segment



COMMIT Processing



Summary of the Oracle Database Architecture

